SAMSUNG

Samsung Innovation Campus

Curso de Inteligencia Artificial

Together for Tomorrow!
Enabling People
Education for Future Generations

Capítulo 3.

Análisis de datos exploratorios: NumPy Arrays para computación numérica optimizada y Pandas

Curso de IA

Unidad 2.

Exploración óptima de data a través de pandas

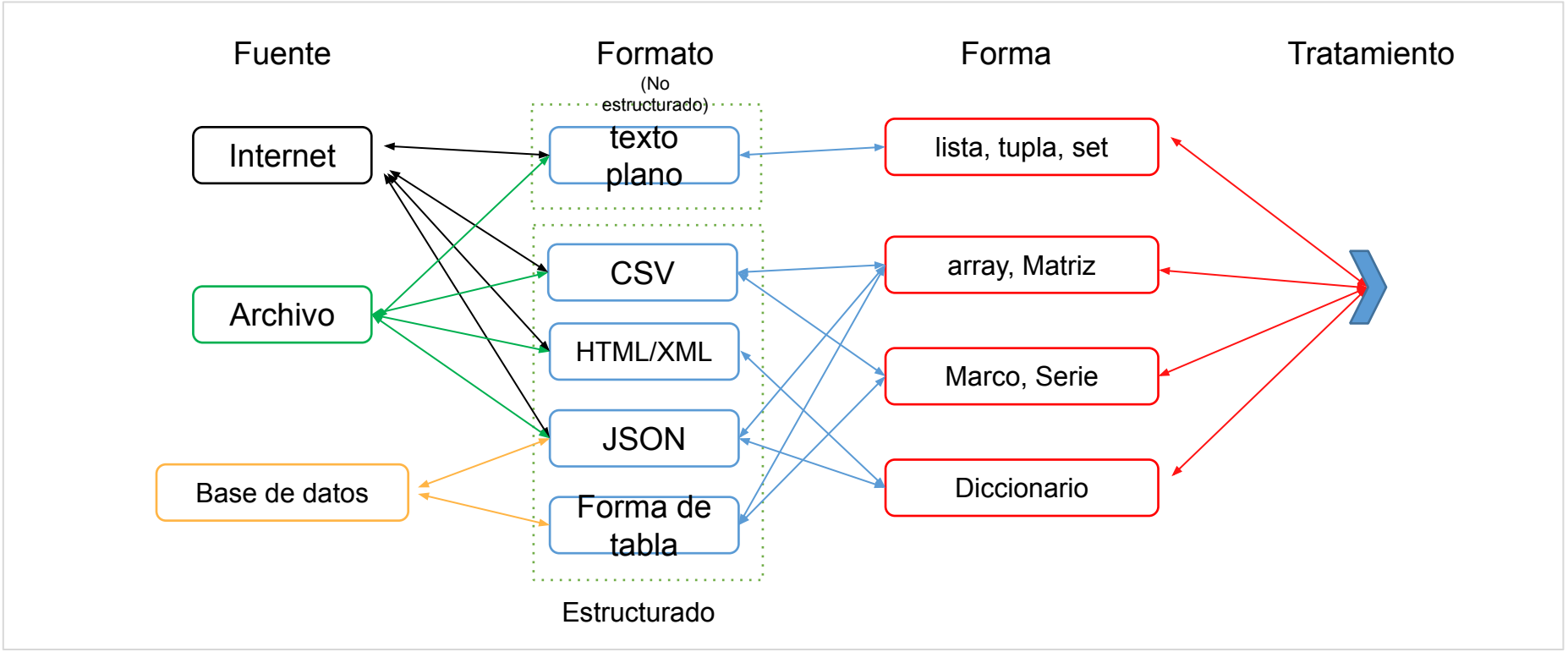
- 2.1. Pipelines: Estructuras data Según Tipos data
- 2.2. Serie pandas y marcos de datos
- 2.3. Fusionar y vincular marcos de datos
- 2.4. DataFrame Clasificación e índice múltiple
- 2.5. Examinar las características de data mediante estadísticas descriptivas y muestras de data

Pipelines

- Perspectiva estructural de los tipos data
  - data recopilado, from tiene una perspectiva estructural (estructura de esquema o computabilidad) y se puede dividir en tres categorías: data estructurado, no estructurado y semiestructurado.
    - data estructurado se refiere a data que tiene una forma basada en estructura del esquema estructurado (formulario) y se almacena en campos fijos como RDB y hoja de cálculo y es consistente en valor y formato.
    - data no estructurado se refiere a data que no tiene una estructura de esquema y no se almacena en campos fijos como redes sociales, tableros de anuncios web y NoSQL.
    - data semiestructurado se refiere a data que tiene una estructura de esquema (formal) y contiene metadatos como XML, HTML, blogs, registros del sistema y alarmas, y es inconsistente en valor y formato.

2.1. Pipelines: Estructuras data Según Tipos data

- Python Data Structure Pipeline
  - Data recopilada de varias fuentes tienen diversas formas y propiedades, como se muestra en la siguiente figura. Por tanto, para el análisis, es necesario integrar varios tipos de data en el mismo formato para que los ordenadores puedan entenderlos.



Unidad 2.

Exploración óptima de data a través de pandas

- 2.1. Pipelines: Estructuras data Según Tipos data
- 2.2. Serie pandas y marcos de datos
- 2.3. Fusionar y vincular marcos de datos
- 2.4. DataFrame Clasificación e índice múltiple
- 2.5. Examinar las características de data mediante estadísticas descriptivas y muestras de data

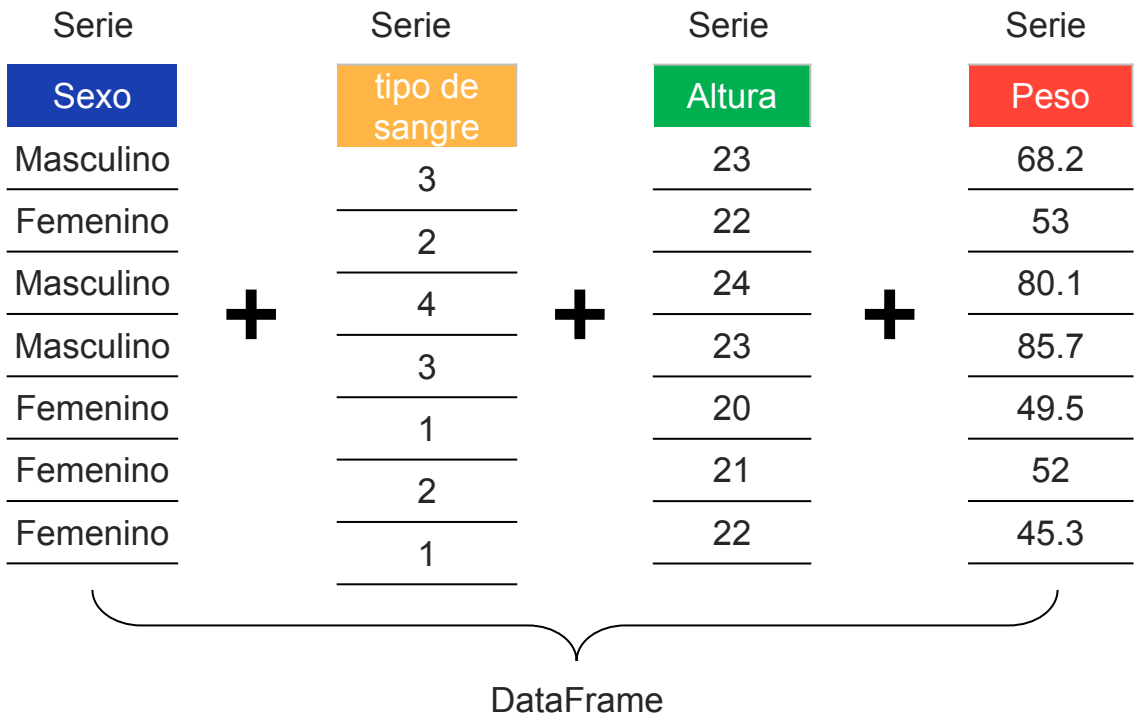
pandas Esquema

- Una tabla es la forma más óptima de data que una persona puede entender. Por lo tanto, la capacidad de manejar bien el data tabular es la base del análisis. En python, el formulario de tabla se denomina DataFrame y se implementa como una biblioteca pandas.
  - La biblioteca pandas está construida sobre la base de numpy pero se especializa en análisis data más complejos.
  - Mientras que numpy procesa solo el mismo array de los tipos data, pandas puede procesar diferentes tipos de data.
  - La biblioteca pandas es una herramienta óptima para recopilar y organizar data.
  - pandas es una herramienta importante que puede manejar la mayor parte del trabajo científico de data y es esencial para los científicos de data.

pandas para análisis data

- Biblioteca esencial para el análisis data
  - Representación efectiva basada en estadísticas y resumen de recopilación data
  - Alineación y manipulación de data para fusionar y tejer varios tipos de data
  - El procesamiento data, como la recopilación, la transformación y la aplicación de funciones, se puede aplicar a todo el paquete data.
  - Dado que numpy es una biblioteca aritmética general basada en procesamiento data, pandas debe usarse para procesar data tabular para estadísticas o análisis. pandas también proporciona capacidades de procesamiento de series temporales de las que carece numpy.

- Acerca de la estructura pandas data:
  - Para conocer pandas, es importante familiarizarse con las estructuras Series y DataFrame data.



  - Las estructuras básicas data de pandas incluyen Serie (1D) y DataFrame (2D). DataFrame es un contenedor para Series y Series es un contenedor para escalar (0D). Pueden agregar o eliminar data en forma de diccionario.

Serie

- Una serie es una forma de matrices unidimensionales enumeradas secuencialmente.
  - Como es un array, todos los elementos de la serie deben pertenecer a un data type.
  - Se pueden crear series simples de secuencias como listas, tuplas y matrices.
  - La palabra "serie" es singular Y plural. Su origen latino, 'serere', significa unir o conectar.
  - El valor predeterminado de la serie es un índice entero. La etiqueta del primer elemento es 0, la etiqueta del segundo elemento es 1 y aumenta de esta manera.
  - El valor del atributo de la serie es list de todos los valores de la serie.
  - El valor del atributo del índice significa un índice de una serie.
  - El valor del atributo de index.values es un array de todos los valores de índice.

- Cómo crear una serie

```
[1]: import pandas as pd
```

  - Creando de una lista

```
[2]: pd.Series(['Male', 'Female', 'Male', 'Male', 'Female', 'Female', 'Female'])
```

```
[2]: 0      Male
     1    Female
     2      Male
     3      Male
     4    Female
     5    Female
     6    Female
     dtype: object
```

- Cómo crear una serie

```
[3]: import numpy as np
```

- Creando desde NumPy arrays. Consulte la ayuda (pd.Series) para obtener más detalles.

```
[4]: pd.Series(np.array(['Male', 'Female', 'Male', 'Male', 'Female', 'Female', 'Female']))
```

```
[4]: 0      Male
     1    Female
     2      Male
     3      Male
     4    Female
     5    Female
     6    Female
     dtype: object
```

- Cómo crear una serie
  - Las series también se pueden crear con valores similares a Array, iterables, diccionarios o valores escalares.,

```
[5]: ser=pd.Series(np.array(['Male', 'Female', 'Male', 'Male', 'Female', 'Female', 'Female']))

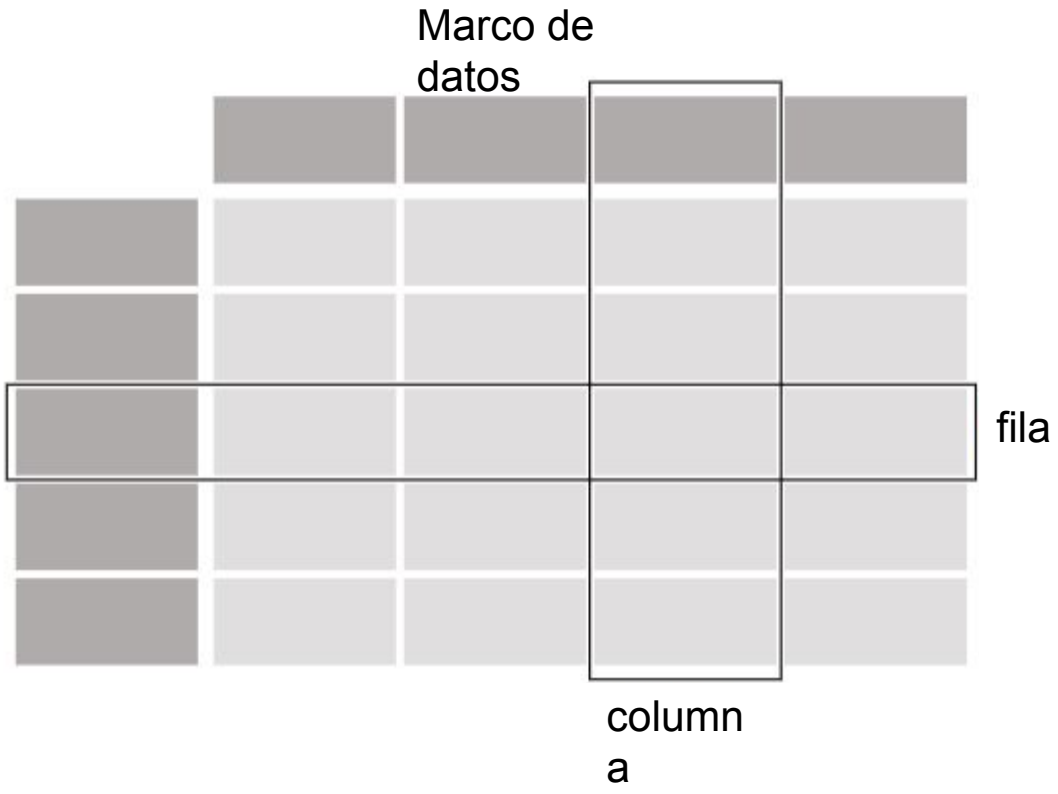
[6]: type(ser)

[6]: pandas.core.series.Series
```

Línea 6

- Podemos saber que el tipo de datos es una serie.

- Cómo crear una serie
  - Para lidiar con DataFrames en el futuro, primero debe comprender los datos de la serie y pensar en la parte de la columna como una serie en la siguiente figura.



- Crear una serie con diccionario

```
[7]: dict_data={'a': 1, 'b': 2, 'c': 3}

[8]: pd.Series(dict_data)

[8]: a    1
     b    2
     c    3
     dtype: int64

[9]: ser1 = pd.Series(dict_data)

[10]: type(ser1)

[10]: pandas.core.series.Series
```

- Crear una serie con diccionario
  - El índice arrays se puede seleccionar por separado utilizando los atributos de índice de la clase de serie.

```
[11]: ser1.index

[11]: Index(['a', 'b', 'c'], dtype='object')
```

  - También es posible seleccionar el valor de datos Array por separado. En ese momento se utiliza el atributo de valores de la clase serie.

```
[12]: ser1.values

[12]: array([1, 2, 3])
```

- Creando una serie con lista

```
[13]: list_data = ['2019-01-02', 3.14, 'ABC', 100, True]

[14]: ser2=pd.Series(list_data)

[15]: ser2

[15]: 0    2019-01-02
      1          3.14
      2           ABC
      3           100
      4          True
      dtype: object

[16]: ser2.index
```

Línea 16

- El índice está representado por un objeto RangeIndex de tipo entero en el rango de 0 y 4. En este momento, el último valor no está incluido.

- El valor de datos Array mantiene el orden del elemento de lista Array de list_data, que son los datos originales.

- Marco de datos y serie
  - Creemos un DataFrame simple y aprendamos a manipular datos en una serie. Estos datos almacenan datos de pasajeros del Titanic. Se supone que se conocen los datos del nombre del pasajero (caracter), edad (entero) y sexo (masculino/femenino).

Nombre	Edad	Sexo
Braund, Sr. Owen Harris	22	masculino
Allen, Sr.William Henry	35	masculino
Bonnell, señorita Elizabeth	58	femenino

- Marco de datos y serie
  - Se debe utilizar un marco de datos para ver la tabla de procedimientos y almacenar datos en forma de tabla. El nombre de la columna, también llamado encabezado de columna, se establece principalmente como un valor clave de un diccionario.

[17]:
```
pd.DataFrame({'Name': ['Braund, Mr, Owen Harries',
                       'Allen, Mr. William Henry',
                       'Bonnel, Miss. Elizabeth'],
              'Age': [22, 35, 58],
              'Sex': ['male', 'male', 'female']})
```

[17]:

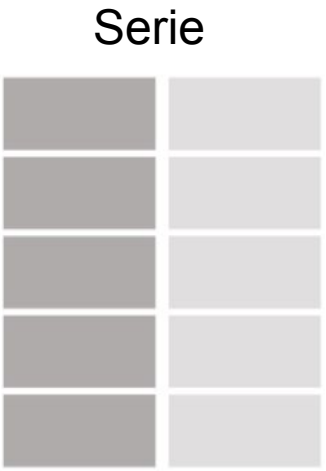
	Name	Age	Sex
0	Braund, Mr, Owen Harries	22	male
1	Allen, Mr. William Henry	35	male
2	Bonnel, Miss. Elizabeth	58	female

- Marco de datos y serie

	Name	Age	Sex
0	Braund, Mr. Owen Harris	22	male
1	Allen, Mr. William Henry	35	male
2	Bonnell, Miss. Elizabeth	58	female

- A Un DataFrame es una estructura de datos bidimensional que almacena varios tipos de datos (letras, números enteros, valores de punto flotante, datos categóricos, etc.) en una columna.
- Cada tabla tiene tres columnas con una etiqueta de columna. Las etiquetas de las columnas son "Nombre", "Edad" y "Sexo".
- La columna "Nombre" consta de datos de texto en los que cada valor es string, la columna "Edad" es un número entero y la columna "Sexo" son datos de texto.
- Es similar a la representación de una tabla de datos en una hoja de cálculo.

- La columna de un DataFrame es una serie.



- La columna de un DataFrame es una serie.
  - Si solo está interesado en la columna "Edad" y desea extraer la columna, utilice el valor de la clave del diccionario.

```
[18]: df = pd.DataFrame({'Name': ['Braund, Mr, Owen Harries',
                                  'Allen, Mr. William Henry',
                                  'Bonnel, Miss. Elizabeth'],
                        'Age': [22, 35, 58],
                        'Sex': ['male', 'male', 'female']})

[19]: df['Age']

[19]: 0    22
      1    35
      2    58
      Name: Age, dtype: int64

[20]: type(df['Age'])

[20]: pandas.core.series.Series
```

Línea 19

- Si está familiarizado con el diccionario, seleccionar una sola columna es muy similar a seleccionar un valor del diccionario basado en una clave.

- La columna de un DataFrame es una serie.
  - Si solo está interesado en la columna "Edad" y desea extraer la columna, utilice el valor de la clave del diccionario.

```
[21]: df['Age'][0]

[21]: np.int64(22)
```

Línea 21

- Es posible indexar una serie.

- La columna de un DataFrame es una serie.
  - Convirtamos un DataFrame en una lista de tipos de listas y organicémoslo en forma bidimensional.

```
[22]: df=pd.DataFrame([[1, 2, 3], [4, 5, 6], [7, 8, 0]])

[23]: df.columns

[23]: RangeIndex(start=0, stop=3, step=1)

[24]: df[0]

[24]: 0    1
      1    4
      2    7
      Name: 0, dtype: int64

[25]: type(df[0])

[25]: pandas.core.series.Series
```

Línea 23

- Como no se especifica el nombre de la columna, aparece como un objeto RangeIndex.

- Crear una serie designando índice.

```
[26]: eye=pd.Series({'Brown': 220, 'Blue': 215, 'Hazel': 93, 'Green': 64})

[27]: eye.index

[27]: Index(['Brown', 'Blue', 'Hazel', 'Green'], dtype='object')

[28]: type(eye)

[28]: pandas.core.series.Series
```

Línea 27

- Tenga en cuenta que los tipos de datos string de la serie se reconocen en forma de objetos.

- Crear una serie designando índice.

```
[29]: eye.name
```

  - Los atributos de la serie son los siguientes.

```
[30]: eye.name = 'eye_color'
```

```
[31]: eye
```

```
[31]: Brown    220
      Blue     215
      Hazel     93
      Green     64
      Name: eye_color, dtype: int64
```

Línea 30

- Designe el nombre del índice con el atributo de nombre.

- Crear una serie designando índice.

```
[32]: eye.name

[32]: 'eye_color'

[33]: eye.index

[33]: Index(['Brown', 'Blue', 'Hazel', 'Green'], dtype='object')

[34]: eye.values

[34]: array([220, 215,  93,  64])
```

- Crear una serie designando índice.
  - Los métodos de la serie son los siguientes.

```
[35]: eye.sort_values()

[35]: Green     64
      Hazel     93
      Blue     215
      Brown    220
      Name: eye_color, dtype: int64

[36]: eye.sort_index()

[36]: Blue     215
      Brown    220
      Green     64
      Hazel     93
      Name: eye_color, dtype: int64
```

- Los métodos de la serie son los siguientes.

```
[37]: eye.unique()

[37]: array([220, 215,  93,  64])

[38]: eye.nunique()

[38]: 4
```

Línea 37

- Sólo se devuelven valores no duplicados.

Línea 38

- Este es el número de valores que no están duplicados.

- Los métodos de la serie son los siguientes.
  - Ahora, intentemos duplicar los valores.

```
[39]: my_data2=[220, 215, 93, 64, 64]

[40]: eye2=pd.Series(data=my_data2, index=['Brown', 'Blue', 'Blue', 'Hazel', 'Green'])

[41]: eye2

[41]: Brown    220
      Blue     215
      Blue      93
      Hazel     64
      Green     64
      dtype: int64
```

- Los métodos de la serie son los siguientes.
  - Ahora, intentemos duplicar los valores.

```
[42]: eye2.unique()

[42]: array([220, 215,  93,  64])

[43]: eye2.nunique()

[43]: 4

[44]: eye2.value_counts()

[44]: 64     2
      220    1
      215    1
      93     1
      Name: count, dtype: int64
```

Línea 42

- Sólo se devuelven valores no duplicados.

Línea 44

- Usando el método value_counts(), se agrega el mismo valor al número. Es un método que se utiliza con bastante frecuencia, así que no olvides recordarlo.

- Los métodos de la serie son los siguientes.
  - Las series se pueden indexar y dividir como NumPy Array.

```
[45]: ser=pd.Series([0, 10, 20, 30, 40], index=['a', 'b', 'c', 'd', 'e'])

[46]: ser.iloc[1]

[46]: np.int64(10)

[47]: ser.loc['b']

[47]: np.int64(10)

[48]: ser.iloc[1: 3]

[48]: b    10
      c    20
      dtype: int64
```

- Los métodos de la serie son los siguientes.
  - También son posibles operaciones en serie.

```
[49]: ser1=pd.Series([0, 1, 2, 3, 4], index=[0, 1, 2, 3, 4])
      ser2=pd.Series([0, 1, 2, 3, 4], index=[4, 3, 2, 1, 0])

[50]: ser1

[50]: 0    0
      1    1
      2    2
      3    3
      4    4
      dtype: int64

[51]: ser2

[51]: 4    0
      3    1
      2    2
      1    3
      0    4
      dtype: int64
```

- Los métodos de la serie son los siguientes.
  - También son posibles operaciones en serie.

```
[52]: ser1 + ser2

[52]: 0    4
      1    4
      2    4
      3    4
      4    4
      dtype: int64
```

ser1	ser2	ser1+ser2
0	4	4
1	3	4
2	2	4
3	1	4
4	0	4

- Los métodos de la serie son los siguientes.
  - También son posibles operaciones en serie.

```
[53]: ser1 * ser2

[53]: 0    0
      1    3
      2    4
      3    3
      4    0
      dtype: int64
```

ser1	ser2	ser1*ser2
0	4	0
1	3	3
2	2	4
3	1	3
4	0	0

- Los métodos de la serie son los siguientes.
  - También son posibles operaciones en serie.

```
[54]: ser1 / ser2

[54]: 0    0.000000
      1    0.333333
      2    1.000000
      3    3.000000
      4         inf
      dtype: float64
```

- Los métodos de la serie son los siguientes.
  - Métodos relacionados con la estadística de la serie

```
[55]: ser1.sum()

[55]: np.int64(10)

[56]: ser1.mean()

[56]: np.float64(2.0)

[57]: ser1.median()

[57]: 2.0
```

- Los métodos de la serie son los siguientes.
  - Métodos relacionados con la estadística de la serie

```
[58]: ser1.max()

[58]: 4

[59]: ser1.min()

[59]: 0

[60]: ser1.std()

[60]: 1.5811388300841898

[61]: ser1.sort_values()

[61]: 0    0
      1    1
      2    2
      3    3
      4    4
      dtype: int64
```

- Los métodos de la serie son los siguientes.
  - La función lambda se puede aplicar a la serie mediante la función aplicar.

```
[62]: ser_height = pd.Series([160, 170, 180],  name='height')

[63]: plus_10 = lambda x: x+10

[64]: plus_10(5)

[64]: 15

[65]: ser_height.apply(lambda x: x+10)

[65]: 0    170
      1    180
      2    190
      Name: height, dtype: int64
```

Línea 63

- Reemplazaremos la función lambda con lo que aprendimos en Conceptos básicos de Python.

Línea 65

- Suma 10 a cada elemento y devuélvelo a la serie.

Práctica de la serie Pandas y el marco de datos

- Bidimensional Array
  - Se mencionó que DataFrame es un Array bidimensional que consta de filas y columnas. El Array bidimensional es la forma de datos más utilizada por las computadoras.
  - En el campo del procesamiento de imágenes y análisis de datos, la mayoría de los datos se buscan y procesan en forma de Array bidimensional. La estructura de datos DataFrame de pandas se originó a partir de DataFrame de R, un popular paquete estadístico.
  - Es similar al NumPy Array bidimensional. El tipo de datos de la columna puede ser diferente. Es ideal para almacenar datos de archivos CSV, hojas de cálculo de Excel, tablas SQL, etc. Hay propiedades como columnas, índices, etc.
  - DataFrame se puede considerar como una colección de objetos de serie. Pandas DataFrames permite una representación de datos similar a las hojas de cálculo de Excel.
  - DataFrame consta de filas y columnas. Las filas son observaciones o instancias. Las columnas son variables.

```
[1]: import pandas as pd
```

- Serie Pandas y práctica de marcos de datos
  - pandas DataFrame es muy conveniente al leer un conjunto de datos externos, especialmente cuando los datos se almacenan en un formato separado por comas (CSV).

```
[2]: pd.read_csv('data_iris.csv')
```

[2]:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

- Serie Pandas y práctica de marcos de datos

```
[3]: iris_df = pd.read_csv('data_iris.csv')

[4]: type(iris_df)

[4]: pandas.core.frame.DataFrame
```

- Serie Pandas y práctica de marcos de datos
  - Los objetos DataFrame se pueden crear a partir de objetos de diccionario.

[5]:
```
data = {'NAME':['Jake', 'Jeniffer', 'Paul', 'Andrew'],
        'AGE': [24, 21, 25, 19],
        'GENDER': ['M', 'F', 'M', 'M']}
```

[6]:
```
pd.DataFrame(data)
```

[6]:

	NAME	AGE	GENDER
0	Jake	24	M
1	Jeniffer	21	F
2	Paul	25	M
3	Andrew	19	M

[7]:
```
df = pd.DataFrame(data)
```

- Cosas que se deben comprobar después de incorporar el DataFrame

1) df.info()
2) df.cabeza()

  - Este método imprime información sobre un DataFrame, incluido el tipo de índice y las columnas, los valores no nulos y el uso de memoria. Consulte ayuda (df.info) para obtener más detalles.

```
[8]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   NAME    4 non-null      object
 1   AGE     4 non-null      int64 
 2   GENDER  4 non-null      object
dtypes: int64(1), object(2)
memory usage: 228.0+ bytes
```

- Cosas que se deben comprobar después de incorporar el DataFrame

```
[9]: iris_df.head()
```

[9]:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Línea 9

- Solo muestra los primeros cinco datos. Se utiliza para ver rápidamente el estado o valor de los datos.

- Cosas que se deben comprobar después de incorporar el DataFrame

```
[10]: iris_df.tail()
```

[10]:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

Línea 10

- Solo muestra los últimos cinco datos.

- Cosas que se deben comprobar después de incorporar el DataFrame

```
[11]: iris_df.columns

[11]: Index(['Sepal.Length', 'Sepal.Width', 'Petal.Length', 'Petal.Width',
             'Species'],
            dtype='object')
```

Línea 11

- Se utiliza para comprobar el nombre de la columna.
- iris_df.columns() #precaución: Dado que no es un método, () no se utiliza.

- Cosas que se deben comprobar después de incorporar el DataFrame

```
[12]: iris_df.index

[12]: RangeIndex(start=0, stop=150, step=1)
```

Línea 12

- Usó el índice de verificación.

- Cosas que se deben comprobar después de incorporar el DataFrame
  - También puede cambiar el nombre de la columna, como se muestra a continuación.

```
[13]: iris_df.columns = ['Sepal_Length', 'Sepal_Width', 'Petal_Length', 'Petal_Width', 'Species']

[14]: iris_df
```

[14]:

	Sepal_Length	Sepal_Width	Petal_Length	Petal_Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...

Línea 14

- Puedes ver que ha cambiado de . a _.

- Cosas que se deben comprobar después de incorporar el DataFrame
  - La indexación y el corte también son posibles en DataFrames.

```
[15]: iris_df.Sepal_Length

[15]: 0      5.1
      1      4.9
      2      4.7
      3      4.6
      4      5.0
            ...
      145    6.7
      146    6.3
      147    6.5
      148    6.2
      149    5.9
      Name: Sepal_Length, Length: 150, dtype: float64

[16]: iris_df['Sepal_Length']

[16]: 0      5.1
      1      4.9
      2      4.7
      3      4.6
      4      5.0
            ...
      145    6.7
      146    6.3
      147    6.5
      148    6.2
      149    5.9
      Name: Sepal_Length, Length: 150, dtype: float64
```

- Varias columnas se convierten en forma bidimensional, como se muestra a continuación.

```
[17]: iris_df[['Sepal_Length', 'Sepal_Width']]
```

[17]:

	Sepal_Length	Sepal_Width
0	5.1	3.5
1	4.9	3.0
2	4.7	3.2
3	4.6	3.1
4	5.0	3.6
...	...	...
145	6.7	3.0
146	6.3	2.5
147	6.5	3.0
148	6.2	3.4
149	5.9	3.0

150 rows × 2 columns

- Serie Pandas y práctica de marcos de datos
  - También se puede crear un DataFrame en un Array bidimensional. Convirtamos la siguiente tabla en un Array bidimensional de DataFrames.

Nombre	Edad	Sexo	Escuela
Tomás	15	Masculino	medio
Alicia	10	Femenino	elemental

- Serie Pandas y práctica de marcos de datos

```
[18]: df = pd.DataFrame([['Name', 'Age', 'Sex', 'School'],
                         ['Tom', 15, 'Male', 'middle'],
                         ['Alice', 10, 'FeMale', 'elementary']])
```

- Como se muestra arriba, el nombre de la columna no se reconoce cuando se ingresan los datos. Por lo tanto, debe configurar las columnas usted mismo y crear un DataFrame.

```
[19]: df
```

[19]:

	0	1	2	3
0	Name	Age	Sex	School
1	Tom	15	Male	middle
2	Alice	10	FeMale	elementary

- Serie Pandas y práctica de marcos de datos

[20]:
```
df = pd.DataFrame([['Tom', 15, 'Male', 'middle'],
                   ['Alice', 10, 'FeMale', 'elementary']],
                  columns=['Name', 'Age', 'Sex', 'School'])
```

[21]:
```
df
```

[21]:

	Name	Age	Sex	School
0	Tom	15	Male	middle
1	Alice	10	FeMale	elementary

- Serie Pandas y práctica de marcos de datos
  - El índice de fila y el objeto de nombre de columna del df del DataFrame, representado como df.index y df.columns, se pueden cambiar asignando una nueva lista a los atributos del índice de fila y la columna.
    - #Cambio de índice de fila: DataFrame Object.index = nueva lista de índice de fila
    - #Cambio de nombre de columna: DataFrame Object.columns = nueva lista de nombres de columnas
  - No hay ningún índice en este momento.

```
[22]: df.index

[22]: RangeIndex(start=0, stop=2, step=1)
```

- Serie Pandas y práctica de marcos de datos

[23]:
```
df.index = ['stu1', 'stu2']
```

[24]:
```
df
```

[24]:

	Name	Age	Sex	School
stu1	Tom	15	Male	middle
stu2	Alice	10	FeMale	elementary

[25]:
```
df.index
```

[25]:
```
Index(['stu1', 'stu2'], dtype='object')
```

- Serie Pandas y práctica de marcos de datos
  - Las columnas también se pueden cambiar.

```
[26]: df.columns = ['student_name', 'years', 'sex2', 'school2']
```

```
[27]: df
```

[27]:

	student_name	years	sex2	school2
stu1	Tom	15	Male	middle
stu2	Alice	10	FeMale	elementary

- Serie Pandas y práctica de marcos de datos
  - También es posible seleccionar y reemplazar un índice de fila o una parte del nombre de la columna usando el método de cambio de nombre en el DataFrame. Sin embargo, cabe señalar que el objeto original no se modifica directamente, sino que se devuelve un nuevo objeto DataFrame. Utilice la opción inplace= True para cambiar el objeto original.
    - # Cambio de índice de fila: DataFrame Object.rename(index={Índice existente: Nuevo índice,…})
    - # Cambio de nombre de columna: DataFrame Object.renames=(columnas={Nombre existente: Nuevo nombre,…})

- Serie Pandas y práctica de marcos de datos

[28]:
```
df.rename(index={'stu1': 'student1'})
```

[28]:

	student_name	years	sex2	school2
student1	Tom	15	Male	middle
stu2	Alice	10	FeMale	elementary

[29]:
```
df
```

[29]:

	student_name	years	sex2	school2
stu1	Tom	15	Male	middle
stu2	Alice	10	FeMale	elementary

Línea 29

- Mirándolo de nuevo, observe que no cambió. Probemos la opción inplace=True.

- Serie Pandas y práctica de marcos de datos

```
[30]: df.rename(index={'stu1': 'student1'}, inplace=True)
```

```
[31]: df
```

[31]:

	student_name	years	sex2	school2
student1	Tom	15	Male	middle
stu2	Alice	10	FeMale	elementary

- Serie Pandas y práctica de marcos de datos
  - Si no desea utilizar la opción inplace=True, puede reasignar nuevos objetos con el mismo nombre, pero es un poco engorroso.
  - Si desea cambiar el índice 'stu2' a 'estudiante2', hágalo de la siguiente manera.

[32]:
```
df=df.rename(index={'stu2': 'student2'})
```

[33]:
```
df
```

[33]:

	student_name	years	sex2	school2
student1	Tom	15	Male	middle
student2	Alice	10	FeMale	elementary

- Serie Pandas y práctica de marcos de datos
  - Los nombres de las columnas se pueden crear de la misma manera.
  - Al cambiar 'student_name' a 'stu_name', se pueden utilizar los dos métodos siguientes.‘ to '

```
[34]: df.rename(columns={'student_name': 'stu_name'}, inplace=True)
```

```
[35]: df
```

[35]:

	stu_name	years	sex2	school2
student1	Tom	15	Male	middle
student2	Alice	10	FeMale	elementary

- Eliminar filas y columnas
  - Al eliminar una fila o columna de un DataFrame, utilice el método drop(). Al eliminar una fila, ingrese Axis=0 como opción de eje o no ingrese nada en absoluto. Por otro lado, si se ingresa Eje=1 como opción de eje, la columna se elimina. Si desea eliminar varias filas o columnas al mismo tiempo, ingréselas en forma de lista.
    - Eliminar fila: DataFrame Object.drop (índice de fila o lista, eje = 0)
    - Eliminar columna: DataFrame Object.drop (nombre de columna o lista, eje = 1)

```
[36]: exam_data = {'math': [90, 80, 70],
                  'eng': [98, 89, 95],
                  'music': [85, 95, 100],
                  'science': [100, 90, 90]}

[37]: exam_data

[37]: {'math': [90, 80, 70],
       'eng': [98, 89, 95],
       'music': [85, 95, 100],
       'science': [100, 90, 90]}
```

Línea 36

- Conversión de un DataFrame con una función DataFrame(). Guardar en la variable df.

- Eliminar filas y columnas
  - Creemos un DataFrame con un índice de la siguiente manera.

```
[38]: df = pd.DataFrame(exam_data, index=['stu1', 'stu2', 'stu3'])

[39]: df
```

[39]:

	math	eng	music	science
stu1	90	98	85	100
stu2	80	89	95	90
stu3	70	95	100	90

  - Tenga siempre la costumbre de copiar el original antes de eliminar la fila. Esto se debe a que si los datos se eliminan incorrectamente al verificarlos antes de eliminarlos, es necesario recuperarlos del original. Ésta es la única forma de ahorrar tiempo si la cantidad de datos es grande.

- Eliminar filas y columnas

```
[40]: df2 = df.copy()

[41]: df2.drop('stu2', inplace=True)
```

- Eliminar filas y columnas

```
[42]: df2
```

[42]:

	math	eng	music	science
stu1	90	98	85	100
stu3	70	95	100	90

```
[43]: df
```

[43]:

	math	eng	music	science
stu1	90	98	85	100
stu2	80	89	95	90
stu3	70	95	100	90

Línea 43

- El original permanece.

- Eliminar filas y columnas

```
[44]: df3 = df.copy()
```

```
[45]: df3.drop(['stu2', 'stu3'], axis=0, inplace=True)
```

```
[46]: df3
```

[46]:

	math	eng	music	science
stu1	90	98	85	100

- Eliminar filas y columnas
  - Lo mismo ocurre con la eliminación de columnas.

```
[47]: df
```

[47]:

	math	eng	music	science
stu1	90	98	85	100
stu2	80	89	95	90
stu3	70	95	100	90

```
[48]: df4 = df.copy()
```

Línea 48

- Replica el DataFrame df y guárdalo en la variable df4. Elimine una columna de df4.

- Eliminar filas y columnas
  - Lo mismo ocurre con la eliminación de columnas.

```
[49]: df4.drop('math', axis=1, inplace=True)
```

```
[50]: df4
```

[50]:

	eng	music	science
stu1	98	85	100
stu2	89	95	90
stu3	95	100	90

- Eliminar filas y columnas
  - Lo mismo ocurre con la eliminación de columnas.

```
[51]: df
```

[51]:

	math	eng	music	science
stu1	90	98	85	100
stu2	80	89	95	90
stu3	70	95	100	90

```
[52]: df5 = df.copy()
```

- Eliminar filas y columnas
  - Lo mismo ocurre con la eliminación de columnas.

```
[53]: df5.drop(['eng', 'science'], axis=1, inplace=True)
```

```
[54]: df5
```

[54]:

	math	music
stu1	90	85
stu2	80	95
stu3	70	100

- Eliminar filas y columnas
  - Utilice el índice de fila para seleccionar una fila.

```
[55]: df
```

[55]:

	math	eng	music	science
stu1	90	98	85	100
stu2	80	89	95	90
stu3	70	95	100	90

```
[56]: label1 = df.loc['stu1']
      position1 = df.iloc[0]
```

Línea 56-1

- Utilice el indexador loc.

Línea 56-2

- Utilice el indexador iloc.

- Eliminar filas y columnas
  - Utilice el índice de fila para seleccionar una fila.

```
[57]: label1

[57]: math       90
      eng        98
      music      85
      science   100
      Name: stu1, dtype: int64

[58]: position1

[58]: math       90
      eng        98
      music      85
      science   100
      Name: stu1, dtype: int64
```

- Eliminar filas y columnas
  - Utilice el índice de fila para seleccionar una fila.

[59]:
```
label2 = df.loc[['stu1', 'stu2']]
position2 = df.iloc[[0, 1]]
```

[60]:
```
label2
```

[60]:

	math	eng	music	science
stu1	90	98	85	100
stu2	80	89	95	90

[61]:
```
position2
```

[61]:

	math	eng	music	science
stu1	90	98	85	100
stu2	80	89	95	90

- Seleccionar columnas

```
[62]: math1 = df['math']

[63]: math1

[63]: stu1    90
      stu2    80
      stu3    70
      Name: math, dtype: int64
```

Línea 62

- Seleccione solo los datos de puntuación de “matemáticas”. Guárdelo en la variable math1.

- Seleccionar columnas

```
[64]: english = df.eng

[65]: english

[65]: stu1    98
      stu2    89
      stu3    95
      Name: eng, dtype: int64
```

Línea 64

- Seleccione solo los datos de puntuación en “inglés”. Guárdelo en inglés variable..

- Seleccionar columnas
  - Seleccione datos de partituras de “música” y “ciencia”. Guárdelo en la variable music_sci..

```
[66]: music_sci = df[['music', 'science']]
```

```
[67]: music_sci
```

[67]:

	music	science
stu1	85	100
stu2	95	90
stu3	100	90

- Seleccionar columnas
  - Si desea almacenarlo en forma bidimensional y solo hay una columna, use el formulario de lista [].

```
[68]: df['math']
```

```
[68]: stu1    90
      stu2    80
      stu3    70
      Name: math, dtype: int64
```

```
[69]: df[['math']]
```

[69]:

	math
stu1	90
stu2	80
stu3	70

- Índice de designación

[70]:
```
exam_data = {'name': ['hongildong', 'hongeeedong', 'hongsamdong'],
             'math': [90, 80, 70],
             'eng': [98, 89, 95],
             'music': [85, 95, 100],
             'phys_tra': [100, 90, 90]}
```

[71]:
```
df = pd.DataFrame(exam_data)
```

[72]:
```
df.index
```

[72]:
```
RangeIndex(start=0, stop=3, step=1)
```

- Índice de designación
  - No hay índice, pero se puede seleccionar un índice entre las columnas.
  - Designe la columna "nombre" como un nuevo índice y refleje los cambios en el objeto df.

```
[73]: df.set_index('name', inplace=True)

[74]: df.index

[74]: Index(['hongildong', 'hongeeedong', 'hongsamdong'], dtype='object', name='name')

[75]: df
```

[75]:

name	math	eng	music	phys_tra
hongildong	90	98	85	100
hongeeedong	80	89	95	90
hongsamdong	70	95	100	90

Línea 73

- Si la parte del nombre se mueve hacia abajo, se establece como índice.

- Índice de designación

```
[76]: a = df.loc['hongildong', 'music']

[77]: a

[77]: np.int64(85)
```

Línea 76

- Seleccione un elemento específico del DataFrame df (partitura de “música” de “hongildong”).”).

- Índice de designación

```
[78]: c = df.loc['hongildong', ['music', 'phys_tra']]

[79]: c

[79]: music       85
      phys_tra   100
      Name: hongildong, dtype: int64
```

Línea 78

- Seleccione dos o más elementos del DataFrame df (partituras de “música” y “phys_tra” de “hongildong”). df”).

- Índice de designación

```
[80]: d = df.iloc[0, [2, 3]]

[81]: d

[81]: music        85
      phys_tra    100
      Name: hongildong, dtype: int64
```

Línea 80

- Esto también se puede hacer con el índice entero iloc..

- Índice de designación

```
[82]: e = df.loc['hongildong', 'music': 'phys_tra']

[83]: e

[83]: music        85
      phys_tra    100
      Name: hongildong, dtype: int64
```

Línea 83

- También es posible rebanarlo.

- Índice de designación

```
[84]: f = df.iloc[0, 2:]

[85]: f

[85]: music        85
      phys_tra    100
      Name: hongildong, dtype: int64
```

- Índice de designación

```
[86]: g = df.loc[['hongildong', 'hongeeedong'], ['music', 'phys_tra']]

[87]: g
```

[87]:

name	music	phys_tra
hongildong	85	100
hongeeedong	95	90

Línea 86

- Seleccione un elemento de dos o más filas y columnas de df (partituras de “música” y “phys_tra” de “hongildong” y “hongeeedong”).df”).

- Índice de designación

```
[88]: h = df.iloc[[0, 1], [2, 3]]
```

```
[89]: h
```

[89]:

name	music	phys_tra
hongildong	85	100
hongeeedong	95	90

- Índice de designación

```
[90]: i = df.loc['hongildong': 'hongeeedong', 'music': 'phys_tra']
```

```
[91]: i
```

[91]:

name	music	phys_tra
hongildong	85	100
hongeeedong	95	90

- Índice de designación

```
[92]: df
```

[92]:

name	math	eng	music	phys_tra
hongildong	90	98	85	100
hongeeedong	80	89	95	90
hongsamdong	70	95	100	90

- Agregar una columna

```
[93]: df['kor'] = 80

[94]: df
```

[94]:

name	math	eng	music	phys_tra	kor
hongildong	90	98	85	100	80
hongeeedong	80	89	95	90	80
hongsamdong	70	95	100	90	80

Línea 93

- Agregue una columna de puntuación "kor" al DataFrame df. El valor de los datos es 80.

- Agregar una fila

```
[95]: df
```

[95]:

name	math	eng	music	phys_tra	kor
hongildong	90	98	85	100	80
hongeeedong	80	89	95	90	80
hongsamdong	70	95	100	90	80

- Agregar una fila

```
[96]: df.loc[3] = 0
```

```
[97]: df
```

[97]:

name	math	eng	music	phys_tra	kor
hongildong	90	98	85	100	80
hongeeedong	80	89	95	90	80
hongsamdong	70	95	100	90	80
3	0	0	0	0	0

Línea 96

- Agregue una nueva fila: ingrese el mismo valor de elemento.

- Agregar una fila

```
[98]: df.loc[3] = [90, 80, 70, 60, 77]
```

```
[99]: df
```

[99]:

name	math	eng	music	phys_tra	kor
hongildong	90	98	85	100	80
hongeeedong	80	89	95	90	80
hongsamdong	70	95	100	90	80
3	90	80	70	60	77

Línea 98

- Agregue una nueva fila: ingrese un array de valores de elementos.

- Agregar una fila

```
[100]: df.index

[100]: Index(['hongildong', 'hongeeedong', 'hongsamdong', 3], dtype='object', name='name')

[101]: df.index[3]

[101]: 3
```

- Agregar una fila
  - ayuda (df.reset_index))

[102]:
```
df
```

[102]:

name	math	eng	music	phys_tra	kor
hongildong	90	98	85	100	80
hongeeedong	80	89	95	90	80
hongsamdong	70	95	100	90	80
3	90	80	70	60	77

[103]:
```
df = df.reset_index()
```

[104]:
```
df
```

[104]:

	name	math	eng	music	phys_tra	kor
0	hongildong	90	98	85	100	80
1	hongeeedong	80	89	95	90	80
2	hongsamdong	70	95	100	90	80
3	3	90	80	70	60	77

- Agregar una fila

[105]:
```
df.iloc[3, 0] = 'hongsadong'
```

[106]:
```
df
```

[106]:

	name	math	eng	music	phys_tra	kor
0	hongildong	90	98	85	100	80
1	hongeeedong	80	89	95	90	80
2	hongsamdong	70	95	100	90	80
3	hongsadong	90	80	70	60	77

- Agregar una fila

```
[107]: df.set_index('name', inplace=True)

[108]: df
```

[108]:

name	math	eng	music	phys_tra	kor
hongildong	90	98	85	100	80
hongeeedong	80	89	95	90	80
hongsamdong	70	95	100	90	80
hongsadong	90	80	70	60	77

Línea 108

- Cuando el índice se configura de esta manera, preste mucha atención al agregar filas.

- Cambiar el elemento del marco de datos

[109]:
```
exam_data = {'name': ['stu1', 'stu2', 'stu3'],
             'math': [90, 80, 70],
             'eng': [98, 89, 95],
             'mus': [85, 95, 100],
             'phy': [100, 90, 90]}
df = pd.DataFrame(exam_data)
```

[110]:
```
df
```

[110]:

	name	math	eng	mus	phy
0	stu1	90	98	85	100
1	stu2	80	89	95	90
2	stu3	70	95	100	90

[111]:
```
df.index
```

[111]:
```
RangeIndex(start=0, stop=3, step=1)
```

- Cambiar el elemento del marco de datos

```
[112]: df.set_index('name', inplace=True)

[113]: df
```

[113]:

name	math	eng	mus	phy
stu1	90	98	85	100
stu2	80	89	95	90
stu3	70	95	100	90

Línea 112

- Designe la columna "nombre" como un nuevo índice y refleje los cambios en el objeto df.

- Cambiar el elemento del marco de datos

```
[114]: df.iloc[0, 3] = 80
```

```
[115]: df
```

[115]:

name	math	eng	mus	phy
stu1	90	98	85	80
stu2	80	89	95	90
stu3	70	95	100	90

Línea 114

- Un método para cambiar un elemento específico para el DataFrame df: Existen varios métodos para cambiar una puntuación "phy" de "stu1".

- Cambiar el elemento del marco de datos

[116]:
```
df.loc['stu1', 'phy'] = 90
```

[117]:
```
df
```

[117]:

name	math	eng	mus	phy
stu1	90	98	85	90
stu2	80	89	95	90
stu3	70	95	100	90

- Cambiar el elemento del marco de datos

```
[118]: df.loc['stu1', 'phy'] = 100
```

```
[119]: df
```

[119]:

name	math	eng	mus	phy
stu1	90	98	85	100
stu2	80	89	95	90
stu3	70	95	100	90

- Cambiar el elemento del marco de datos

```
[120]: df.loc['stu1', ['mus', 'phy']] = 50
```

```
[121]: df
```

[121]:

name	math	eng	mus	phy
stu1	90	98	50	50
stu2	80	89	95	90
stu3	70	95	100	90

Línea 120

- Un método para cambiar varios elementos de stu1 df: puntuaciones “mus” y “phy” de “stu1”.

- Cambiar el elemento del marco de datos

```
[122]: df.loc['stu1', ['mus', 'phy']] = 100, 100
```

```
[123]: df
```

[123]:

name	math	eng	mus	phy
stu1	90	98	100	100
stu2	80	89	95	90
stu3	70	95	100	90

- Transponer el marco de datos

[124]:
```
df.reset_index(inplace=True)
```

[125]:
```
df
```

[125]:

	name	math	eng	mus	phy
0	stu1	90	98	100	100
1	stu2	80	89	95	90
2	stu3	70	95	100	90

- Transponer el marco de datos

```
[126]: dff = df.transpose()

[127]: dff
```

[127]:

	0	1	2
name	stu1	stu2	stu3
math	90	80	70
eng	98	89	95
mus	100	95	100
phy	100	90	90

Línea 126

- Transponiendo el DataFrame df (usando el método).

- Transponer el marco de datos

```
[128]: dfff = dff.T

[129]: dfff
```

[129]:

	name	math	eng	mus	phy
0	stu1	90	98	100	100
1	stu2	80	89	95	90
2	stu3	70	95	100	90

Línea 128

- Transponiendo el DataFrame df nuevamente (usando atributos de clase).

- Transponer el marco de datos
  - Configuración de índice

```
[130]: exam_data = {'name': ['stu1', 'stu2', 'stu3'],
                    'math': [90, 80, 70],
                    'eng': [98, 89, 95],
                    'mus':  [85, 95, 100],
                    'phy': [100, 90, 90]}
       df = pd.DataFrame(exam_data)

[131]: ndf = df.set_index(['name'])

[132]: ndf
```

[132]:

name	math	eng	mus	phy
stu1	90	98	85	100
stu2	80	89	95	90
stu3	70	95	100	90



Línea 131

- Una columna específica se establece como índice de fila de un DataFrame.

- Transponer el marco de datos

```
[133]: ndf2 = ndf.set_index(['mus'])
```

```
[134]: ndf2
```

[134]:

mus	math	eng	phy
85	90	98	100
95	80	89	90
100	70	95	90

- Transponer el marco de datos

```
[135]: ndf3 = ndf.set_index(['math', 'mus'])
```

```
[136]: ndf3
```

[136]:

		eng	phy
math	**mus**		
90	**85**	98	100
80	**95**	89	90
70	**100**	95	90

Línea 135

- Índice múltiple

- Transponer el marco de datos
  - Índice

[137]:
```
1 dict_data = {'c0': [1, 2, 3],
2              'c1': [4, 5, 6],
3              'c2': [7, 8, 9],
4              'c3': [10, 11, 12],
5              'c4': [13, 14, 15]}
6 df = pd.DataFrame(dict_data, index=['r0', 'r1', 'r2'])
```

Línea 137-1

- Definición del diccionario

Línea 137-6

- Convirtiendo el diccionario en un DataFrame. Especifique el índice como [r0, r1, r2].

- Transponer el marco de datos

[138]:
```
new_index = ['r0', 'r1', 'r2', 'r3', 'r4']
ndf = df.reindex(new_index)
ndf
```

[138]:

	c0	c1	c2	c3	c4
r0	1.0	4.0	7.0	10.0	13.0
r1	2.0	5.0	8.0	11.0	14.0
r2	3.0	6.0	9.0	12.0	15.0
r3	NaN	NaN	NaN	NaN	NaN
r4	NaN	NaN	NaN	NaN	NaN

Línea 138

- Vuelva a designar el índice a [r0, r1, r2, r3, r4]. [r0, r1, r2, r3, r4].

- Transponer el marco de datos

[139]:
```
new_index = ['r0', 'r1', 'r2', 'r3', 'r4']
ndf2 = df.reindex(new_index, fill_value=0)
ndf2
```

[139]:

	c0	c1	c2	c3	c4
r0	1	4	7	10	13
r1	2	5	8	11	14
r2	3	6	9	12	15
r3	0	0	0	0	0
r4	0	0	0	0	0

Línea 139

- Complete el valor NaN generado al reindexar con el número 0.

- Transponer el marco de datos

[140]:
```
1 dict_data = {'c0': [1, 2, 3],
2              'c1': [4, 5, 6],
3              'c2': [7, 8, 9],
4              'c3': [10, 11, 12],
5              'c4': [13, 14, 15]}
6 df = pd.DataFrame(dict_data, index=['r0', 'r1', 'r2'])
7 df
```

[140]:

	c0	c1	c2	c3	c4
r0	1	4	7	10	13
r1	2	5	8	11	14
r2	3	6	9	12	15

Línea 140-1

- Definición del diccionario

Línea 140-2

- Convirtiendo el diccionario en un DataFrame. Especifique el índice como [r0, r1, r2].

- Transponer el marco de datos

```
[141]: ndf = df.reset_index()
```

```
[142]: ndf
```

[142]:

	index	c0	c1	c2	c3	c4
0	r0	1	4	7	10	13
1	r1	2	5	8	11	14
2	r2	3	6	9	12	15

Línea 141

- Restablezca el índice de la fila a un número entero.

- Transponer el marco de datos

[143]:
```
1 dict_data = {'c0': [1, 2, 3],
2              'c1': [4, 5, 6],
3              'c2': [7, 8, 9],
4              'c3': [10, 11, 12],
5              'c4': [13, 14, 15]}
6 df = pd.DataFrame(dict_data, index=['r0', 'r1', 'r2'])
7 df
```

[143]:

	c0	c1	c2	c3	c4
r0	1	4	7	10	13
r1	2	5	8	11	14
r2	3	6	9	12	15

Línea 143-1

- Definición del diccionario

Línea 143-2

- Convirtiendo el diccionario en un DataFrame. Especifique el índice como [r0, r1, r2].

- Transponer el marco de datos

[144]:
```
ndf = df.sort_index(ascending=False)
ndf
```

[144]:

	c0	c1	c2	c3	c4
r2	3	6	9	12	15
r1	2	5	8	11	14
r0	1	4	7	10	13

Línea 144

- Ordene los índices de filas en orden descendente.

- Transponer el marco de datos

[145]:
```
1 dict_data = {'c0': [1, 2, 3],
2              'c1': [4, 5, 6],
3              'c2': [7, 8, 9],
4              'c3': [10, 11, 12],
5              'c4': [13, 14, 15]}
6 df = pd.DataFrame(dict_data, index=['r0', 'r1', 'r2'])
7 df
```

[145]:

	c0	c1	c2	c3	c4
r0	1	4	7	10	13
r1	2	5	8	11	14
r2	3	6	9	12	15

Línea 145-1

- Definición del diccionario

Línea 145-2

- Convirtiendo el diccionario en un DataFrame. Especifique el índice como [r0, r1, r2].

- Transponer el marco de datos

```
[146]: ndf = df.sort_values(by='c1', ascending=False)
       print(ndf)
```

```
    c0  c1  c2  c3  c4
r2   3   6   9  12  15
r1   2   5   8  11  14
r0   1   4   7  10  13
```

Línea 146

- Ordene en orden descendente según la columna c1.

Operaciones en serie

- Serie versus números
  - Agregar un número a un objeto de serie agrega un número a cada uno de los elementos individuales de la serie y convierte el resultado calculado en un objeto de serie.

[1]:
```
import numpy as np
import pandas as pd
```

- Serie versus números
  - Agregar un número a un objeto de serie agrega un número a cada uno de los elementos individuales de la serie y convierte el resultado calculado en un objeto de serie.

```
[2]: student1 = pd.Series({'kor': 100, 'eng': 80, 'math': 90})
     student1

[2]: kor     100
     eng      80
     math     90
     dtype: int64

[3]: percentage = student1 / 200
     percentage

[3]: kor     0.50
     eng     0.40
     math    0.45
     dtype: float64
```

Línea 2

- Crea una serie de pandas con datos de un diccionario.

Línea 3

- Divida las puntuaciones del estudiante por 200 por materia.

- Serie vs Serie
  - Para todos los índices de la serie, se calculan los elementos con el mismo índice.

```
[4]:
1 student1 = pd.Series({'kor': 100, 'eng': 80, 'math': 90})
2 student2 = pd.Series({'math': 80, 'kor': 90, 'eng': 80})
3
4 addition = student1 + student2
5 subtraction = student1 - student2
6 multiplication = student1 * student2
7 division = student1 / student2
```

Línea 4

- Crea una serie de pandas con datos de un diccionario.

Línea 4-4 ~ 4-7

- Realiza los cuatro cálculos aritméticos fundamentales en base a las puntuaciones de cada alumno por materia.

- Serie vs Serie

```
[5]: result = pd.DataFrame([addition, subtraction, multiplication, division],
                           index=['addition', 'subtraction', 'multiplication', 'division'])
     result
```

[5]:

	eng	kor	math
addition	160.0	190.000000	170.000
subtraction	0.0	10.000000	10.000
multiplication	6400.0	9000.000000	7200.000
division	1.0	1.111111	1.125

Línea 5

- Combine los resultados de la operación aritmética en un DataFrame (serie -> DataFrame).

- En el ejemplo anterior, el orden de los nombres de los sujetos proporcionados por índice es diferente. Sin embargo, pandas busca y clasifica el mismo nombre de sujeto (índice) y agrega puntuaciones del mismo nombre de sujeto (índice).

- Serie vs Serie
  - Si el número de elementos en las dos series es diferente o el tamaño de la serie es el mismo, se trata como si no fuera un número (NaN), lo que significa que no hay un valor válido.

[6]:
```
1 student1 = pd.Series({'kor': np.nan, 'eng': 80, 'math': 90})
2 student2 = pd.Series({'math': 80, 'kor': 90})
3
4 addition = student1 + student2
5 subtraction = student1 - student2
6 multiplication = student1 * student2
7 division = student1 / student2
```

Línea 6

- Crea dos series de pandas con datos de diccionarios.

Línea 6-4 ~ 6-7

- Realiza los cuatro cálculos aritméticos fundamentales en base a las puntuaciones de cada alumno por materia. (Serie vs Serie)

- Serie vs Serie

[7]:
```
result = pd.DataFrame([addition, subtraction, multiplication, division],
                      index=['addition', 'subtraction', 'multiplication', 'division'])
result
```

[7]:

	eng	kor	math
addition	NaN	NaN	170.000
subtraction	NaN	NaN	10.000
multiplication	NaN	NaN	7200.000
division	NaN	NaN	1.125

Línea 7

- Combine los resultados de la operación aritmética en un DataFrame (serie -> DataFrame).

- Método de operación
  - Como aprendió en diapositivas anteriores, cuando no hay un índice común o NAN en los valores, devuelve NaN. Para que no se produzca este resultado, establezca el valor de relleno para el conjunto de datos.

[8]:
```
1 student1 = pd.Series({'kor': np.nan, 'eng': 80, 'math': 90})
2 student2 = pd.Series({'math': 80, 'kor': 90})
3
4 sr_add = student1.add(student2, fill_value=0)
5 sr_sub = student1.sub(student2, fill_value=0)
6 sr_mul = student1.mul(student2, fill_value=0)
7 sr_div = student1.div(student2, fill_value=0)
```

Línea 8

- Crea dos series de pandas con datos de diccionarios.

Línea 8-4 ~ 8-7

- Realiza los cuatro cálculos aritméticos fundamentales en base a las puntuaciones de cada alumno por materia. (Utilice el método de operación)

- Método de operación

```
[9]: result = pd.DataFrame([sr_add, sr_sub, sr_mul, sr_div],
                           index=['addition', 'subtraction', 'multiplication', 'division'])
     result
```

[9]:

	eng	kor	math
addition	80.0	90.0	170.000
subtraction	80.0	-90.0	10.000
multiplication	0.0	0.0	7200.000
division	inf	0.0	1.125

Línea 9

- Combine los resultados de la operación aritmética en un DataFrame (serie -> DataFrame).

Operaciones de marco de datos

Los DataFrames pueden entenderse como conceptos que amplían las operaciones en serie. Primero, se ordena según el índice de fila/columna y se calcula entre los elementos correspondientes uno por uno.

- Marco de datos frente a números
  - La librería seaborn se tratará más adelante en visualización, pero aquí se utilizó para importar datos.

```
[10]: import seaborn as sns

      titanic = sns.load_dataset('titanic')
```

Línea 10

- Cree un DataFrame seleccionando dos columnas, edad y tarifa, del conjunto de datos del Titanic.

- Marco de datos frente a números

[11]:
```
1 titanic = sns.load_dataset('titanic')
2 df = titanic.loc[:, ['age', 'fare']]
3 df.head()
```

[11]:

	age	fare
0	22.0	7.2500
1	38.0	71.2833
2	26.0	7.9250
3	35.0	53.1000
4	35.0	8.0500

Línea 11-2

- Cree un DataFrame seleccionando dos columnas, edad y tarifa, del conjunto de datos del Titanic.

Línea 11-3

- Muestra sólo las primeras cinco líneas.

- Marco de datos frente a números

[12]:
```
1 addition = df + 10
2 addition.head()
```

[12]:

	age	fare
0	32.0	17.2500
1	48.0	81.2833
2	36.0	17.9250
3	45.0	63.1000
4	45.0	18.0500

Línea 12-1

- Agregue 10 al marco de datos.

Línea 12-2

- Muestra sólo las primeras cinco líneas.

- Mientras se mantiene la forma del DataFrame existente, solo el valor del elemento se reemplaza con un nuevo valor calculado y se devuelve como un nuevo objeto DataFrame.

- Marco de datos frente a marco de datos
  - Se calculan los elementos en las mismas posiciones de fila y columna de cada DataFrame. Si el elemento no está presente en ninguno de los lados o en NaN, el resultado del cálculo se trata como NaN.

```
[13]: titanic = sns.load_dataset('titanic')
      df = titanic.loc[:, ['age', 'fare']]
```

Línea 13

- Cree un DataFrame seleccionando dos columnas, edad y tarifa, del conjunto de datos del Titanic.

- Marco de datos frente a números

```
[14]: addition = df + 10
      addition
```

[14]:

	age	fare
0	32.0	17.2500
1	48.0	81.2833
2	36.0	17.9250
3	45.0	63.1000
4	45.0	18.0500
...	...	...

Línea 14

- Agregue 10 al marco de datos.

- Marco de datos frente a números

```
[15]: 1 subtraction = addition - df
      2 subtraction.tail()
```

[15]:

	age	fare
886	10.0	10.0
887	10.0	10.0
888	NaN	10.0
889	10.0	10.0
890	10.0	10.0

Línea 15-1

- Calcular entre DataFrames (additon - df). df).

Línea 15-2

- Muestra sólo las últimas cinco líneas.

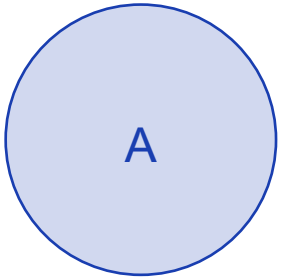
Unidad 2.

Exploración óptima de datos a través de Pandas

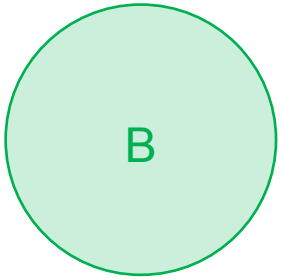
- 2.1. Pipelines: estructuras de datos según tipos de datos
- 2.2. Serie Pandas y DataFrames
- 2.3. Fusionar y vincular marcos de datos
- 2.4. Clasificación de marcos de datos e índice múltiple
- 2.5. Examinar las características de los datos mediante estadísticas descriptivas y muestras de datos

Manipulación de marcos de datos

- Fusionando marcos de datos:

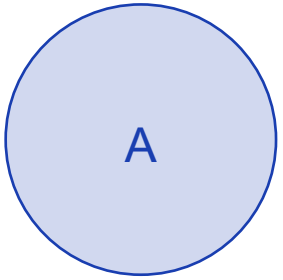


Nombre	Género	Edad
harry potter	Masculino	23
David Baker	Masculino	31
Juan Smith	Masculino	22
Juan Martínez	Masculino	36
Jane Connor	Femenino	30

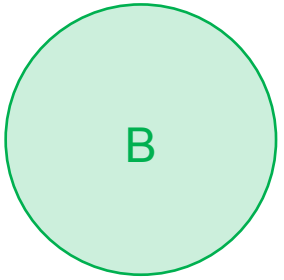


Nombre	Posición	Salario
Juan Smith	Interno	25000
Alex DuBois	Líder del equipo	75000
Joanna Rowling	Gerente	90000
Jane Connor	Gerente	70000

- Fusión de marcos de datos: A.Name y B.Name utilizados como claves

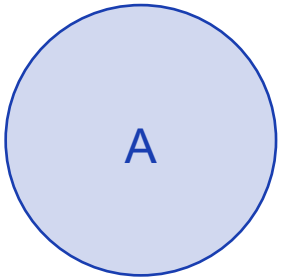


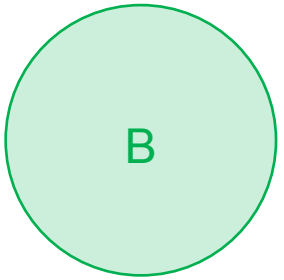
Nombre	Género	Edad
harry potter	Masculino	23
David Baker	Masculino	31
Juan Smith	Masculino	22
Juan Martínez	Masculino	36
Jane Connor	Femenino	30



Nombre	Posición	Salario
Juan Smith	Interno	25000
Alex DuBois	Líder del equipo	75000
Joanna Rowling	Gerente	90000
Jane Connor	Gerente	70000

- Fusión de marcos de datos: unión interna

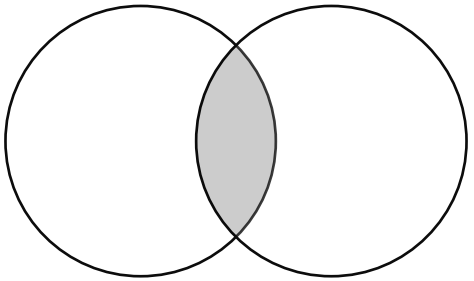




Nombre	Género	Edad
~~harry potter~~	~~Masculino~~	~~23~~
~~David Baker~~	~~Masculino~~	~~31~~
Juan Smith	Masculino	22
~~Juan Martínez~~	~~Masculino~~	~~36~~
Jane Connor	Femenino	30

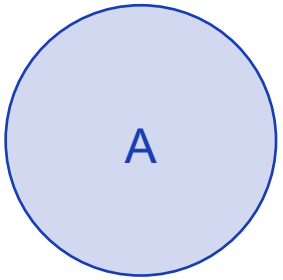
Nombre	Posición	Salario
Juan Smith	Interno	25000
~~Alex DuBois~~	~~Líder del equipo~~	~~75000~~
~~Joanna Rowling~~	~~Gerente~~	~~90000~~
Jane Connor	Gerente	70000

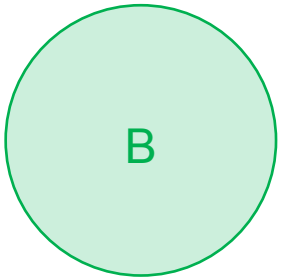
- Fusión de marcos de datos: unión interna



A.Nombre	Género	Edad	B.Nombre	Posición	Salario
Juan Smith	Masculino	22	Juan Smith	Interno	25000
Jane Connor	Femenino	30	Jane Connor	Gerente	70000

- Fusionar marcos de datos: unión izquierda

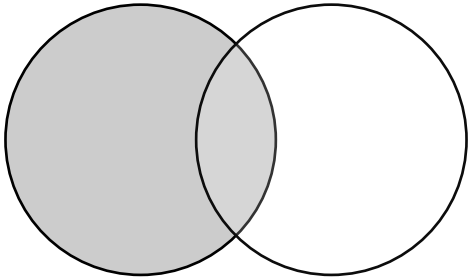




Nombre	Género	Edad
harry potter	Masculino	23
David Baker	Masculino	31
Juan Smith	Masculino	22
Juan Martínez	Masculino	36
Jane Connor	Femenino	30

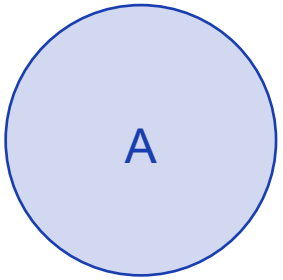
Nombre	Posición	Salario
Juan Smith	Interno	25000
~~Alex DuBois~~	~~Líder del equipo~~	~~75000~~
~~Joanna Rowling~~	~~Gerente~~	~~90000~~
Jane Connor	Gerente	70000

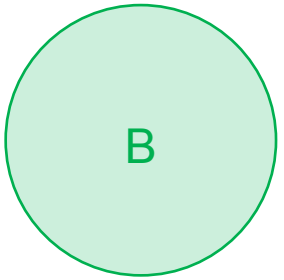
- Fusionar marcos de datos: unión izquierda



A.Nombre	Género	Edad	B.Nombre	Posición	Salario
harry potter	Masculino	23	NA	NA	NA
David Baker	Masculino	31	NA	NA	NA
Juan Smith	Masculino	22	Juan Smith	Interno	25000
Juan Martínez	Masculino	36	NA	NA	NA
Jane Connor	Femenino	30	Jane Connor	Gerente	70000

- Fusionando DataFrames: Unión derecha

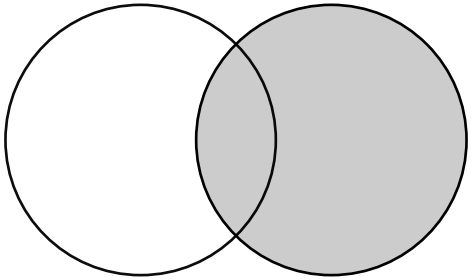




Nombre	Género	Edad
~~harry potter~~	~~Masculino~~	~~23~~
~~David Baker~~	~~Masculino~~	~~31~~
Juan Smith	Masculino	22
~~Juan Martínez~~	~~Masculino~~	~~36~~
Jane Connor	Femenino	30

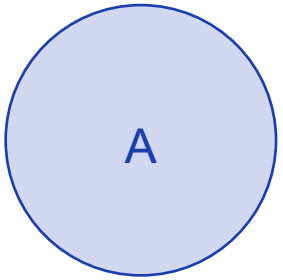
Nombre	Posición	Salario
Juan Smith	Interno	25000
Alex DuBois	Líder del equipo	75000
Joanna Rowling	Gerente	90000
Jane Connor	Gerente	70000

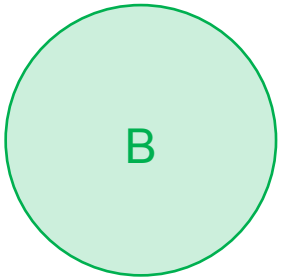
- Fusionando DataFrames: Unión derecha



A.Nombre	Género	Edad	B.Nombre	Posición	Salario
Juan Smith	Masculino	22	Juan Smith	Interno	25000
NA	NA	NA	Alex DuBois	Líder del equipo	75000
NA	NA	NA	Joanna Rowling	Gerente	90000
Jane Connor	Femenino	30	Jane Connor	Gerente	70000

- Fusión de marcos de datos: unión externa completa

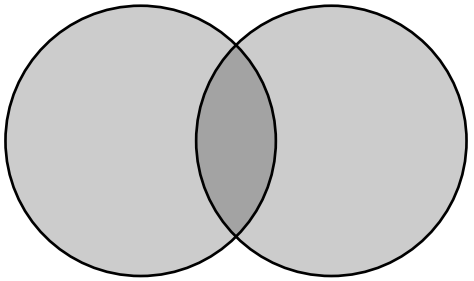




Nombre	Género	Edad
harry potter	Masculino	23
David Baker	Masculino	31
Juan Smith	Masculino	22
Juan Martínez	Masculino	36
Jane Connor	Femenino	30

Nombre	Posición	Salario
Juan Smith	Interno	25000
Alex DuBois	Líder del equipo	75000
Joanna Rowling	Gerente	90000
Jane Connor	Gerente	70000

- Fusión de marcos de datos: unión externa completa



A.Nombre	Género	Edad	B.Nombre	Posición	Salario
harry potter	Masculino	23	NA	NA	NA
David Baker	Masculino	31	NA	NA	NA
Juan Smith	Masculino	22	Juan Smith	Interno	25000
Juan Martínez	Masculino	36	NA	NA	NA
Jane Connor	Femenino	30	Jane Connor	Gerente	70000
NA	NA	NA	Alex DuBois	Líder del equipo	75000
NA	NA	NA	Joanna Rowling	Gerente	90000

- Fusionar y vincular marcos de datos:

```
[ ]:
1 # pd.merge(df_left, df_right, on='NAME')
2 # pd.merge(df_left, df_right, left_on='NAME', right_on='NAME', how='inner')
3 # pd.merge(df_left, df_right, left_on='NAME', right_on='NAME', how='left')
4 # pd.merge(df_left, df_right, left_on='NAME', right_on='NAME', how='right')
5 # pd.merge(df_left, df_right, left_on='NAME', right_on='NAME', how='outer')
```

Línea 1, 2

- unión interna

Línea 3

- unirse a la izquierda

Línea 4

- Unirse a la derecha

Línea 5

- Unión exterior completa

2.3. Fusionar y vincular marcos de datos

- Fusionar y vincular marcos de datos:

```
[ ]:
1 # pd.concat([df_A, df_B], sort=True)
2 # pd.concat([df_A, df_B], axis=1, sort=True)
```

Línea 1

- Enlace verticalmente haciendo coincidir los nombres de las columnas.

Línea 2

- Unir horizontalmente haciendo coincidir los índices.

- Vincular marcos de datos
  - Cuando los datos se dividen en varios lugares, puede ser necesario combinarlos en uno o vincularlos. En pandas, las funciones utilizadas para combinar o vincular DataFrames incluyen concat(), merge() y join.
  - pandas.concat (lista de marcos de datos)
  - Si no se especifica la dirección axial, se aplica la opción predeterminada eje=0 y se conecta en la dirección de fila arriba/abajo.

- Vincular marcos de datos

[1]:
```
import pandas as pd
```

[2]:
```
df1 = pd.DataFrame({'a': ['a0', 'a1', 'a2', 'a3'],
                    'b': ['b0', 'b1', 'b2', 'b3'],
                    'c': ['c0', 'c1', 'c2', 'c3']},
                   index=[0, 1, 2, 3])
```

[3]:
```
df1
```

[3]:

	a	b	c
0	a0	b0	c0
1	a1	b1	c1
2	a2	b2	c2
3	a3	b3	c3

- Vincular marcos de datos

[4]:
```
df2 = pd.DataFrame({'a': ['a2', 'a3', 'a4', 'a5'],
                    'b': ['b2', 'b3', 'b4', 'b5'],
                    'c': ['c2', 'c3', 'c4', 'c5'],
                    'd': ['d2', 'd3', 'd4', 'd5']},
                   index=[2, 3, 4, 5])
```

[5]:
```
df2
```

[5]:

	a	b	c	d
2	a2	b2	c2	d2
3	a3	b3	c3	d3
4	a4	b4	c4	d4
5	a5	b5	c5	d5

- Vincular marcos de datos
  - Las filas 0, 1, 2 y 3 derivadas de df1 se ingresan como NaN porque no hay una columna "d".

```
[6]: pd.concat([df1, df2])
```

[6]:

	a	b	c	d
0	a0	b0	c0	NaN
1	a1	b1	c1	NaN
2	a2	b2	c2	NaN
3	a3	b3	c3	NaN
2	a2	b2	c2	d2
3	a3	b3	c3	d3
4	a4	b4	c4	d4
5	a5	b5	c5	d5

- Vincular marcos de datos
  - ayuda (pd.concat). índice_ignorar).

```
[7]: pd.concat([df1, df2], ignore_index=True)
```

[7]:

	a	b	c	d
0	a0	b0	c0	NaN
1	a1	b1	c1	NaN
2	a2	b2	c2	NaN
3	a3	b3	c3	NaN
4	a2	b2	c2	d2
5	a3	b3	c3	d3
6	a4	b4	c4	d4
7	a5	b5	c5	d5

- Vincular marcos de datos
  - La opción Axis=1 vincula el DataFrame in en las direcciones de las columnas izquierda/derecha.

[8]:
```
pd.concat([df1, df2], axis=1)
```

[8]:

	a	b	c	a	b	c	d
0	a0	b0	c0	NaN	NaN	NaN	NaN
1	a1	b1	c1	NaN	NaN	NaN	NaN
2	a2	b2	c2	a2	b2	c2	d2
3	a3	b3	c3	a3	b3	c3	d3
4	NaN	NaN	NaN	a4	b4	c4	d4
5	NaN	NaN	NaN	a5	b5	c5	d5

- Fusionar marcos de datos
  - La función concat() es un concepto que fusiona los dos DataFrames según un determinado criterio de manera similar al comando de unión de SQL. En este caso, la columna o índice que es la referencia se denomina clave. La clave debe existir en ambos DataFrames.

[9]:
```
# Creating a DataFrame with Stock Market Data
df1 = pd.read_excel('./stock price.xlsx')
df2 = pd.read_excel('./stock valuation.xlsx')
```

[10]:
```
df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   id          10 non-null     int64
 1   stock_name  10 non-null     object
 2   value       10 non-null     float64
 3   price       10 non-null     int64
dtypes: float64(1), int64(2), object(1)
memory usage: 452.0+ bytes
```

- Fusionar marcos de datos

```
[11]: df1.head()
```

[11]:

	id	stock_name	value	price
0	128940	Hanmi Pharmaceutical	59385.666667	421000
1	130960	CJ E&M	58540.666667	98900
2	138250	NS Shopping	14558.666667	13200
3	139480	E-mart	239230.833333	254500
4	142280	Green Cross Medical Science Corporation	468.833333	10200

- Fusionar marcos de datos

```
[12]: df2.head()
```

[12]:

	id	name	eps	bps	per	pbr
0	130960	CJ E&M	6301.333333	54068	15.695091	1.829178
1	136480	Harim Co.	274.166667	3551	11.489362	0.887074
2	138040	Meritz Financial Group	2122.333333	14894	6.313806	0.899691
3	139480	E-mart	18268.166667	295780	13.931338	0.860437
4	145990	Samyang	5741.000000	108090	14.283226	0.758627

- La clave aquí debe ser una identificación con un valor que no se superponga.

2.3. Fusionar y vincular marcos de datos

- Fusionar marcos de datos
  - Consulte la ayuda (pd.merge) para obtener más detalles. Pongamos dos DataFrames como parámetros y fusionémoslos

```
[13]: pd.merge(df1, df2)
```

[13]:

	id	stock_name	value	price	name	eps	bps	per	pbr
0	130960	CJ E&M	58540.666667	98900	CJ E&M	6301.333333	54068	15.695091	1.829178
1	139480	E-mart	239230.833333	254500	E-mart	18268.166667	295780	13.931338	0.860437
2	145990	Samyang	82750.000000	82000	Samyang	5741.000000	108090	14.283226	0.758627
3	185750	Chong Kun Dang	40293.666667	100500	Chong Kun Dang	3990.333333	40684	25.185866	2.470259
4	204210	Mode Tour Reit	3093.333333	3475	Mode Tour Reit	85.166667	5335	40.802348	0.651359

- Fusionar marcos de datos

[14]:
```
df_1 = pd.read_excel('./stock price.xlsx')
df_2 = pd.read_excel('./stock valuation.xlsx')
```

[15]:
```
pd.merge(df_1, df_2)
```

[15]:

	id	stock_name	value	price	name	eps	bps	per	pbr
0	130960	CJ E&M	58540.666667	98900	CJ E&M	6301.333333	54068	15.695091	1.829178
1	139480	E-mart	239230.833333	254500	E-mart	18268.166667	295780	13.931338	0.860437
2	145990	Samyang	82750.000000	82000	Samyang	5741.000000	108090	14.283226	0.758627
3	185750	Chong Kun Dang	40293.666667	100500	Chong Kun Dang	3990.333333	40684	25.185866	2.470259
4	204210	Mode Tour Reit	3093.333333	3475	Mode Tour Reit	85.166667	5335	40.802348	0.651359

  - Si no hay nombres de columnas comunes, se produce un error.

- Fusionar marcos de datos
  - A continuación se explica cómo solucionar el error.
  - Traiga el archivo nuevamente.

[16]:
```
df_1 = pd.read_excel('./stock price.xlsx')
df_2 = pd.read_excel('./stock valuation.xlsx')
```

[17]:
```
help(pd.merge)
```

```
Help on function merge in module pandas.core.reshape.merge:

merge(
    left: 'DataFrame | Series',
    right: 'DataFrame | Series',
    how: 'MergeHow' = 'inner',
    on: 'IndexLabel | AnyArrayLike | None' = None,
    left_on: 'IndexLabel | AnyArrayLike | None' = None,
    right_on: 'IndexLabel | AnyArrayLike | None' = None,
    left_index: 'bool' = False,
    right_index: 'bool' = False,
    sort: 'bool' = False,
```

- Fusionar marcos de datos
  - A continuación se explica cómo solucionar el error.

```
[18]: pd.merge(df1, df2)
```

[18]:

	id	stock_name	value	price	name	eps	bps	per	pbr
0	130960	CJ E&M	58540.666667	98900	CJ E&M	6301.333333	54068	15.695091	1.829178
1	139480	E-mart	239230.833333	254500	E-mart	18268.166667	295780	13.931338	0.860437
2	145990	Samyang	82750.000000	82000	Samyang	5741.000000	108090	14.283226	0.758627
3	185750	Chong Kun Dang	40293.666667	100500	Chong Kun Dang	3990.333333	40684	25.185866	2.470259
4	204210	Mode Tour Reit	3093.333333	3475	Mode Tour Reit	85.166667	5335	40.802348	0.651359

- Fusionar marcos de datos
  - Las opciones on=Ninguno y how='inner' se aplican como valores predeterminados. La opción on=Ninguno significa fusionar todas las columnas que pertenecen en común a los dos DataFrames en una referencia (clave).
  - La opción how='inner' significa que los datos de la columna de referencia se extraen solo cuando los datos son una intersección común a ambos DataFrames.
  - Fusionamos y devolvemos cinco acciones comúnmente existentes según la columna "id".

2.3. Fusionar y vincular marcos de datos

- Fusionar marcos de datos
  - Cambiemos deliberadamente y veamos qué sucede si hay columnas "id_1" y "nombre_1" en el lado izquierdo del DataFrame y si hay columnas "id_1" y "nombre_1" en su lado derecho.

```
[19]: df1_3 = pd.read_excel('./stock price.xlsx')
      df2_3 = pd.read_excel('./stock valuation.xlsx')
```

```
[20]: pd.merge(df1_3, df2_3)
```

[20]:

	id	stock_name	value	price	name	eps	bps	per	pbr
0	130960	CJ E&M	58540.666667	98900	CJ E&M	6301.333333	54068	15.695091	1.829178
1	139480	E-mart	239230.833333	254500	E-mart	18268.166667	295780	13.931338	0.860437
2	145990	Samyang	82750.000000	82000	Samyang	5741.000000	108090	14.283226	0.758627
3	185750	Chong Kun Dang	40293.666667	100500	Chong Kun Dang	3990.333333	40684	25.185866	2.470259
4	204210	Mode Tour Reit	3093.333333	3475	Mode Tour Reit	85.166667	5335	40.802348	0.651359

  - Cuando las dos columnas son iguales, los resultados se combinan en función de los valores del nombre de columna común.

- Fusionar marcos de datos
  - Si usa how='left', se devolverán todas las empresas en el DataFrame izquierdo y las que no estén a la derecha se tratarán como NaN.

[21]:
```
df1 = pd.read_excel('./stock price.xlsx')
df2 = pd.read_excel('./stock valuation.xlsx')
```

[22]:
```
pd.merge(df1, df2, on='id', how='left')
```

[22]:

	id	stock_name	value	price	name	eps	bps	per	pbr
0	128940	Hanmi Pharmaceutical	59385.666667	421000	NaN	NaN	NaN	NaN	NaN
1	130960	CJ E&M	58540.666667	98900	CJ E&M	6301.333333	54068.0	15.695091	1.829178
2	138250	NS Shopping	14558.666667	13200	NaN	NaN	NaN	NaN	NaN
3	139480	E-mart	239230.833333	254500	E-mart	18268.166667	295780.0	13.931338	0.860437
4	142280	Green Cross Medical Science Corporation	468.833333	10200	NaN	NaN	NaN	NaN	NaN
5	145990	Samyang	82750.000000	82000	Samyang	5741.000000	108090.0	14.283226	0.758627
6	185750	Chong Kun Dang	40293.666667	100500	Chong Kun Dang	3990.333333	40684.0	25.185866	2.470259
7	192400	Cuckoo Holdings	179204.666667	177500	NaN	NaN	NaN	NaN	NaN
8	199800	Toolgen	-2514.333333	115400	NaN	NaN	NaN	NaN	NaN
9	204210	Mode Tour Reit	3093.333333	3475	Mode Tour Reit	85.166667	5335.0	40.802348	0.651359

- Fusionar marcos de datos
  - Si how='right', se devolverán todas las empresas en el DataFrame derecho y las que no estén en el izquierdo se tratarán como NaN.

[23]:
```
pd.merge(df1, df2, on='id', how='right')
```

[23]:

	id	stock_name	value	price	name	eps	bps	per	pbr
0	130960	CJ E&M	58540.666667	98900.0	CJ E&M	6301.333333	54068	15.695091	1.829178
1	136480	NaN	NaN	NaN	Harim Co.	274.166667	3551	11.489362	0.887074
2	138040	NaN	NaN	NaN	Meritz Financial Group	2122.333333	14894	6.313806	0.899691
3	139480	E-mart	239230.833333	254500.0	E-mart	18268.166667	295780	13.931338	0.860437
4	145990	Samyang	82750.000000	82000.0	Samyang	5741.000000	108090	14.283226	0.758627
5	161390	NaN	NaN	NaN	Hankook Tire	5648.500000	51341	7.453306	0.820007
6	181710	NaN	NaN	NaN	NHN Entertainment	2110.166667	78434	30.755864	0.827447
7	185750	Chong Kun Dang	40293.666667	100500.0	Chong Kun Dang	3990.333333	40684	25.185866	2.470259
8	204210	Mode Tour Reit	3093.333333	3475.0	Mode Tour Reit	85.166667	5335	40.802348	0.651359
9	207940	NaN	NaN	NaN	Samsung BioLogics	4644.166667	60099	89.790059	6.938551

- Fusionar marcos de datos
  - Si usa how="outer", se devuelven todos los datos de la izquierda y la derecha.

```
[24]: pd.merge(df1, df2, on='id', how='outer')
```

[24]:

	id	stock_name	value	price	name	eps	bps	per	pbr
0	128940	Hanmi Pharmaceutical	59385.666667	421000.0	NaN	NaN	NaN	NaN	NaN
1	130960	CJ E&M	58540.666667	98900.0	CJ E&M	6301.333333	54068.0	15.695091	1.829178
2	136480	NaN	NaN	NaN	Harim Co.	274.166667	3551.0	11.489362	0.887074
3	138040	NaN	NaN	NaN	Meritz Financial Group	2122.333333	14894.0	6.313806	0.899691
4	138250	NS Shopping	14558.666667	13200.0	NaN	NaN	NaN	NaN	NaN
5	139480	E-mart	239230.833333	254500.0	E-mart	18268.166667	295780.0	13.931338	0.860437
6	142280	Green Cross Medical Science Corporation	468.833333	10200.0	NaN	NaN	NaN	NaN	NaN
7	145990	Samyang	82750.000000	82000.0	Samyang	5741.000000	108090.0	14.283226	0.758627
8	161390	NaN	NaN	NaN	Hankook Tire	5648.500000	51341.0	7.453306	0.820007
9	181710	NaN	NaN	NaN	NHN Entertainment	2110.166667	78434.0	30.755864	0.827447
10	185750	Chong Kun Dang	40293.666667	100500.0	Chong Kun Dang	3990.333333	40684.0	25.185866	2.470259
11	192400	Cuckoo Holdings	179204.666667	177500.0	NaN	NaN	NaN	NaN	NaN
12	199800	Toolgen	-2514.333333	115400.0	NaN	NaN	NaN	NaN	NaN
13	204210	Mode Tour Reit	3093.333333	3475.0	Mode Tour Reit	85.166667	5335.0	40.802348	0.651359
14	207940	NaN	NaN	NaN	Samsung BioLogics	4644.166667	60099.0	89.790059	6.938551

Unidad 2.

Exploración óptima de datos a través de Pandas

- 2.1. Pipelines: estructuras de datos según tipos de datos
- 2.2. Serie Pandas y DataFrames
- 2.3. Fusionar y vincular marcos de datos
- 2.4. Clasificación de marcos de datos e índice múltiple
- 2.5. Examinar las características de los datos mediante estadísticas descriptivas y muestras de datos

Manipulación de marcos de datos

- Clasificación:
  - Es posible ordenar las filas de un DataFrame usando una o más columnas.

```
[ ]: # df.sort_values(by='bloodtype')
     # df.sort_values(by='bloodtype', ascending=False)
     # df.sort_values(by=['bloodtype', 'gender'])
```

Línea 1

- Ordenar en orden ascendente.

Línea 2

- Ordenar en orden descendente.

Línea 3

- Ordene usando dos columnas.

- Indexación jerárquica con MultiIndex:

[1]:
```
1 import numpy as np
2 import pandas as pd
```

[2]:
```
1 my_header = ['a', 'b', 'c']
2 my_index_out = ['G1']*3 + ['G2']*3
3 my_index_in = [1, 2, 3]*2
4 my_index_zipped = list(zip(my_index_out, my_index_in))
5 my_index = pd.MultiIndex.from_tuples(my_index_zipped)
6 df = pd.DataFrame(data=np.random.randn(6, 3), index=my_index, columns=my_header)
7 df
```

Línea 2-1

- Nombres de columnas

Línea 2-2

- Etiquetas para la capa exterior.

Línea 2-3

- Etiquetas para la capa interior.

- Indexación jerárquica con MultiIndex:

[1]:
```
1 import numpy as np
2 import pandas as pd
```

[2]:
```
1 my_header = ['a', 'b', 'c']
2 my_index_out = ['G1']*3 + ['G2']*3
3 my_index_in = [1, 2, 3]*2
4 my_index_zipped = list(zip(my_index_out, my_index_in))
5 my_index = pd.MultiIndex.from_tuples(my_index_zipped)
6 df = pd.DataFrame(data=np.random.randn(6, 3), index=my_index, columns=my_header)
7 df
```

Línea 2-4

- Crea una lista de tuplas con las etiquetas.

Línea 2-5

- Cree el índice múltiple.

Línea 2-6

- Aplicar el MultiIndex.

- Indexación jerárquica con MultiIndex:

[2]:

		a	b	c
G1	**1**	0.434185	0.773500	-0.816949
	2	0.122308	-0.226593	-1.397873
	3	1.206406	1.180414	0.764996
G2	**1**	0.167183	0.096283	-0.320553
	2	-0.445008	1.901970	-0.224632
	3	1.982042	2.130970	0.633304

Unidad 2.

Exploración óptima de datos a través de Pandas

- 2.1. Pipelines: estructuras de datos según tipos de datos
- 2.2. Serie Pandas y DataFrames
- 2.3. Fusionar y vincular marcos de datos
- 2.4. Clasificación de marcos de datos e índice múltiple
- 2.5. Examinar las características de los datos mediante estadísticas descriptivas y muestras de datos

Resumen de marco de datos

- Agrupar y resumir:

```
[1]: 1 import pandas as pd

[2]: 1 df=pd.read_csv('data_studentlist.csv')

[3]: 1 df.head(3)
```

[3]:

	name	gender	age	grade	absence	bloodtype	height	weight
0	Jared Diamond	M	23	3	Y	O	165.3	68.2
1	Sarah O'Donnel	F	22	2	N	AB	170.1	53.0
2	Brian Martin	M	24	4	N	B	175.0	80.1

```
[4]: 1 df.groupby(['gender', 'bloodtype'])['height'].mean()
```

```
[4]: gender  bloodtype
     F       A            172.450000
             AB           170.100000
             B            158.200000
             O            164.433333
     M       A            165.700000
             AB           181.050000
             B            174.550000
             O            166.200000
     Name: height, dtype: float64
```

- Pivotando:
  - Manipule los índices y las columnas y luego resuma.

[5]:
```
my_dict = {'Size': ['L', 'L', 'M', 'M', 'M', 'S', 'S', 'S', 'S'],
           'Type': ['A', 'A', 'A', 'B', 'B', 'A', 'A', 'B', 'B'],
           'Location': ['L1', 'L1', 'L1', 'L2', 'L2', 'L1', 'L2', 'L2', 'L1'],
           'A': [1, 2, 2, 3, 3, 4, 5, 6, 7],
           'B': [2, 4, 5, 5, 6, 6, 8, 9, 9]}
df = pd.DataFrame(my_dict)
df
```

[5]:

	Size	Type	Location	A	B
0	L	A	L1	1	2
1	L	A	L1	2	4
2	M	A	L1	2	5
3	M	B	L2	3	5
4	M	B	L2	3	6
5	S	A	L1	4	6
6	S	A	L2	5	8
7	S	B	L2	6	9
8	S	B	L1	7	9

- Pivotando:
  - Índice por 'Tamaño' y 'Tipo'. Columnas por 'Ubicación'. Valores proporcionados por la columna 'B'.

[6]:
```
1 dfr = pd.pivot_table(df, index=['Size', 'Type'], columns='Location', values='B')
2 dfr
```

[6]:

Size	Type	Location L1	L2
L	A	3.0	NaN
M	A	5.0	NaN
	B	NaN	5.5
S	A	6.0	8.0
	B	9.0	9.0

- Pivotando:

```
[7]: 1 dfr.columns

[7]: Index(['L1', 'L2'], dtype='object', name='Location')

[8]: 1 dfr.index

[8]: MultiIndex([('L', 'A'),
                 ('M', 'A'),
                 ('M', 'B'),
                 ('S', 'A'),
                 ('S', 'B')],
                names=['Size', 'Type'])
```

- Pivotando:

```
[9]: 1 pd.pivot_table(df, index=['Size', 'Type'], columns='Location', values='B', fill_value=0)
```

[9]:

Size	Type	Location L1	Location L2
L	A	3.0	0.0
M	A	5.0	0.0
	B	0.0	5.5
S	A	6.0	8.0
	B	9.0	9.0

  - Lo mismo que el gráfico de la derecha, pero completa los valores que faltan

[6]:

Size	Type	Location L1	Location L2
L	A	3.0	NaN
M	A	5.0	NaN
	B	NaN	5.5
S	A	6.0	8.0
	B	9.0	9.0

- Pivotando:

```
[10]: 1 pd.pivot_table(df, index=['Size', 'Type'], columns='Location', values='B', aggfunc='mean', fill_value=0)
```

[10]:

Size	Type	Location L1	L2
L	A	3.0	0.0
M	A	5.0	0.0
	B	0.0	5.5
S	A	6.0	8.0
	B	9.0	9.0

- Lo mismo que el gráfico de la derecha con la función de agregación especificada.

[9]:

Size	Type	Location L1	L2
L	A	3.0	0.0
M	A	5.0	0.0
	B	0.0	5.5
S	A	6.0	8.0
	B	9.0	9.0

- Pivotando:
  - Índice por 'Ubicación'. Columnas por 'Tamaño' y 'Tipo'. Valores proporcionados por la columna 'B'.

[11]:
```
1 dfr = pd.pivot_table(df, index='Location', columns=['Size', 'Type'], values='B')
2 dfr
```

[11]:

Size	L		M		S
Type	A	A	B	A	B
Location					
L1	3.0	5.0	NaN	6.0	9.0
L2	NaN	NaN	5.5	8.0	9.0

- Pivotando:

```
[12]: 1 dfr.index

[12]: Index(['L1', 'L2'], dtype='object', name='Location')

[13]: 1 dfr.columns

[13]: MultiIndex([('L', 'A'),
                  ('M', 'A'),
                  ('M', 'B'),
                  ('S', 'A'),
                  ('S', 'B')],
                 names=['Size', 'Type'])
```

Línea 13

- Ahora, objeto MultiIndex para las columnas.

- Pivotando:
  - La función de agregación es la "mediana".

```
[14]: 1 pd.pivot_table(df, index=['Size', 'Type'], columns='Location', values='B', aggfunc='median', fill_value=0)
```

[14]:

Size	Type	Location L1	L2
L	A	3.0	0.0
M	A	5.0	0.0
	B	0.0	5.5
S	A	6.0	8.0
	B	9.0	9.0

- Pivotando:
  - Promedios de grupo de las columnas 'A' y 'B'

```
[15]: 1 pd.pivot_table(df, index=['Size', 'Type'], values=['A', 'B'], aggfunc='mean')
```

[15]:

Size	Type	A	B
L	A	1.5	3.0
M	A	2.0	5.0
	B	3.0	5.5
S	A	4.5	7.0
	B	6.5	9.0

- Pivotando:
  - Ahora, con el método groupby(). El resultado es el mismo.

```
[16]: 1 df.groupby(['Size', 'Type'])[['A', 'B']].mean()
```

[16]:

Size	Type	A	B
L	A	1.5	3.0
M	A	2.0	5.0
	B	3.0	5.5
S	A	4.5	7.0
	B	6.5	9.0

- Pivotando:
  - Agregue las columnas 'A' y 'B' de manera diferente.

```
[17]: 1 pd.pivot_table(df, index=['Size', 'Type'], values=['A', 'B'], aggfunc={'A':'max', 'B':'min'})
```

[17]:

		A	B
Size	**Type**		
L	**A**	2	2
M	**A**	2	5
	B	3	5
S	**A**	5	6
	B	7	9

- Estadística:

```
[ ]:
1 # df.sum(axis=0)
2 # df.sum(axis=1)
3 # df.mean(axis=0, skipna=False)
4 # df.describe()
5 # df.count(axis=0)
6 # df.A.corr(df.B)
7 # df.corr()
8 # df.corrwith(df.A)
```

Línea 1

- Sumas de columnas

Línea 2

- Sumas de filas

Línea 3

- Promedios de columnas sin omitir los valores faltantes

Línea 4

- Estadísticas descriptivas de las columnas (variables)

- Estadística:

```
[ ]:
1 # df.sum(axis=0)
2 # df.sum(axis=1)
3 # df.mean(axis=0, skipna=False)
4 # df.describe()
5 # df.count(axis=0)
6 # df.A.corr(df.B)
7 # df.corr()
8 # df.corrwith(df.A)
```

Línea 5

- Valores no faltantes a lo largo de las columnas

Línea 6

- Correlación entre la columna 'A' y la columna 'B'

Línea 7

- Matriz de correlación tomando las variables numéricas por pares

Línea 8

- Correlaciones entre 'A' y las otras variables numéricas

- Detección y procesamiento de valores faltantes

```
[ ]:
1 # df.isnull()
2 # (df.isnull()).sum(axis=0)
3 # (df.isnull()).mean(axis=0)
4 # df.dropna(axis=0)
5 # df.dropna(axis=1)
6 # df.dropna(axis=0, thresh=3)
7 # df.fillna(value=0)
```

Línea 1

- Un DataFrame con True donde se encuentran los valores faltantes.

Línea 2

- Cuente los valores faltantes para cada columna.

Línea 3

- Proporciones de los valores faltantes para cada columna.

Línea 4

- Elimine las filas donde se encuentren uno o más valores faltantes.

- Detección y procesamiento de valores faltantes.

```
[ ]:
1 # df.isnull()
2 # (df.isnull()).sum(axis=0)
3 # (df.isnull()).mean(axis=0)
4 # df.dropna(axis=0)
5 # df.dropna(axis=1)
6 # df.dropna(axis=0, thresh=3)
7 # df.fillna(value=0)
```

Línea 5

- Elimine las columnas donde se encuentren uno o más valores faltantes.

Línea 6

- Elimine las filas con menos de 3 valores normales.

Línea 7

- Complete los valores faltantes con 0.

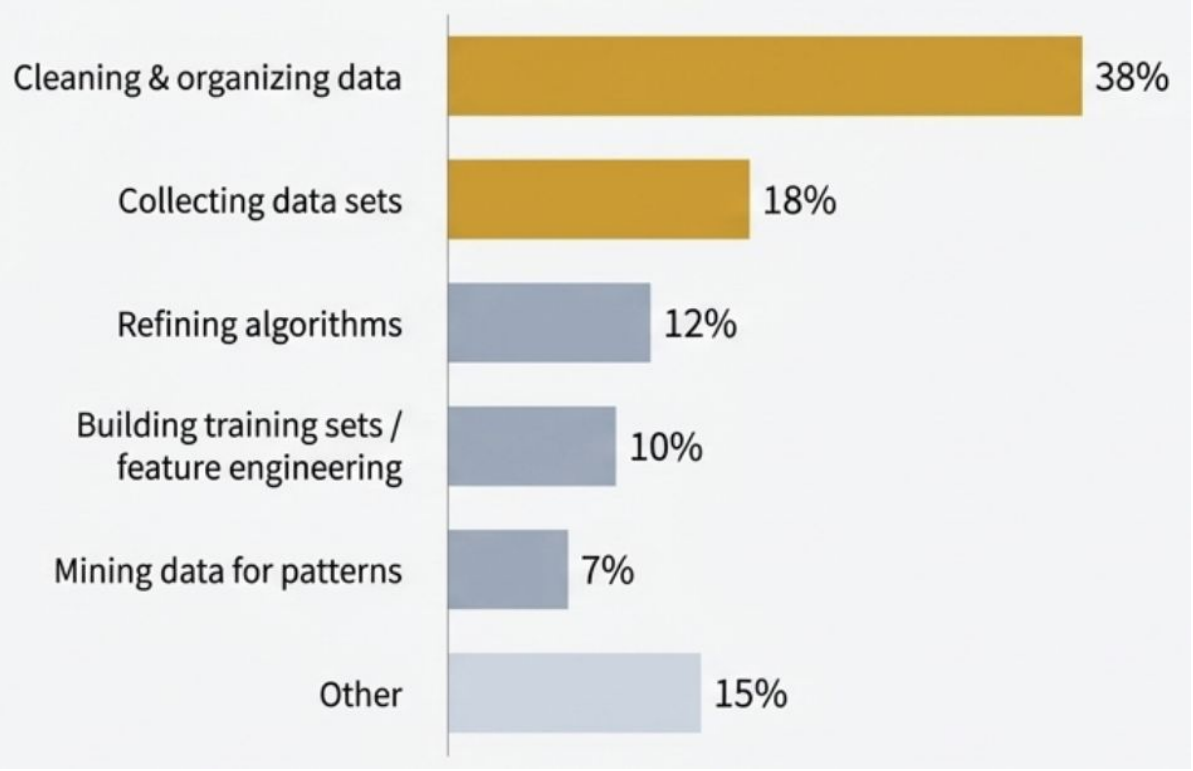
Unidad 3.

Preprocesamiento de datos de Pandas para una ejecución óptima del modelo

- 3.1. Preprocesamiento de datos
- 3.2. Identificar las propiedades de los datos
- 3.3. Comprobación de datos faltantes
- 3.4. Comprobación y procesamiento de datos duplicados
- 3.5. Ingeniería de características de datos

Preprocesamiento de datos

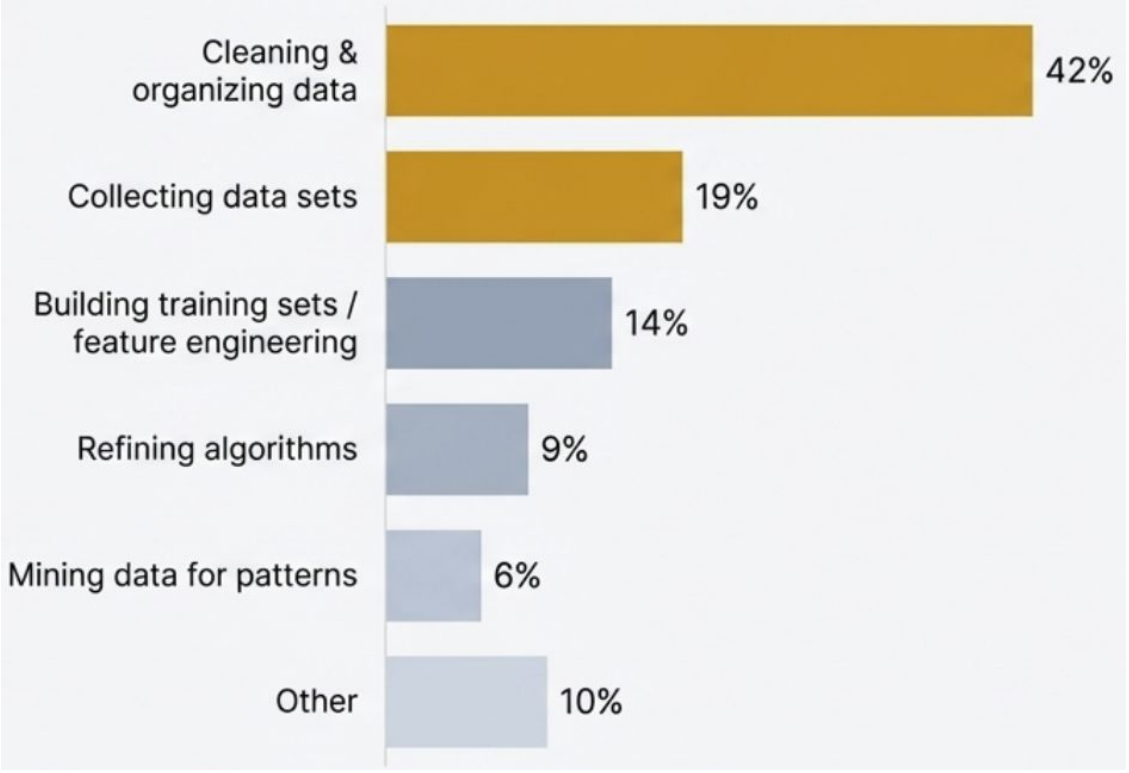
- Encuesta a científicos de datos:
  - ¿A qué dedican más tiempo los científicos de datos?



- Este gráfico se basa en hallazgos de encuestas industriales ampliamente citadas que muestran cómo los científicos de datos asignan su tiempo entre las tareas.
- Las encuestas a gran escala realizadas por Kaggle y Anaconda informan consistentemente que la preparación de datos (incluida la limpieza, organización y recopilación de datos) representa la mayor parte de la carga de trabajo de un científico de datos.

Fuente: Encuesta de Kaggle Data Science y Machine Learning, Informe sobre el estado de la ciencia de datos de Anaconda. Cifras adaptadas para reflejar las tendencias persistentes de la industria

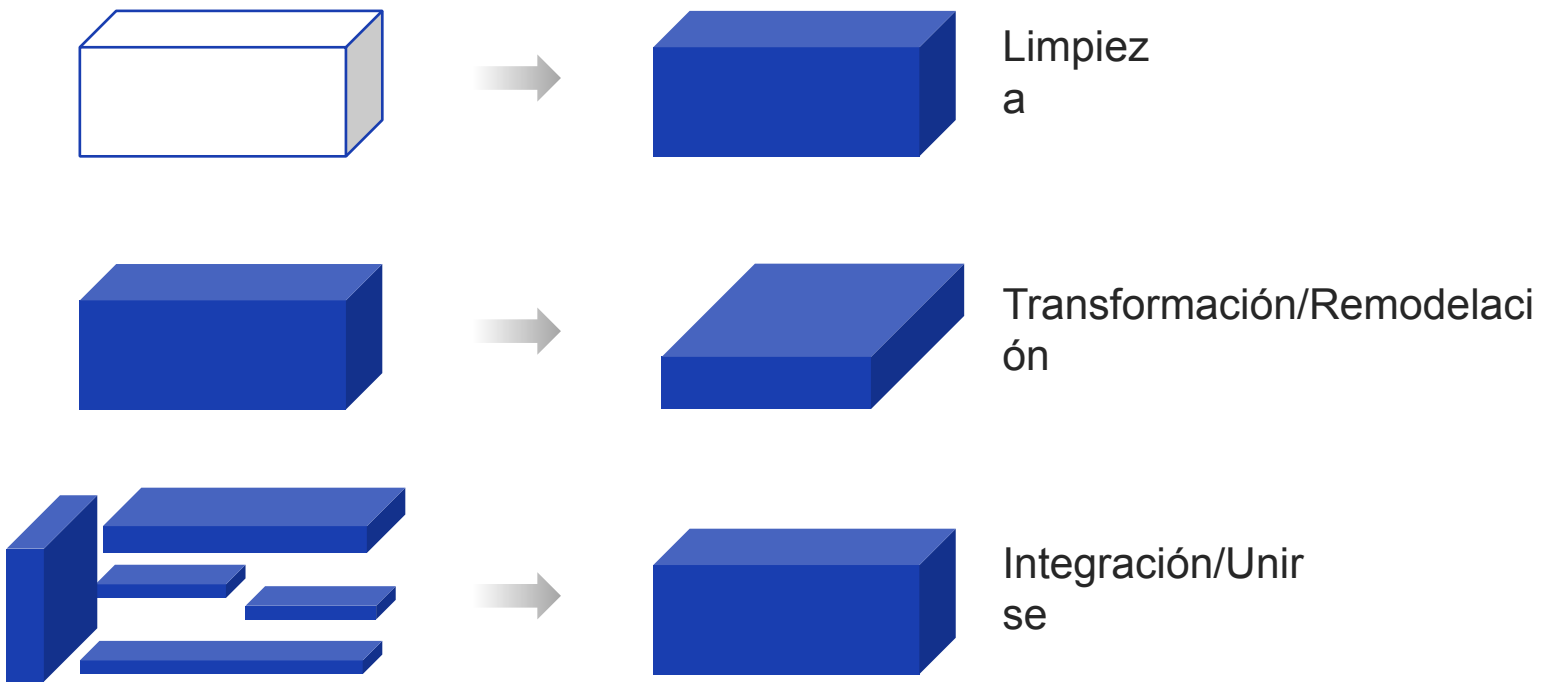
- Encuesta a científicos de datos:
  - ¿Qué es lo que menos les gusta hacer a los científicos de datos?



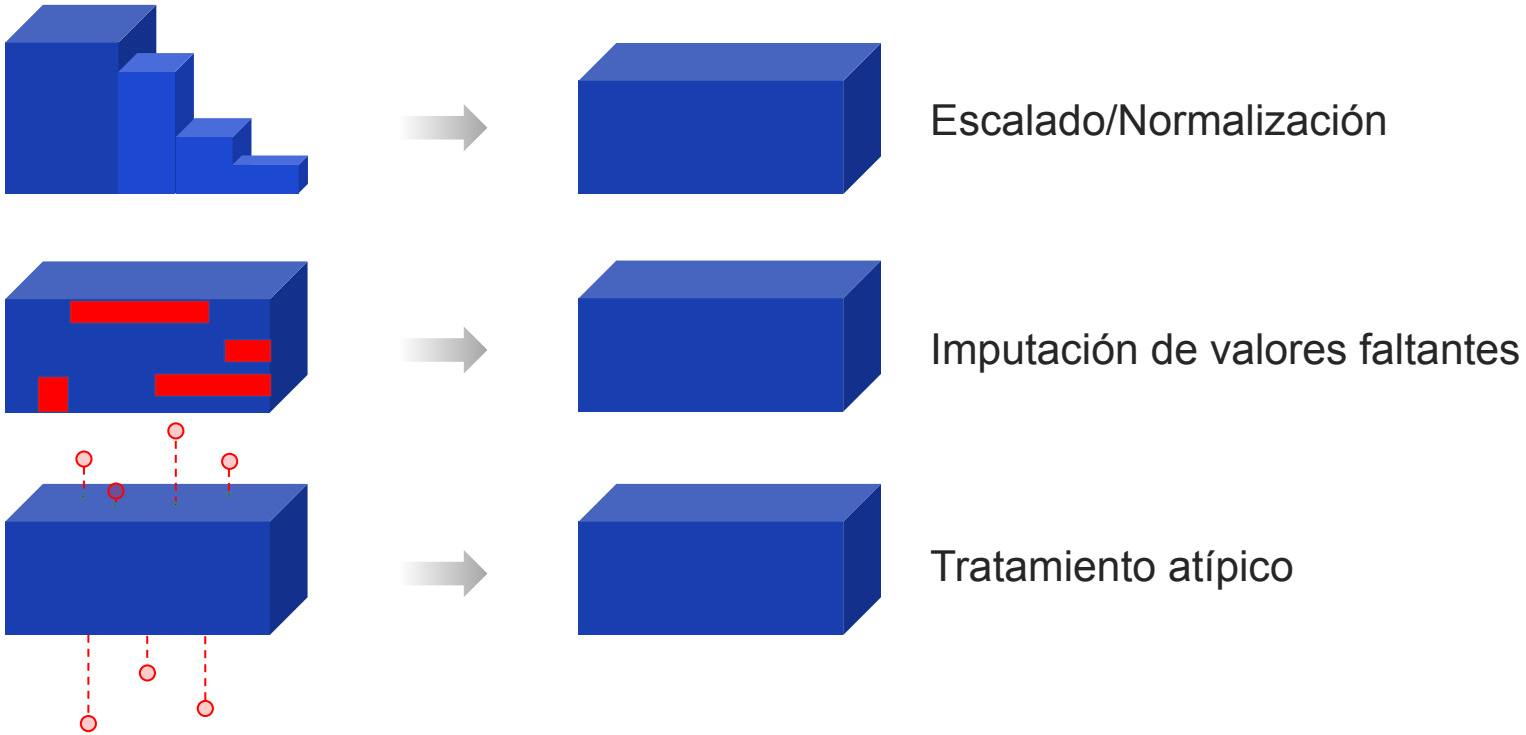
- Este gráfico resume los resultados de la encuesta sobre qué tareas los científicos de datos encuentran menos agradables.
- Según encuestas de la industria realizadas por Kaggle y Anaconda, la limpieza y preparación de datos se identifican constantemente como las tareas que menos gustan, a pesar de su importancia crítica.
- Esto refleja un desajuste de larga data entre dónde pasan la mayor parte de su tiempo los científicos de datos y qué actividades encuentran más gratificantes.

Fuente: Encuesta de Kaggle Data Science y Machine Learning, Informe sobre el estado de la ciencia de datos de Anaconda. Cifras adaptadas para reflejar las tendencias persistentes de la industria

- Operaciones:



- Operaciones:



- Consideraciones sobre el algoritmo:
  - A menudo, los algoritmos Machine Learning se pueden clasificar en "tipo árbol" y "no tipo árbol".
    - a) Algoritmos tipo árbol: Árbol, Bosque aleatorio, AdaBoost, XGMBoost, etc.
    - b) Algoritmos que no son de árbol: regresión lineal, regresión logística, SVM, red neuronal, etc.
  - El escalado/normalización y el tratamiento de valores atípicos pueden ser necesarios para algoritmos que no son de tipo árbol, pero no para algoritmos de tipo árbol.
  - Esto se debe a que los algoritmos tipo árbol dividen el espacio de configuración en "parches", que en su mayoría no se ven afectados por escalas o valores atípicos.
  - Otras operaciones de preprocesamiento son igualmente aplicables tanto para tipo árbol como para no tipo árbol.

Unidad 3.

Preprocesamiento de datos de Pandas para una ejecución óptima del modelo

- 3.1. Preprocesamiento de datos
- 3.2. Identificar las propiedades de los datos
- 3.3. Comprobación de datos faltantes
- 3.4. Comprobación y procesamiento de datos duplicados
- 3.5. Ingeniería de características de datos

Escala

- Clasificación de datos
  - Los datos continuos y los datos categóricos son los tipos más básicos de datos estructurados.
  - Los datos continuos incluyen datos de continuidad, como la velocidad y duración del viento, y datos discretos, como la frecuencia de ocurrencia de eventos.
  - Los datos categóricos hacen referencia al nombre de las ciudades de cada país (Washington, Nueva York, Los Ángeles) o al tipo de coche (autobús, taxi, camión).
  - Los datos binarios son un caso especial de tener cualquier valor, como 0 y 1, sí/no o verdadero/falso, entre los tipos categóricos.
  - Entre los tipos categóricos, las calificaciones (1,2,3,4,5) en las que se clasifican los valores dentro de la categoría se denominan datos ordinales.

- ¿Por qué clasificar datos?
  - El software de ciencia de datos como Python utiliza esta información de tipo de datos para el rendimiento computacional.
  - El tipo de datos determina cómo realizar el cálculo relacionado con la variable.
  - Por ejemplo, en R o Python, los datos ordinales se clasifican en factores ordenados y se utilizan para mantener el orden deseado por los usuarios en gráficos, tablas y modelos estadísticos.

Unidad 3.

Preprocesamiento de datos de Pandas para una ejecución óptima del modelo

- 3.1. Preprocesamiento de datos
- 3.2. Identificar las propiedades de los datos
- 3.3. Comprobación de datos faltantes
- 3.4. Comprobación y procesamiento de datos duplicados
- 3.5. Ingeniería de características de datos

Valor faltante

- Valor faltante
  - Los valores faltantes se refieren a datos faltantes y datos vacíos.
  - Los valores faltantes pueden distorsionar los resultados del análisis o provocar errores en los cálculos.
  - En algunos casos, los valores faltantes generados en la variable no están relacionados con otras variables, por lo que los valores faltantes deben eliminarse y reemplazarse según la situación.
  - Suponiendo que cada variable sigue una distribución de probabilidad específica, los parámetros de distribución se estiman y reemplazan.
  - Hay reemplazo de medias, reemplazo de medianas y reemplazo de modas.

- Comprobación de datos faltantes

[1]:
```
1 import pandas as pd
2 df = pd.read_csv('./Mobile_Customers.csv')
3 nan_college = df['college'].value_counts(dropna=False)
4 print(nan_college)
```

```
college
NaN    6988
1.0    5484
0.0    5327
Name: count, dtype: int64
```

Línea 1-1

- Importar library.

Línea 1-2

- Cargue el conjunto de datos de clientes móviles.

- Comprobación de datos faltantes

[1]:
```
1 import pandas as pd
2 df = pd.read_csv('./Mobile_Customers.csv')
3 nan_college = df['college'].value_counts(dropna=False)
4 print(nan_college)
```

```
college
NaN    6988
1.0    5484
0.0    5327
Name: count, dtype: int64
```

Línea 1-3

- Calcule la cantidad de NaN en la columna de la universidad.

- Comprobación de datos faltantes

```
[2]: 1 print(df.head().isnull())
```

```
      id  college  income  overage  leftover  house  init_payment  over_15min  \
0  False    False   False    False     False  False         False       False
1  False    False   False    False     False  False         False       False
2  False     True   False    False     False  False         False       False
3  False     True   False    False     False  False         False       False
4  False     True   False    False     False  False         False       False

   call_duration  satisfaction  usage_level  change_plan  brand_churn
0          False         False        False        False        False
1          False         False        False        False        False
2          False         False        False        False        False
3          False         False        False        False        False
4          False         False        False        False        False
```

Línea 2

- Encuentre los datos que faltan usando el método isnull().

- Comprobación de datos faltantes

```
[3]: 1 print(df.head().notnull())
```

```
     id  college  income  overage  leftover  house  init_payment  over_15min  \
0  True     True    True     True      True   True          True        True
1  True     True    True     True      True   True          True        True
2  True    False    True     True      True   True          True        True
3  True    False    True     True      True   True          True        True
4  True    False    True     True      True   True          True        True

   call_duration  satisfaction  usage_level  change_plan  brand_churn
0           True          True         True         True         True
1           True          True         True         True         True
2           True          True         True         True         True
3           True          True         True         True         True
4           True          True         True         True         True
```

Línea 3

- Encuentre los datos que faltan usando el método notnull().

- Comprobación de datos faltantes

```
[4]: 1 print(df.head().isnull().sum(axis=0))
```

```
id              0
college         3
income          0
overage         0
leftover        0
house           0
init_payment    0
over_15min      0
call_duration   0
satisfaction    0
usage_level     0
change_plan     0
brand_churn     0
dtype: int64
```

Línea 4

- Calcule la cantidad de datos faltantes usando el método isnull().

- Al especificar umbral=15000 en el método dropna() se elimina cualquier columna que no tenga al menos 15 000 valores que no sean NaN.

```
[5]: 1 df_thresh = df.dropna(axis=1, thresh=15000)
     2 print(df_thresh.columns)
```

```
Index(['id', 'income', 'overage', 'leftover', 'house', 'init_payment',
       'over_15min', 'call_duration', 'satisfaction', 'usage_level',
       'change_plan', 'brand_churn'],
      dtype='object')
```

Línea 5

- Elimina cualquier columna que no tenga al menos 15 000 valores que no sean NaN. columna universitaria (10822 valores que no son NaN).

- Limite el subconjunto a la columna "satisfacción".

[6]:
```
1 df_house = df.dropna(subset=['satisfaction'], how='any', axis=0)
2 print(len(df_house))
```

```
16903
```

Línea 6

- Elimine todas las filas que tengan un valor NaN en la columna de satisfacción. columna de satisfacción (896 valores NaN de 17.799).

Reemplazo de valores faltantes

- Reemplazo de los datos faltantes con la media

[7]:
```
1 mean_house = df['house'].mean(axis=0)
2 df['house'] = df['house'].fillna(mean_house)
3 print(df['house'][15:26])
```

```
15    187864.000000
16    281839.000000
17    150852.000000
18    221453.000000
19    281214.000000
20    493036.173661
21    507873.000000
22    403085.000000
23    459225.000000
24    951514.000000
25    414892.000000
Name: house, dtype: float64
```

Línea 7-1

- Calcule la media de la columna de la casa. (Excluidos los valores de NaN).

- Reemplazo de los datos faltantes con la media

```
[7]: 1 mean_house = df['house'].mean(axis=0)
     2 df['house'] = df['house'].fillna(mean_house)
     3 print(df['house'][15:26])
```

```
15    187864.000000
16    281839.000000
17    150852.000000
18    221453.000000
19    281214.000000
20    493036.173661
21    507873.000000
22    403085.000000
23    459225.000000
24    951514.000000
25    414892.000000
Name: house, dtype: float64
```

Línea 7-2

- Complete todos los valores de NaN en la columna de la casa con el valor promedio de la columna de la casa. columna de la casa (213 valores NaN de 17799)

Línea 7-3

- Imprime los 10 datos en la columna de la casa. (En la fila 20, los valores de NaN se reemplazan por la media).

Practica con datos faltantes

- Comprobación de datos faltantes

```
[8]: 1 df = pd.read_csv('./Mobile_Customers.csv')
     2 print(df['change_plan'][865:876])
```

```
865                          no
866    actively_looking_into_it
867                  considering
868    actively_looking_into_it
869                         NaN
870                          no
871                          no
872               never_thought
873               never_thought
874    actively_looking_into_it
875    actively_looking_into_it
Name: change_plan, dtype: object
```

Línea 8-1

- Cargue el conjunto de datos de Clientes móviles.

Línea 8-2

- Imprima los datos NaN de la columna change_plan y la fila 869.

- Reemplazo de los datos faltantes con el modo

[9]:
```
1 most_freq = df['change_plan'].value_counts(dropna=True).idxmax()
2 print(most_freq)
```

```
considering
```

Línea 9

- Verifique el valor que aparece con más frecuencia en la columna change_plan.

- Reemplazo de los datos faltantes con el modo

```
[10]: 1 df['change_plan'] = df['change_plan'].fillna(most_freq)
      2 print(df['change_plan'][865:876])
```

```
865                         no
866    actively_looking_into_it
867                  considering
868    actively_looking_into_it
869                  considering
870                         no
871                         no
872               never_thought
873               never_thought
874    actively_looking_into_it
875    actively_looking_into_it
Name: change_plan, dtype: object
```

Línea 10-1

- Los valores NaN en la columna change_plan se reemplazan con el valor que ocurre con más frecuencia.

Línea 10-2

- Imprima los datos NaN de la fila 869 y la columna change_plan. (El valor NaN se reemplaza por el valor most_freq.). (.)

- Reemplazo de los datos faltantes con el valor de reenvío

```
[11]: 1 df = pd.read_csv('./Mobile_Customers.csv')
      2 df['change_plan'] = df['change_plan'].ffill()
      3 print(df['change_plan'][865:876])
```

```
865                          no
866    actively_looking_into_it
867                  considering
868    actively_looking_into_it
869    actively_looking_into_it
870                          no
871                          no
872               never_thought
873               never_thought
874    actively_looking_into_it
875    actively_looking_into_it
Name: change_plan, dtype: object
```

Línea 11-2

- Cambie el valor de NaN change_plan al valor de reenvío de la fila 869.

Unidad 3.

Preprocesamiento de datos de Pandas para una ejecución óptima del modelo

- 3.1. Preprocesamiento de datos
- 3.2. Identificar las propiedades de los datos
- 3.3. Comprobación de datos faltantes
- 3.4. Comprobación y procesamiento de datos duplicados
- 3.5. Ingeniería de características de datos

Procesamiento de datos duplicados

- Comprobación de datos duplicados

[12]:
```
1 import pandas as pd
2 df = pd.DataFrame({'c1':['a', 'a', 'b', 'a', 'b'],
3                    'c2':[1, 1, 1, 2, 2],
4                    'c3':[1, 1, 2, 2, 2]})
5 print(df)
```

```
  c1  c2  c3
0  a   1   1
1  a   1   1
2  b   1   2
3  a   2   2
4  b   2   2
```



Línea 12

- Crea un DataFrame con datos duplicados.

- Comprobación de datos duplicados

[13]:
```
1 df_dup = df.duplicated()
2 print(df_dup)
```

```
0    False
1     True
2    False
3    False
4    False
dtype: bool
```

Línea 13

- Encuentre los valores duplicados entre toda la fila de datos del DataFrame.

- Los datos de la línea 1 se vuelven Verdaderos porque se superponen con la fila 0 anterior.

```
[14]: 1 col_dup = df['c2'].duplicated()
      2 print(col_dup)
```

```
0    False
1     True
2     True
3    False
4     True
Name: c2, dtype: bool
```

Línea 14

- Encuentre el valor duplicado en los datos de la columna específica del DataFrame.

- Método drop_duplicates()

```
[15]: 1 df2 = df.drop_duplicates()
      2 print(df2)
```

```
  c1  c2  c3
0  a   1   1
2  b   1   2
3  a   2   2
4  b   2   2
```

Línea 15

- Elimine las filas duplicadas del DataFrame..

- Referencia de columna correspondiente a la opción de subconjunto

```
[16]: 1 df3 = df.drop_duplicates(subset=['c2', 'c3'])
      2 print(df3)
```

```
  c1  c2  c3
0  a   1   1
2  b   1   2
3  a   2   2
```

Línea 16

- Elimine filas duplicadas según las columnas c2 y c3.

Unidad 3.

Preprocesamiento de datos de Pandas para una ejecución óptima del modelo

- 3.1. Preprocesamiento de datos
- 3.2. Identificar las propiedades de los datos
- 3.3. Comprobación de datos faltantes
- 3.4. Comprobación y procesamiento de datos duplicados
- 3.5. Ingeniería de características de datos

Ingeniería de características

- Variables derivadas:
  - Cree una o más variables basadas en otras variables.

    Ex De la variable "Fecha", obtenga el "Año", "Mes", "Día", "Día de la semana", etc.

Fecha
2009-8-21
2013-8-27
2019-2-11



Año	Mes	Día	Día laborable
2009	8	21	Viernes
2013	8	27	Martes
2019	2	11	Lunes

- Variables derivadas:
  - Cree una o más variables basadas en otras variables.

Ex De las “coordenadas_X” y “coordenadas_Y”, derive la variable “Distancia”:

Distancia =

coordenada x	coordenada_y
3	4
5	12
9	12



Distancia
5
13
15

- Variables derivadas:
  - Cree una o más variables basadas en otras variables.

Ej

A partir del “Precio” y el “Área(m2)”, deriva la variable “PrecioPorÁrea”:m^2

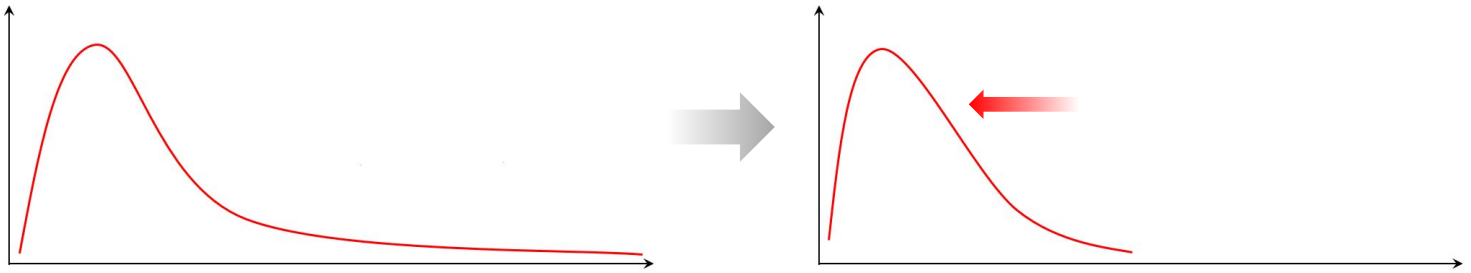
$$PricePerArea = Price / Area$$

Precio	Área
300,000	100
525,000	150
160,000	80



PrecioPorÁrea
3,000
3,500
2,000

- Variables derivadas:
  - Los componentes principales con o sin reducción dimensional pueden considerarse variables derivadas.
  - En general, las coordenadas rotadas pueden considerarse variables derivadas.
  - Podemos aplicar funciones matemáticas como $log()$ a una variable para crear su variable derivada.log



Con log(), podemos mejorar una distribución sesgada positivamente, haciéndola más “normal”.

- Manejo de variables categóricas: variables ficticias
  - Una variable ficticia es una variable derivada que sólo puede tener 0 o 1 como valor.
  - Una variable categórica genera *Category count* – 1 variable ficticia.*Category count*

Ex Variable “género” con dos valores de categoría: “masculino” y “femenino”.

→ → Sólo se requiere una variable ficticia: “Género_masculino”..”

Género		Género_masculino
femenino	⟷	0
masculino		1

- Manejo de variables categóricas: variables ficticias
  - Una variable ficticia es una variable derivada que sólo puede tener 0 o 1 como valor.
  - Una variable categórica genera *Category count* – 1 variable ficticia.*Category count*

Ex Variable “especie” con tres valores de categoría: “setosa”, “versicolor” y “virginica”..”

→ → Sólo se requieren dos variables ficticias: “Species_versicolor” y “Species_virginica”..”

Especies
setosa
versicolor
virginica



Especie_versicolor	Especies_virginica
0	0
1	0
0	1

- Manejo de variables categóricas: variables ficticias
  - Similar a las variables ficticias.
  - En este caso, hay tantas variables ficticias como valores de categoría únicos.

Ex Una codificación activa para una variable que puede tener valores enteros del 0 al 9.

→ Las variables enteras suelen representar una categoría o clase en lugar del valor numérico.

X0	X1	X2	X3	X4	X5	X6	X7	X8	X9
0	0	0	0	0	1	0	0	0	0



- Manejo de variables categóricas: codificación de etiquetas
  - Codifica una variable categórica asignando números enteros a los valores de categoría.
  - Crea una nueva variable numérica (variable derivada).
  - OKEstá bien con algoritmos "similares a un árbol", pero no es seguro con algoritmos "no similares a un árbol".

Ex “Capital” con cinco valores de categoría [“Londres”, “Moscú”, “París”, “Seúl”, “Washington”]
se puede codificar como una variable entera “Capital_int” donde 0 ↔ “Londres”, 1 ↔ “Moscú”
2 ↔ “París”, 3 ↔ “Seúl”, 4 ↔ “Washington”.

- Convertir variables numéricas en categóricas:
  - De una variable numérica podemos derivar una categórica que contenga intervalos como valores.
  - Estos intervalos pueden ser cuantiles o personalizados.
  - A partir de la variable categórica, se pueden generar las variables ficticias correspondientes como de costumbre.

Ex Una variable numérica "Edad" se puede convertir en "Edad_10", "Edad_20", etc.

Edad
8
17
26
63



Edad_10	Edad_20	Edad_30	Edad_40	Edad_50	Edad_60	Edad_70	Edad_80
0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0	0	0	0	0	1	0	0

Hacer coincidir la misma unidad de medida por igual

- Convierta millas por galón en kilómetros por litro (km/L).

[1]:
```
1 import pandas as pd
2 import numpy as np
```

[2]:
```
1 df = pd.read_csv('./auto-mpg.csv', header=None)
2
3 df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
4              'acceleration', 'model year', 'orign', 'name']
5 print(df.head(3))
```

```
    mpg  cylinders  displacement horsepower  weight  acceleration  model year  \
0  18.0          8         307.0        130    3504          12.0          70
1  15.0          8         350.0        165    3693          11.5          70
2  18.0          8         318.0        150    3436          11.0          70

   orign                       name
0      1  chevrolet chevelle malibu
1      1          buick skylark 320
2      1         plymouth satellite
```

Línea 2-1

- Genere df con la función read_csv().

- Convierta millas por galón en kilómetros por litro (km/L).

[1]:
```
1 import pandas as pd
2 import numpy as np
```

[2]:
```
1 df = pd.read_csv('./auto-mpg.csv', header=None)
2
3 df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
4               'acceleration', 'model year', 'orign', 'name']
5 print(df.head(3))
```

```
    mpg  cylinders  displacement horsepower  weight  acceleration  model year  \
0  18.0          8         307.0        130    3504          12.0          70
1  15.0          8         350.0        165    3693          11.5          70
2  18.0          8         318.0        150    3436          11.0          70

   orign                       name
0      1  chevrolet chevelle malibu
1      1          buick skylark 320
2      1         plymouth satellite
```

Línea 2-3

- Designe el nombre de la columna.

- El comando redondear (2) redondea hacia arriba el segundo dígito debajo del punto decimal.

[3]:
```
1 mpg_to_kpl = 1.60934 / 3.78541
2
3 df['kpl'] = df['mpg'] * mpg_to_kpl
4 print(df.head(3))
5 print('\n')
6
7 df['kpl'] = df['kpl'].round(2)
8 print(df.head(3))
```

Línea 3-1

- Convierta millas por galón a kilómetros por litro (mpg_to_kpl ≈ 0,425). (≈ 0.425).

Línea 3-3

- Sume el resultado de multiplicar la columna de mpg por 0,425 a la nueva columna (kpl).).

Línea 3-7

- Redondea la columna kpl a dos decimales.

- El comando redondear (2) redondea hacia arriba el segundo dígito debajo del punto decimal.

```
    mpg  cylinders  displacement horsepower  weight  acceleration  model year  \
0  18.0          8         307.0        130    3504          12.0          70
1  15.0          8         350.0        165    3693          11.5          70
2  18.0          8         318.0        150    3436          11.0          70

   orign                       name       kpl
0      1  chevrolet chevelle malibu  7.652571
1      1           buick skylark 320  6.377143
2      1          plymouth satellite  7.652571


    mpg  cylinders  displacement horsepower  weight  acceleration  model year  \
0  18.0          8         307.0        130    3504          12.0          70
1  15.0          8         350.0        165    3693          11.5          70
2  18.0          8         318.0        150    3436          11.0          70

   orign                       name   kpl
0      1  chevrolet chevelle malibu  7.65
1      1           buick skylark 320  6.38
2      1          plymouth satellite  7.65
```

Conversión de tipo de datos

- Verifique el tipo de datos original.

[4]:
```
1 print(df.dtypes)
2 print('\n')
3
4 print(df['horsepower'].unique())
5 print('\n')
```

```
mpg             float64
cylinders         int64
displacement    float64
horsepower       object
weight            int64
acceleration    float64
model year        int64
orign             int64
name             object
kpl             float64
dtype: object
```

Línea 4-1

- Verifique el tipo de datos de cada columna.

- Verifique el tipo de datos original.

[4]:
```
1 print(df.dtypes)
2 print('\n')
3
4 print(df['horsepower'].unique())
5 print('\n')
```

```
mpg             float64
cylinders         int64
displacement    float64
horsepower       object
weight            int64
acceleration    float64
model year        int64
orign             int64
name             object
kpl             float64
dtype: object
```

Línea 4-4

- Verifique el valor original de la columna de caballos de fuerza.

- Verifique el tipo de datos original.

```
['130' '165' '150' '140' '198' '220' '215' '225' '190' '170' '160' '95'
 '97' '85' '88' '46' '87' '90' '113' '200' '210' '193' '?' '100' '105'
 '175' '153' '180' '110' '72' '86' '70' '76' '65' '69' '60' '80' '54'
 '208' '155' '112' '92' '145' '137' '158' '167' '94' '107' '230' '49' '75'
 '91' '122' '67' '83' '78' '52' '61' '93' '148' '129' '96' '71' '98' '115'
 '53' '81' '79' '120' '152' '102' '108' '68' '58' '149' '89' '63' '48'
 '66' '139' '103' '125' '133' '138' '135' '142' '77' '62' '132' '84' '64'
 '74' '116' '82']
```

- Convierta el tipo de datos entero en un tipo de datos caracter usando el diccionario.

[5]:
```
1 print(df['orign'].unique())
```

```
[1 3 2]
```

[6]:
```
1 df['orign'] = df['orign'].replace({1:'USA', 2:'EU', 3:'JAPAN'})
2
3 print(df['orign'].unique())
4 print(df['orign'].dtypes)
5 print('\n')
```

```
['USA' 'JAPAN' 'EU']
object
```

Línea 5

- Verifique el valor único de la columna de origen.

- Convierta el tipo de datos entero en un tipo de datos caracter usando el diccionario.

[5]:
```
1 print(df['orign'].unique())
```

```
[1 3 2]
```

[6]:
```
1 df['orign'] = df['orign'].replace({1:'USA', 2:'EU', 3:'JAPAN'})
2
3 print(df['orign'].unique())
4 print(df['orign'].dtypes)
5 print('\n')
```

```
['USA' 'JAPAN' 'EU']
object
```

Línea 6-1

- Convierta el tipo de datos entero en un tipo de datos caracter.

Línea 6-3 ~ 6-4

- Verifique el valor original y el tipo de datos de la columna de origen.

- Convierta el string en un tipo categórico.

```
[7]:
1 df['orign'] = df['orign'].astype('category')
2 print(df['orign'].dtypes)
3
4 df['orign'] = df['orign'].astype('str')
5 print(df['orign'].dtypes)
```

```
category
object
```

Línea 7-1

- Convierta el tipo de datos string de la columna de origen en un tipo categórico.

Línea 7-4

- Convierta el tipo categórico nuevamente al tipo string.

- Convierta el string en un tipo categórico.

[8]:
```
1 print(df['model year'].sample(3))
2 df['model year'] = df['model year'].astype('category')
3 print(df['model year'].sample(3))
```

```
197    76
97     73
341    81
Name: model year, dtype: int64
108    73
14     70
132    74
Name: model year, dtype: category
Categories (13, int64): [70, 71, 72, 73, ..., 79, 80, 81, 82]
```

Línea 8

- Convierta el tipo entero de la columna del año del modelo en un tipo categórico.

Conversión de datos categóricos (división de secciones)

- Convierta variables continuas en variables discretas categóricas.

[9]:
```
1 import numpy as np
2 df = pd.read_csv('./auto-mpg.csv', header=None)
3
4 df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
5              'acceleration', 'model year', 'orign', 'name']
6
7 df['horsepower'] = df['horsepower'].replace('?', np.nan)
8 df.dropna(subset=['horsepower'], axis=0, inplace=True)
9 df['horsepower'] = df['horsepower'].astype('float')
```

[10]:
```
1 count, bin_dividers = np.histogram(df['horsepower'], bins=3)
2 print(bin_dividers)
```

```
[ 46.         107.33333333 168.66666667 230.        ]
```

Línea 9-2

- Genere df con la función read_csv().

- Convierta variables continuas en variables discretas categóricas.

[9]:
```
1 import numpy as np
2 df = pd.read_csv('./auto-mpg.csv', header=None)
3
4 df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
5              'acceleration', 'model year', 'orign', 'name']
6
7 df['horsepower'] = df['horsepower'].replace('?', np.nan)
8 df.dropna(subset=['horsepower'], axis=0, inplace=True)
9 df['horsepower'] = df['horsepower'].astype('float')
```

[10]:
```
1 count, bin_dividers = np.histogram(df['horsepower'], bins=3)
2 print(bin_dividers)
```

```
[ 46.         107.33333333 168.66666667 230.        ]
```

Línea 9-4 ~ 9-5

- Designe el nombre de la columna.

Línea 9-7 ~ 9-9

- Encuentre los datos que faltan (“?”) de la columna de caballos de fuerza y conviértalos en un flotador.

- Convierta variables continuas en variables discretas categóricas.

[9]:
```
1 import numpy as np
2 df = pd.read_csv('./auto-mpg.csv', header=None)
3
4 df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
5               'acceleration', 'model year', 'orign', 'name']
6
7 df['horsepower'] = df['horsepower'].replace('?', np.nan)
8 df.dropna(subset=['horsepower'], axis=0, inplace=True)
9 df['horsepower'] = df['horsepower'].astype('float')
```

[10]:
```
1 count, bin_dividers = np.histogram(df['horsepower'], bins=3)
2 print(bin_dividers)
```

```
[ 46.         107.33333333 168.66666667 230.        ]
```

Línea 9-7

- Cambiar "?" a np.nan..

Línea 9-8

- Elimine la fila de datos que falta.

- Convierta variables continuas en variables discretas categóricas.

[9]:
```
1 import numpy as np
2 df = pd.read_csv('./auto-mpg.csv', header=None)
3
4 df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
5               'acceleration', 'model year', 'orign', 'name']
6
7 df['horsepower'] = df['horsepower'].replace('?', np.nan)
8 df.dropna(subset=['horsepower'], axis=0, inplace=True)
9 df['horsepower'] = df['horsepower'].astype('float')
```

[10]:
```
1 count, bin_dividers = np.histogram(df['horsepower'], bins=3)
2 print(bin_dividers)
```

```
[ 46.         107.33333333 168.66666667 230.        ]
```

Línea 9-9

- Convierta el string en un flotador.

Línea 10

- Encuentre una lista de valores límite dividida por tres bins mediante la función np.histogram.

- Utilice la opción include_lowest=True para incluir los valores de límite bajos entre países.

[11]:
```
1 bin_names = ['Low output', 'Normal output', 'High output']
2
3 df['hp_bin'] = pd.cut(x=df['horsepower'],
4                       bins=bin_dividers,
5                       labels=bin_names,
6                       include_lowest=True)
7
8 print(df[['horsepower', 'hp_bin']].head(15))
```

Línea 11-1

- Designe los nombres de los 3 contenedores.

Línea 11-3 ~ 11-6

- Cada dato corresponde a tres contenedores como función pd.cut.

Línea 11-3

- Datos array

- Utilice la opción include_lowest=True para incluir los valores de límite bajos entre países.

[11]:
```
1 bin_names = ['Low output', 'Normal output', 'High output']
2
3 df['hp_bin'] = pd.cut(x=df['horsepower'],
4                       bins=bin_dividers,
5                       labels=bin_names,
6                       include_lowest=True)
7
8 print(df[['horsepower', 'hp_bin']].head(15))
```

Línea 11-4

- Lista de valores límite

Línea 11-5

- Nombres de contenedores

- Utilice la opción include_lowest=True para incluir los valores de límite bajos entre países.

[11]:
```
1 bin_names = ['Low output', 'Normal output', 'High output']
2
3 df['hp_bin'] = pd.cut(x=df['horsepower'],
4                       bins=bin_dividers,
5                       labels=bin_names,
6                       include_lowest=True)
7
8 print(df[['horsepower', 'hp_bin']].head(15))
```

Línea 11-6

- Incluya el primer valor límite.

Línea 11-8

- Imprima las primeras 15 filas de la columna de la glorieta de caballos, columna hp_bin.

- Utilice la opción include_lowest=True para incluir los valores de límite bajos entre países.

```
     horsepower         hp_bin
0         130.0  Normal output
1         165.0  Normal output
2         150.0  Normal output
3         150.0  Normal output
4         140.0  Normal output
5         198.0    High output
6         220.0    High output
7         215.0    High output
8         225.0    High output
9         190.0    High output
10        170.0    High output
11        160.0  Normal output
12        150.0  Normal output
13        225.0    High output
14         95.0     Low output
```

Variable ficticia

- Para utilizar datos categóricos que representen categorías en los algoritmos Machine Learning, conviértalos en variables ficticias representadas por el número 0 o 1.

[12]:

```
1  df = pd.read_csv('./auto-mpg.csv', header=None)
2
3  df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
4                'acceleration', 'model year', 'orign', 'name']
5
6  df['horsepower'] = df['horsepower'].replace('?', np.nan)
7  df.dropna(subset=['horsepower'], axis=0, inplace=True)
8  df['horsepower'] = df['horsepower'].astype('float')
9
10 count, bin_dividers = np.histogram(df['horsepower'], bins=3)
11 bin_names = ['Low output', 'Normal output', 'High output']
```

Línea 12-1

- Genere df con la función read_csv().

- Para utilizar datos categóricos que representen categorías en los algoritmos Machine Learning, conviértalos en variables ficticias representadas por el número 0 o 1.

[12]:
```
1  df = pd.read_csv('./auto-mpg.csv', header=None)
2
3  df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
4                'acceleration', 'model year', 'orign', 'name']
5
6  df['horsepower'] = df['horsepower'].replace('?', np.nan)
7  df.dropna(subset=['horsepower'], axis=0, inplace=True)
8  df['horsepower'] = df['horsepower'].astype('float')
9
10 count, bin_dividers = np.histogram(df['horsepower'], bins=3)
11 bin_names = ['Low output', 'Normal output', 'High output']
```

Línea 12-3 ~ 12-4

- Designe el nombre de la columna.

Línea 12-6 ~ 12-8

- Eliminar los datos faltantes “?” en la columna de caballos de fuerza y convertirlo en un flotador.

- Para utilizar datos categóricos que representen categorías en los algoritmos Machine Learning, conviértalos en variables ficticias representadas por el número 0 o 1.

[12]:
```
1  df = pd.read_csv('./auto-mpg.csv', header=None)
2
3  df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
4                'acceleration', 'model year', 'orign', 'name']
5
6  df['horsepower'] = df['horsepower'].replace('?', np.nan)
7  df.dropna(subset=['horsepower'], axis=0, inplace=True)
8  df['horsepower'] = df['horsepower'].astype('float')
9
10 count, bin_dividers = np.histogram(df['horsepower'], bins=3)
11 bin_names = ['Low output', 'Normal output', 'High output']
```

Línea 12-6

- Cambiar "?" a np.nan..

Línea 12-7

- Elimine la fila de datos que falta.

- Para utilizar datos categóricos que representen categorías en los algoritmos Machine Learning, conviértalos en variables ficticias representadas por el número 0 o 1.

[12]:

```
1  df = pd.read_csv('./auto-mpg.csv', header=None)
2
3  df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
4                'acceleration', 'model year', 'orign', 'name']
5
6  df['horsepower'] = df['horsepower'].replace('?', np.nan)
7  df.dropna(subset=['horsepower'], axis=0, inplace=True)
8  df['horsepower'] = df['horsepower'].astype('float')
9
10 count, bin_dividers = np.histogram(df['horsepower'], bins=3)
11 bin_names = ['Low output', 'Normal output', 'High output']
```

Línea 12-8

- Convierta el string en un flotador.

Línea 12-10

- Encuentre una lista de valores límite dividida por tres contenedores usando np.histogram..

- Para utilizar datos categóricos que representen categorías en los algoritmos Machine Learning, conviértalos en variables ficticias representadas por el número 0 o 1.

[12]:
```
1  df = pd.read_csv('./auto-mpg.csv', header=None)
2
3  df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
4                'acceleration', 'model year', 'orign', 'name']
5
6  df['horsepower'] = df['horsepower'].replace('?', np.nan)
7  df.dropna(subset=['horsepower'], axis=0, inplace=True)
8  df['horsepower'] = df['horsepower'].astype('float')
9
10 count, bin_dividers = np.histogram(df['horsepower'], bins=3)
11 bin_names = ['Low output', 'Normal output', 'High output']
```

Línea 12-11

- Designe el nombre de tres contenedores.

Vector único

- Si se utiliza el método get_dummies(), todos los valores originales de las variables categóricas se convierten en nuevas variables ficticias.

[13]:
```
1 df['hp_bin'] = pd.cut(x=df['horsepower'],
2                       bins=bin_dividers,
3                       labels=bin_names,
4                       include_lowest=True)
5
6 horsepower_dummies = pd.get_dummies(df['hp_bin']).astype('int')
7 print(horsepower_dummies.head(15))
```

Línea 13-1 ~ 13-4

- Asigne cada dato a tres contenedores usando pd.cut..

Línea 13-1

- Datos array

Línea 13-2

- Lista de valores límite

- Si se utiliza el método get_dummies(), todos los valores originales de las variables categóricas se convierten en nuevas variables ficticias.

[13]:
```
1 df['hp_bin'] = pd.cut(x=df['horsepower'],
2                       bins=bin_dividers,
3                       labels=bin_names,
4                       include_lowest=True)
5
6 horsepower_dummies = pd.get_dummies(df['hp_bin']).astype('int')
7 print(horsepower_dummies.head(15))
```

Línea 13-3

- Nombres de contenedores

Línea 13-4

- Incluya el primer valor límite.

Línea 13-6

- Convierta los datos categóricos de la columna hp_bin en variables ficticias.

- Si se utiliza el método get_dummies(), todos los valores originales de las variables categóricas se convierten en nuevas variables ficticias.

	Low output	Normal output	High output
0	0	1	0
1	0	1	0
2	0	1	0
3	0	1	0
4	0	1	0
5	0	0	1
6	0	0	1
7	0	0	1
8	0	0	1
9	0	0	1
10	0	0	1
11	0	1	0
12	0	1	0
13	0	0	1
14	1	0	0

Normalización

- Los DataFrames están normalizados para que no se produzca una disminución del rendimiento debido a la diferencia en el tamaño relativo de los datos numéricos en la columna (cada variable).

[14]:
```
1 print(df.horsepower.describe())
2 print('\n')
3
4 df.horsepower = df.horsepower / abs(df.horsepower.max())
5
6 print(df.horsepower.head())
7 print('\n')
8 print(df.horsepower.describe())
```

Línea 14-1

- Verifique el valor máximo con información de resumen estadístico de la columna de caballos de fuerza.

Línea 14-4

- Todos los datos se dividen y almacenan como el valor absoluto del valor máximo de la columna de caballos de fuerza.

- Los DataFrames están normalizados para que no se produzca una disminución del rendimiento debido a la diferencia en el tamaño relativo de los datos numéricos en la columna (cada variable).

```
count    392.000000
mean     104.469388
std       38.491160
min       46.000000
25%       75.000000
50%       93.500000
75%      126.000000
max      230.000000
Name: horsepower, dtype: float64
```

```
count    392.000000
mean       0.454215
std        0.167353
min        0.200000
25%        0.326087
50%        0.406522
75%        0.547826
max        1.000000
Name: horsepower, dtype: float64
```

```
0    0.565217
1    0.717391
2    0.652174
3    0.652174
4    0.608696
Name: horsepower, dtype: float64
```

- Los DataFrames están normalizados para que no se produzca una disminución del rendimiento debido a la diferencia en el tamaño relativo de los datos numéricos en la columna (cada variable).

[15]:
```
1  print(df.horsepower.describe())
2  print('\n')
3
4  min_x = df.horsepower - df.horsepower.min()
5  min_max = df.horsepower.max() - df.horsepower.min()
6  df.horsepower = min_x / min_max
7
8  print(df.horsepower.head())
9  print('\n')
10 print(df.horsepower.describe())
```

Línea 15-1

- El valor máximo (max) y el valor mínimo (min) se verifican mediante la información de resumen estadístico de la columna de caballos de fuerza.

Línea 15-4 ~ 15-6

- Todos los datos se almacenan como valores normalizados restando el mínimo y dividiendo por el rango.

- Los DataFrames están normalizados para que no se produzca una disminución del rendimiento debido a la diferencia en el tamaño relativo de los datos numéricos en la columna (cada variable).

```
count    392.000000
mean       0.454215
std        0.167353
min        0.200000
25%        0.326087
50%        0.406522
75%        0.547826
max        1.000000
Name: horsepower, dtype: float64
```

```
count    392.000000
mean       0.317768
std        0.209191
min        0.000000
25%        0.157609
50%        0.258152
75%        0.434783
max        1.000000
Name: horsepower, dtype: float64
```

```
0    0.456522
1    0.646739
2    0.565217
3    0.565217
4    0.510870
Name: horsepower, dtype: float64
```

Ejercicio de codificación n.° 0110



Siga los pasos de práctica en el archivo 'ex_0110.ipynb'

Unidad 4.

Visualización de datos para varias escalas de datos

- 4.1. Introducción a la visualización de datos
- 4.2. Gráficos para resumen de datos continuo
- 4.3. Gráficos para resumen de datos categóricos
- 4.4. Visualización para Matplotlib y Pandas
- 4.5. Gráficos avanzados con Seaborn

Explorando datos univariados y datos multivariados

- Se refiere a datos con una variable, divididos en datos cuantitativos y cualitativos.

Variable	Tipo de variable	Tipo de visualización
univariado ((Una variable))	Datos continuos	▪ Histograma ▪ Diagrama de caja ▪ Trama de violín ▪ Curva de densidad del grano
	Datos categóricos (nominal, ordinal)	▪ Gráfico de barras ▪ Gráfico circular

- Se refiere a datos con dos o más variables, divididos en datos cuantitativos y cualitativos.

Variable	Tipo de variable	Tipo de visualización
multivariado ((Dos o más variables))	Datos continuos	▪ Diagrama de dispersión ▪ Trazado lineal ▪ Trama de serie temporal
	Datos categóricos (nominal, ordinal)	▪ Gráfico de mosaico

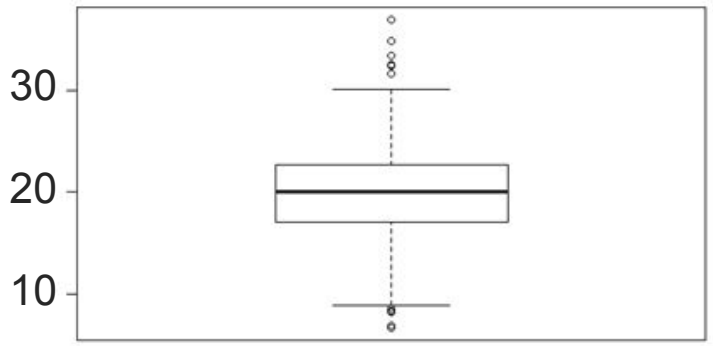
Unidad 4.

Visualización de datos para varias escalas de datos

- 4.1. Introducción a la visualización de datos
- 4.2. Gráficos para resumen de datos continuo
- 4.3. Gráficos para resumen de datos categóricos
- 4.4. Visualización para Matplotlib y Pandas
- 4.5. Gráficos avanzados con Seaborn

Tipos de visualización básicos

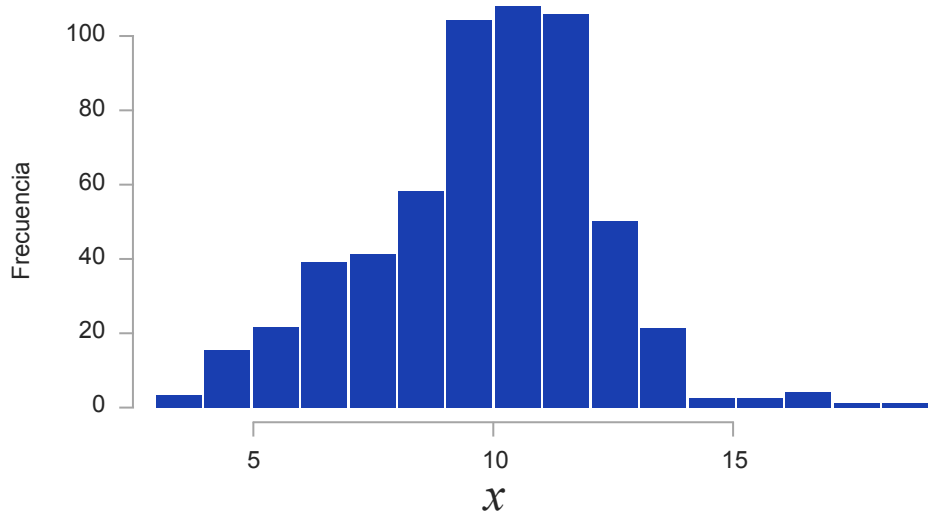
- Visualización univariada:
  - Una variable numérica continua: diagrama de caja



  - Un diagrama de caja se compone de una caja, bigotes, valores atípicos, etc.

4.2. Gráficos para resumen de datos continuo

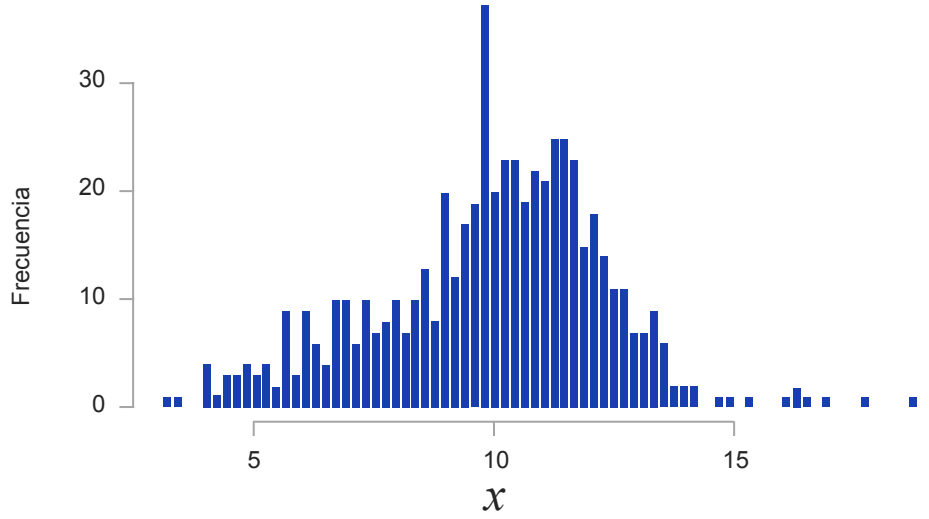
- Visualización univariada:
  - Una variable numérica continua: histograma



  - Muestra las frecuencias absolutas o relativas de cada intervalo.

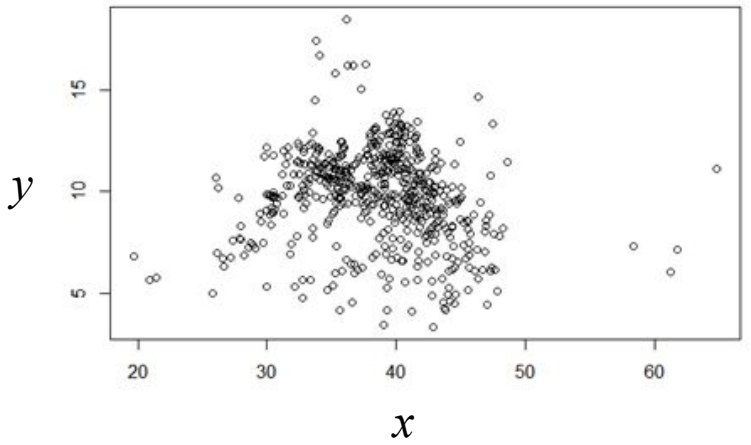
4.2. Gráficos para resumen de datos continuo

- Visualización univariada:
  - Una variable numérica continua: histograma



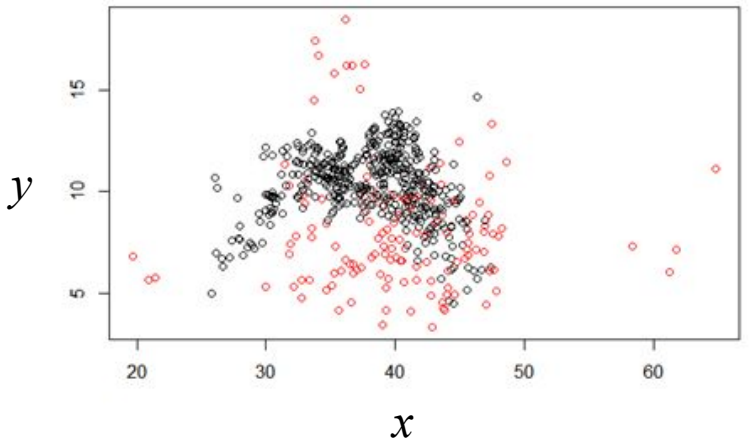
  - El ancho del intervalo se puede ajustar.

- Visualización multivariante:
  - Dos variables numéricas continuas: diagrama de dispersión



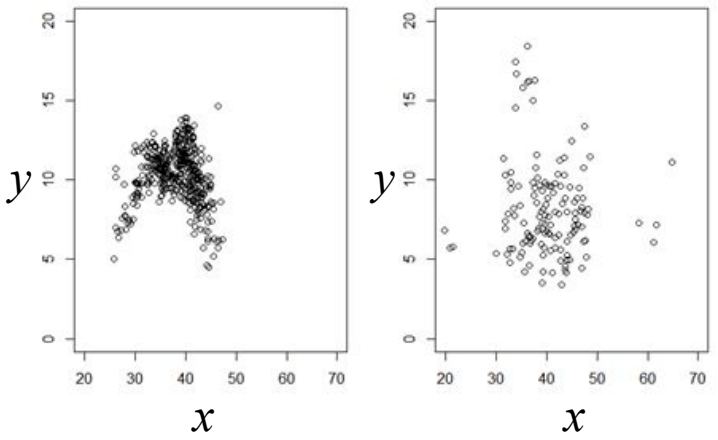
  - Identifique si existe una relación lineal entre las dos variables.

- Visualización multivariante:
  - Dos variables numéricas continuas: diagrama de dispersión



  - Las diferentes categorías pueden indicarse mediante diferentes colores o marcadores (símbolos).

- Visualización multivariante:
  - Dos variables numéricas continuas y una variable categórica: múltiples diagramas de dispersión



  - Se pueden trazar diferentes categorías por separado.
  - Debe asegurarse de que los rangos de los ejes coincidan para una comparación adecuada.

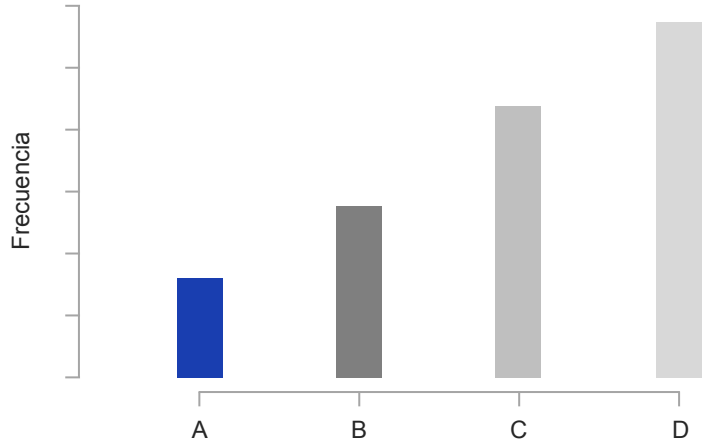
Unidad 4.

Visualización de datos para varias escalas de datos

- 4.1. Introducción a la visualización de datos
- 4.2. Gráficos para resumen de datos continuo
- 4.3. Gráficos para resumen de datos categóricos
- 4.4. Visualización para Matplotlib y Pandas
- 4.5. Gráficos avanzados con Seaborn

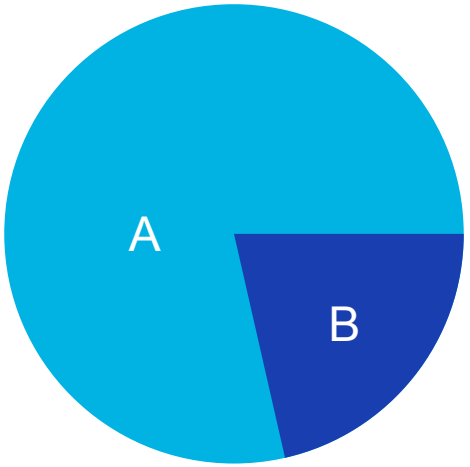
Tipos de visualización básicos

- Visualización univariada:
  - Una variable categórica: diagrama de barras



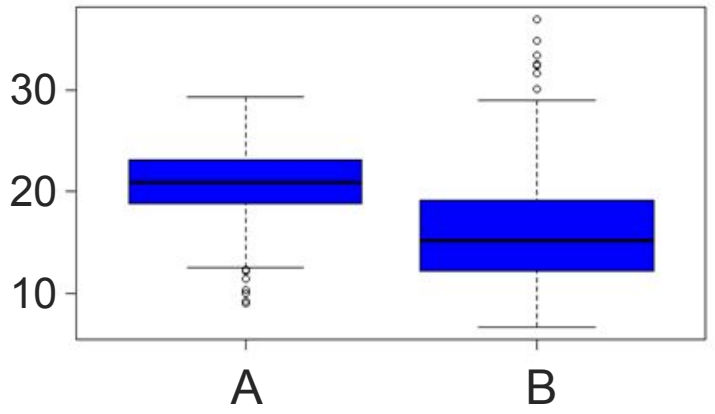
  - Muestra las frecuencias absolutas o relativas de cada categoría (tipo).

- Visualización univariada:
  - Una variable categórica: gráfico circular



  - Muestra las proporciones de cada categoría (tipo).

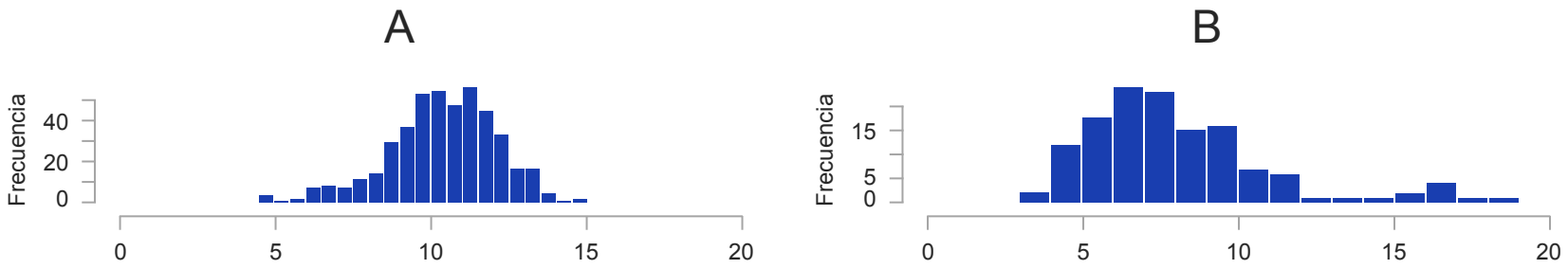
- Visualización bivariada:
  - Una variable numérica continua y una variable categórica: múltiples diagramas de caja



  - El número de categorías (tipos) = el número de diagramas de caja

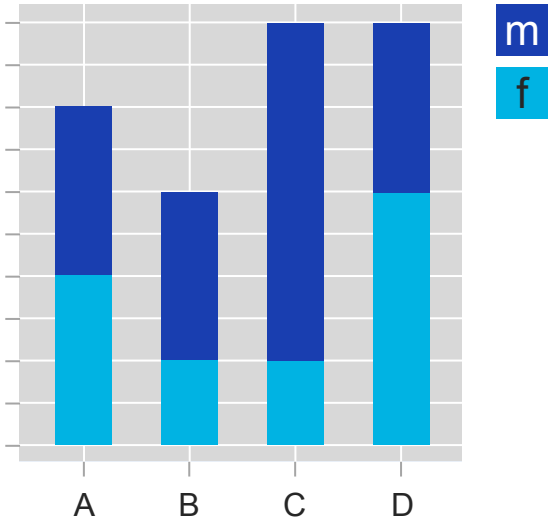
4.3. Gráficos para resumen de datos categóricos

- Visualización bivariada:
  - Una variable numérica continua y una variable categórica: múltiples histogramas



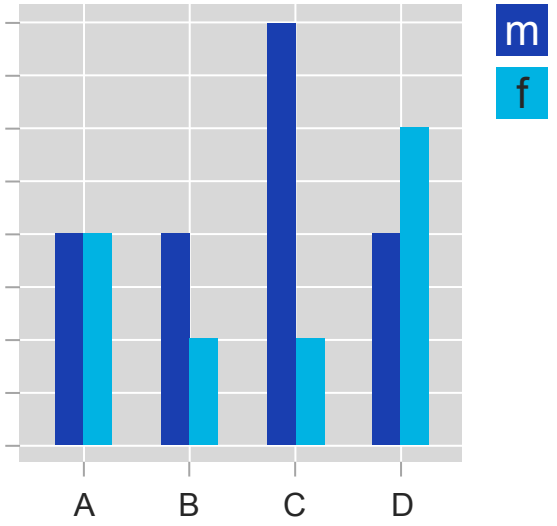
  - El número de categorías (tipos) = el número de histogramas
  - Debe asegurarse de que los rangos de los ejes coincidan para una comparación adecuada.

- Visualización bivariada:
  - Dos variables categóricas: diagrama de barras



  - Utilice colores para distinguir las categorías de la variable secundaria.

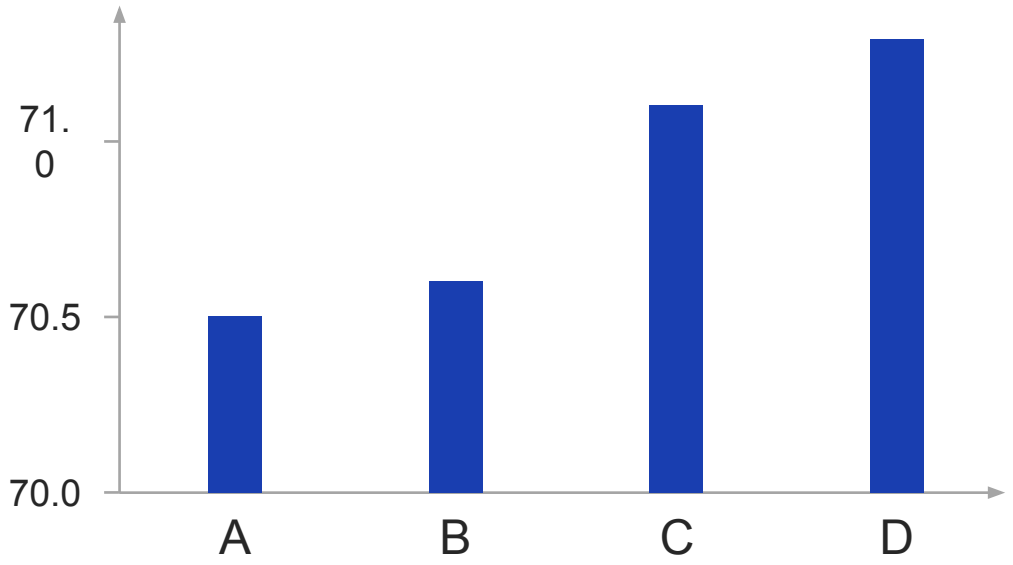
- Visualización bivariada:
  - Dos variables categóricas: diagrama de barras



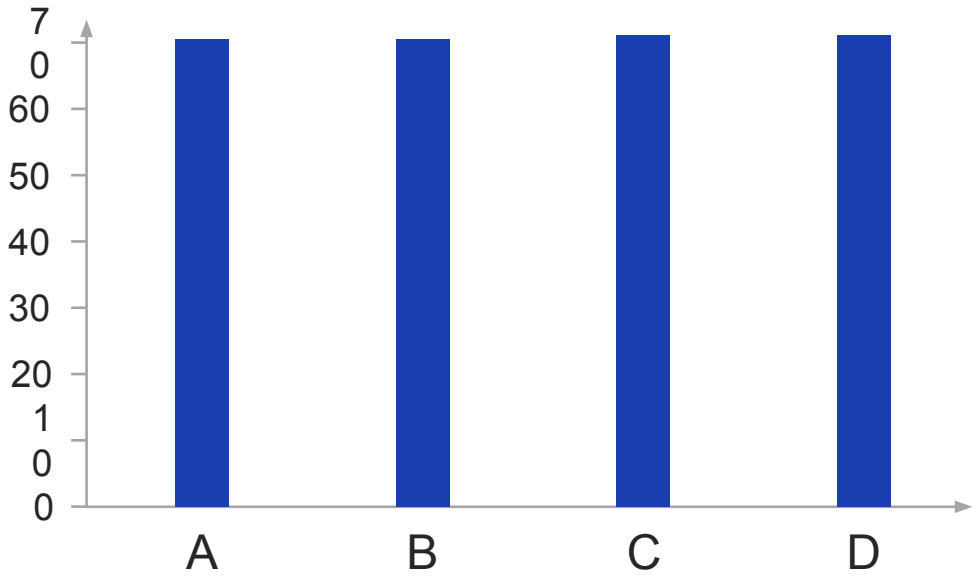
  - Utilice barras de colores y agrupadas para distinguir las categorías de la variable secundaria.

Recomendaciones

- En el siguiente gráfico de barras, ¿puedes ver grandes diferencias entre las categorías?
  - Al parecer, ¿sí?

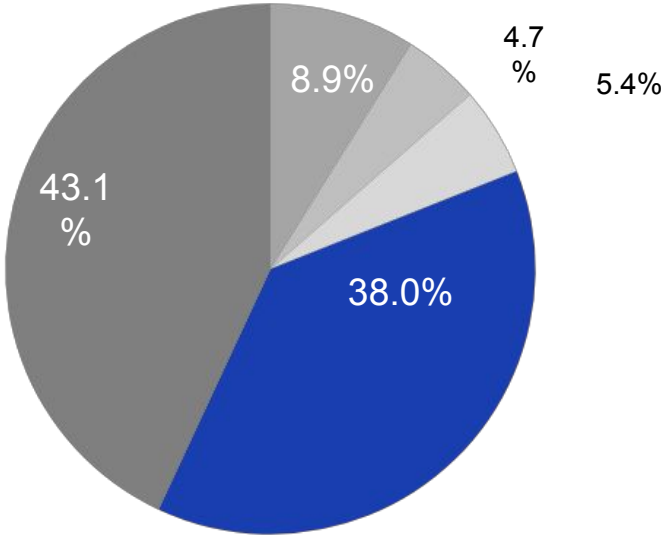


- En el siguiente gráfico de barras, ¿puedes ver una gran diferencia entre las categorías?
  - En este caso donde se muestra el cero vertical, se ve poca diferencia.



4.3. Gráficos para resumen de datos categóricos

- A veces, conviene evitar los efectos 3D.
  - En un gráfico circular 3D, es difícil distinguir las proporciones relativas debido a la perspectiva.



Unidad 4.

Visualización de datos para varias escalas de datos

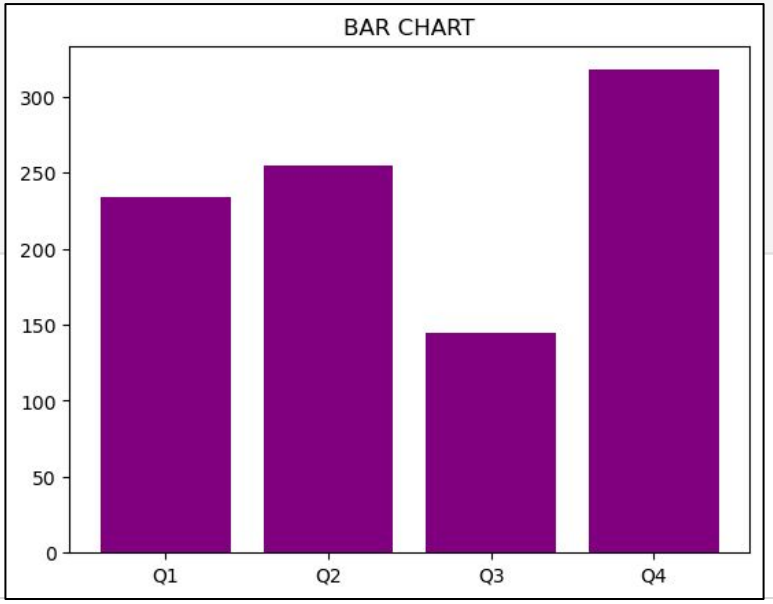
- 4.1. Introducción a la visualización de datos
- 4.2. Gráficos para resumen de datos continuo
- 4.3. Gráficos para resumen de datos categóricos
- 4.4. Visualización para Matplotlib y Pandas
- 4.5. Gráficos avanzados con Seaborn

Visualización básica de Matplotlib

- Trama de barras

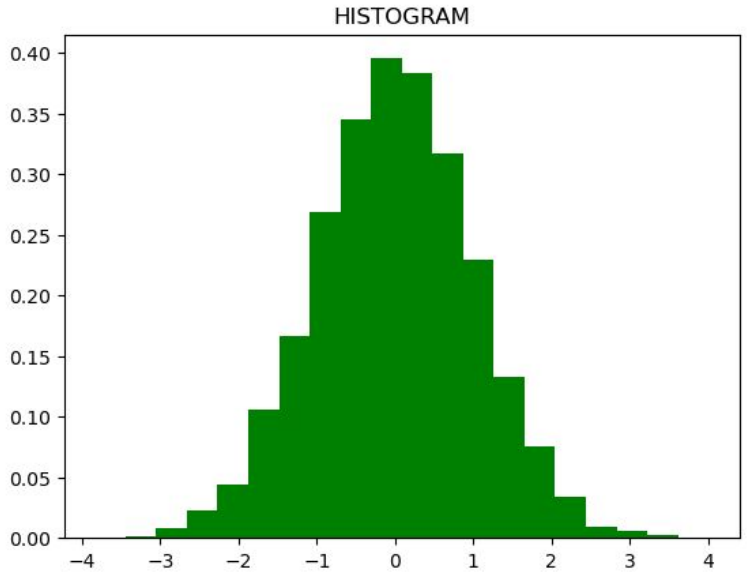
[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

x = np.array(['Q1', 'Q2', 'Q3', 'Q4'])
y = np.array([234.0, 254.7, 144.6, 317.6])
plt.bar(x, y, color='purple')
plt.title('BAR CHART')
plt.show()
```



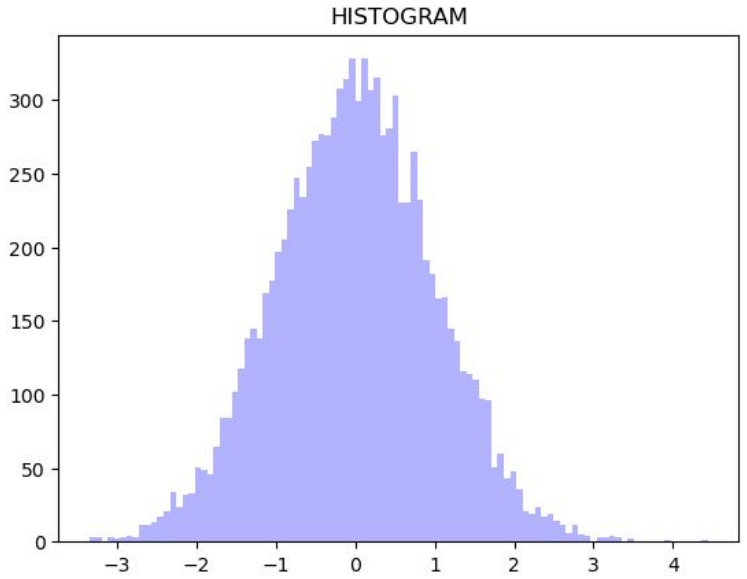
- Histograma

[2]:
```
x = np.random.randn(10000)
plt.hist(x, bins=20, color='green', density=True)
plt.title('HISTOGRAM')
plt.show()
```



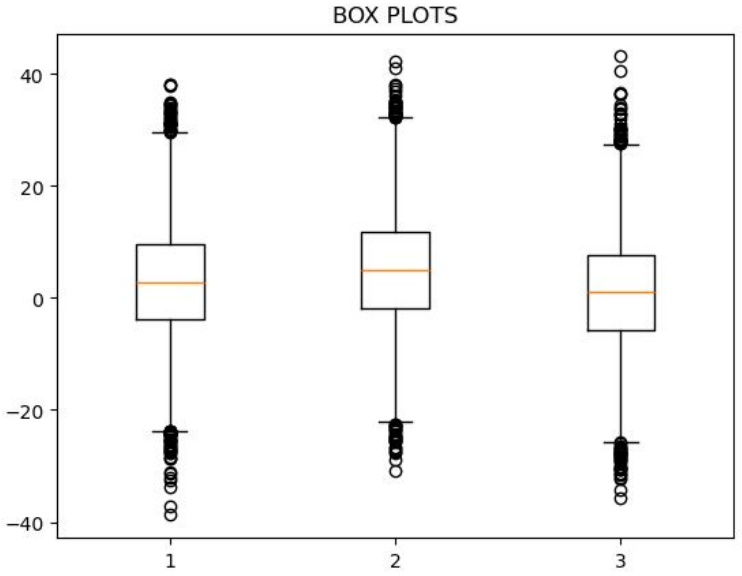
- Histograma

[3]:
```
x = np.random.randn(10000)
plt.hist(x, bins=100, color='blue', density=False, alpha=0.3)
plt.title('HISTOGRAM')
plt.show()
```



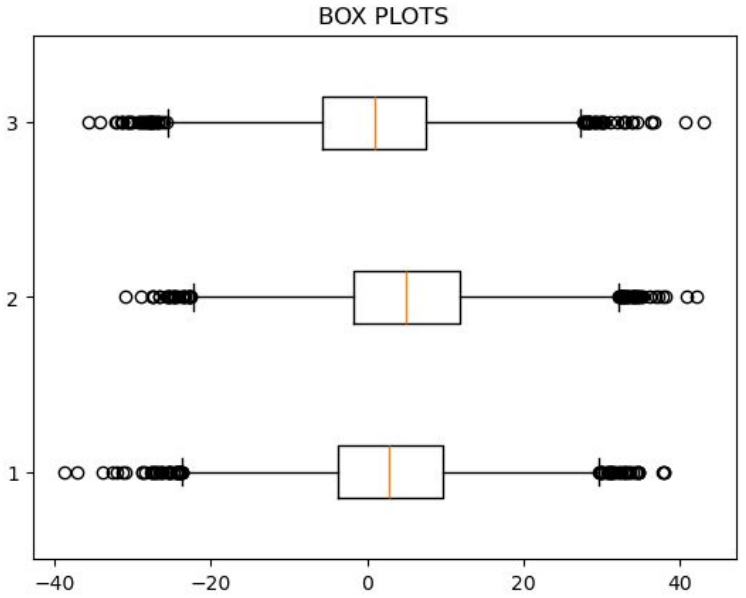
- Múltiples diagramas de caja

[4]:
```
x = np.random.randn(10000)*10 + 3
y = np.random.randn(10000)*10 + 5
z = np.random.randn(10000)*10 + 1
plt.boxplot([x, y, z], notch=False)
plt.title('BOX PLOTS')
plt.show()
```



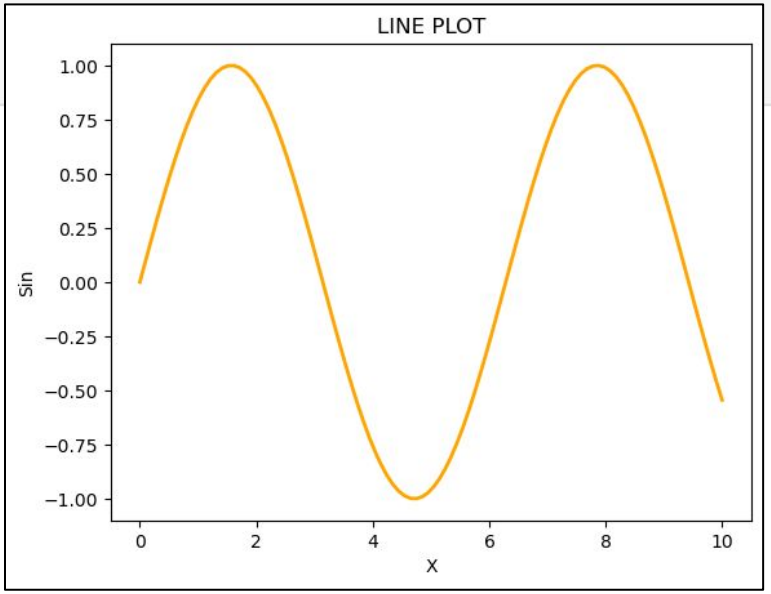
- Múltiples diagramas de caja

```
[5]: plt.boxplot([x, y, z], notch=False, vert=False)
     plt.title('BOX PLOTS')
     plt.show()
```



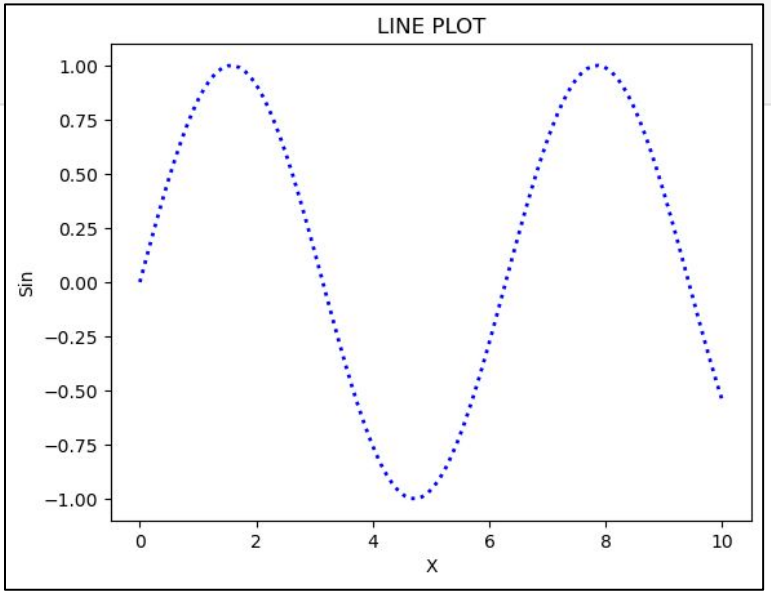
- Trama lineal

[6]:
```
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.plot(x, y, color='orange', linewidth=2)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('LINE PLOT')
plt.show()
```



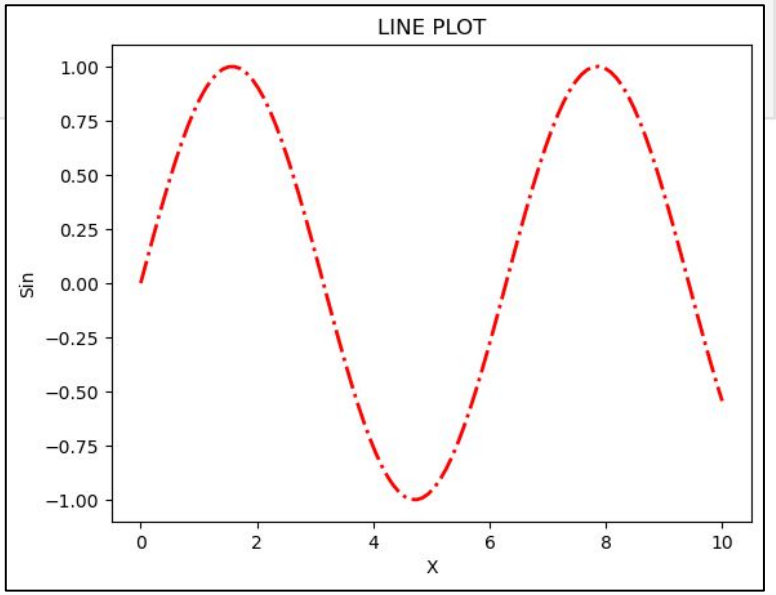
- Trama lineal

[7]:
```
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.plot(x, y, color='blue', linestyle=':', linewidth=2)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('LINE PLOT')
plt.show()
```



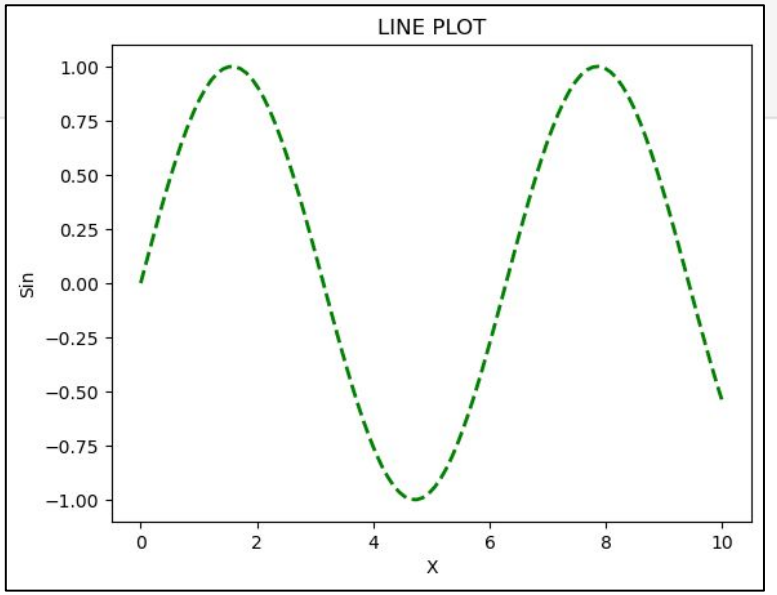
- Trama lineal

```
[8]: x = np.linspace(0, 10, 100)
     y = np.sin(x)
     plt.plot(x, y, color='red', linestyle='-.', linewidth=2)
     plt.xlabel('X')
     plt.ylabel('Sin')
     plt.title('LINE PLOT')
     plt.show()
```



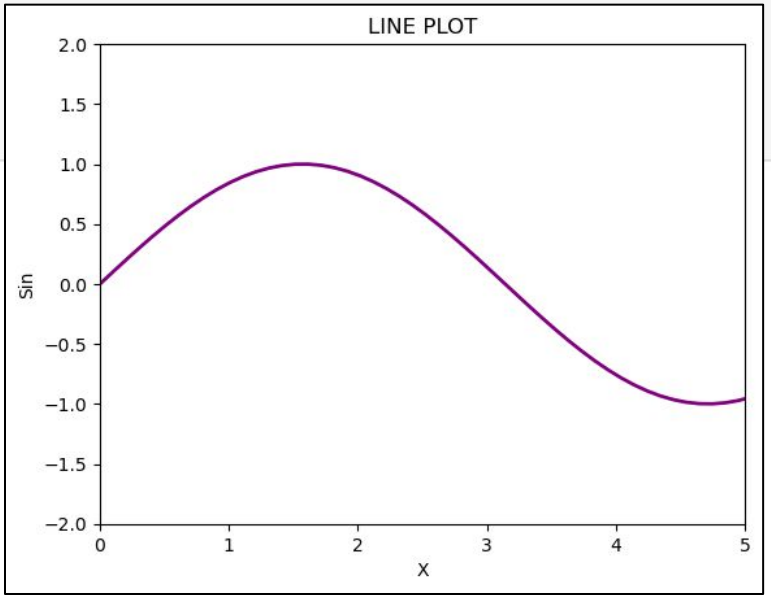
- Trama lineal

[9]:
```
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.plot(x, y, color='green', linestyle='--', linewidth=2)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('LINE PLOT')
plt.show()
```



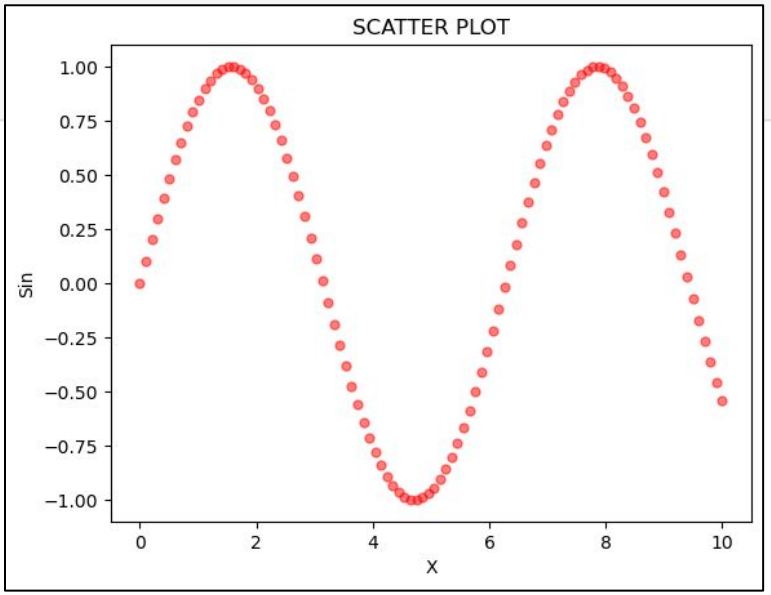
- Trama lineal

```
[10]: x = np.linspace(0, 10, 100)
      y = np.sin(x)
      plt.plot(x, y, color='purple', linestyle='-', linewidth=2)
      plt.xlabel('X')
      plt.ylabel('Sin')
      plt.title('LINE PLOT')
      plt.xlim([0, 5])
      plt.ylim([-2, +2])
      plt.show()
```



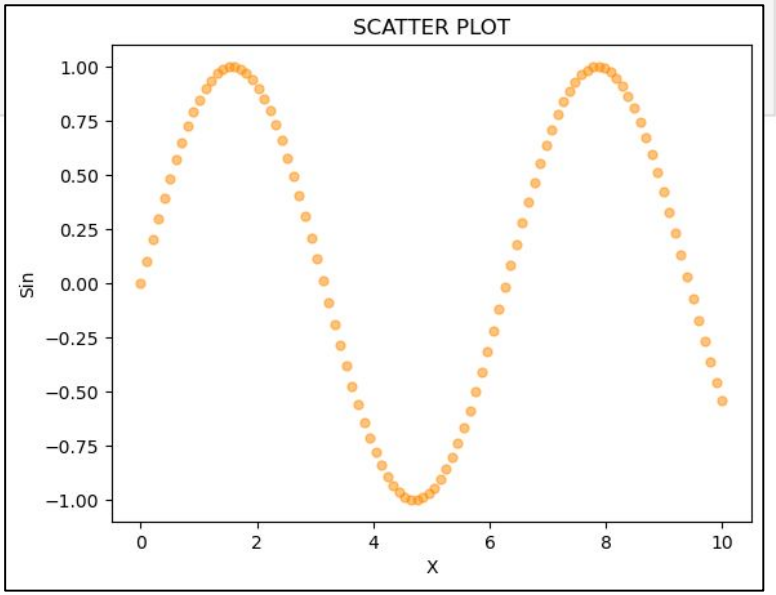
- Gráficos de dispersión con función plot() y estilo de línea ='none':

[11]:
```
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.plot(x, y, color='red', marker='o', linestyle='none', markersize=5, alpha=0.5)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('SCATTER PLOT')
plt.show()
```



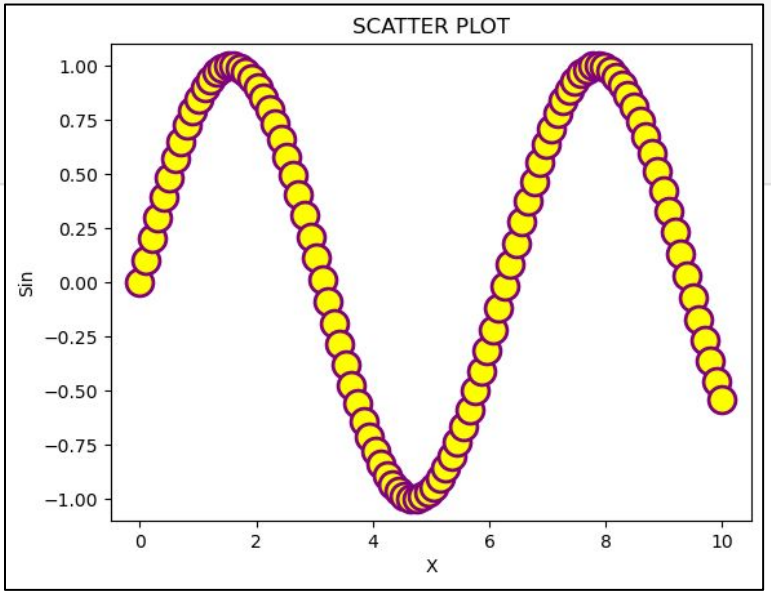
- Gráficos de dispersión con función plot() y estilo de línea ='none':

```
[12]: x = np.linspace(0, 10, 100)
      y = np.sin(x)
      plt.plot(x, y, color='#FF8C00', marker='o', linestyle='none', markersize=5, alpha=0.5)
      plt.xlabel('X')
      plt.ylabel('Sin')
      plt.title('SCATTER PLOT')
      plt.show()
```



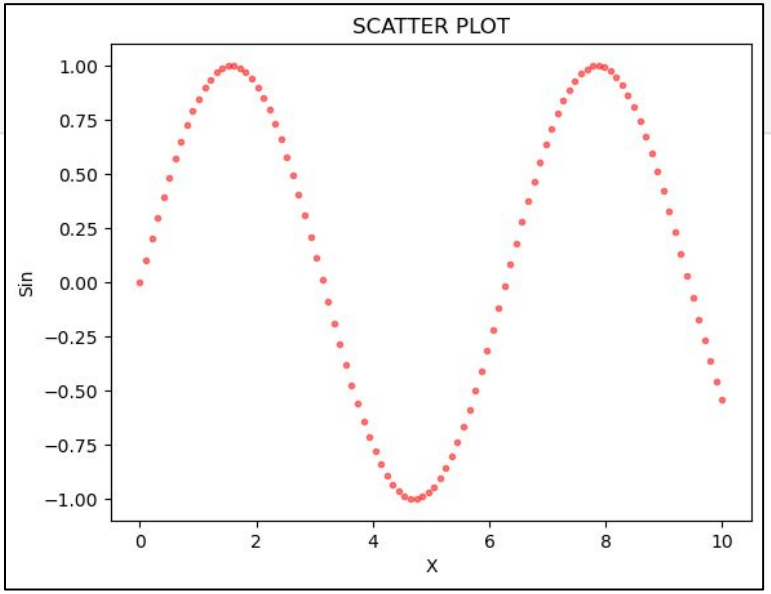
- Gráficos de dispersión con función plot() y estilo de línea ='none':

[13]:
```
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.plot(x, y, marker='o', linestyle='none',
         markersize=15, markerfacecolor='yellow', markeredgecolor='purple',
         markeredgewidth=2)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('SCATTER PLOT')
plt.show()
```



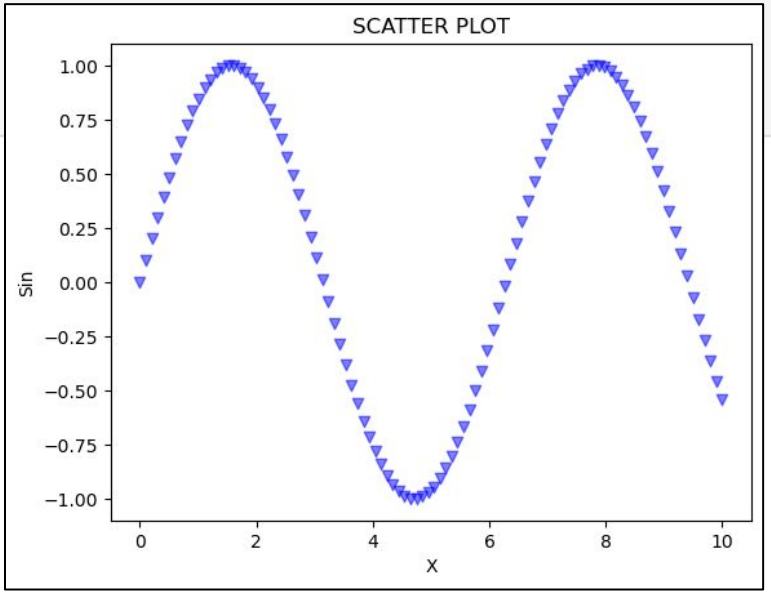
- Gráficos de dispersión con la función scatter():

[14]:
```
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.scatter(x, y, c='red', marker='.', alpha=0.5)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('SCATTER PLOT')
plt.show()
```



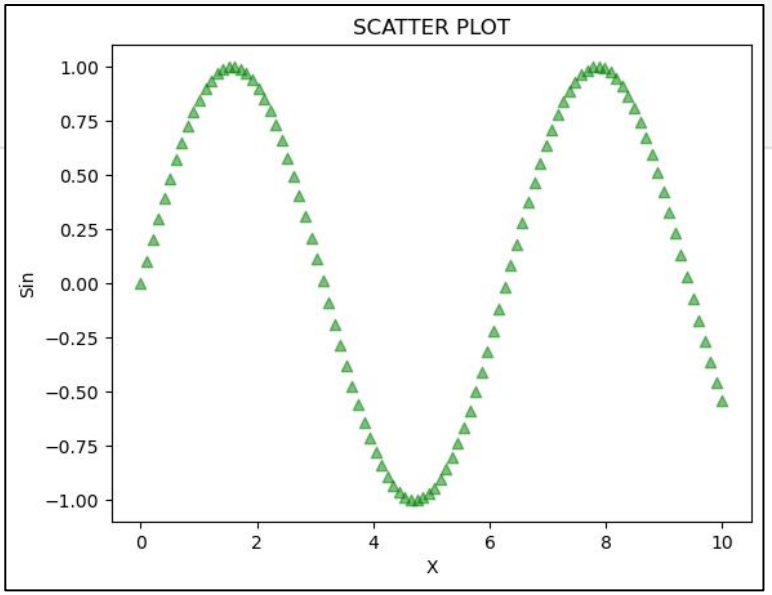
- Gráficos de dispersión con la función scatter():

[15]:
```
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.scatter(x, y, c='blue', marker='v', alpha=0.5)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('SCATTER PLOT')
plt.show()
```



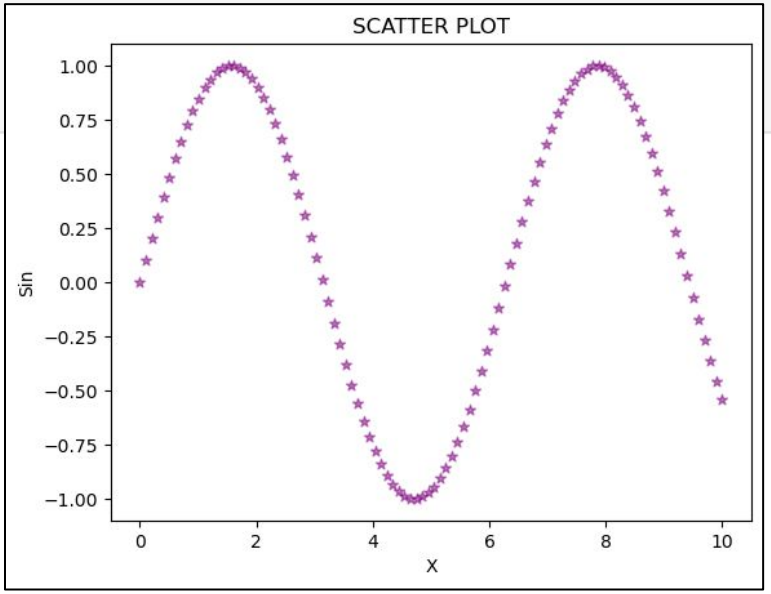
- Gráficos de dispersión con la función scatter():

[16]:
```
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.scatter(x, y, c='green', marker='^', alpha=0.5)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('SCATTER PLOT')
plt.show()
```



- Gráficos de dispersión con la función scatter():

[17]:
```
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.scatter(x, y, c='purple', marker='*', alpha=0.5)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('SCATTER PLOT')
plt.show()
```



- Argumentos de la función plot():

Argumento	Explicación
color	Color
alpha	Transparencia
linewidth	Ancho de línea
linestyle	Estilo de línea
marker	Tipo de marcador
markersize	Tamaño del marcador
markerfacecolor	Color del marcador en el interior.
markeredgecolor	Color del borde del marcador
markeredgewidth	Ancho del borde del marcador

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

- Valores del argumento de estilo de línea:

estilo de línea	Explicación
'none'	Sin linea
':'	Línea de puntos
'--'	Línea discontinua
'-.'	Punto de guión
'-'	linea continua
'steps'	En pasos

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

- Valores del argumento del marcador:

marcador	Explicación
'.'	Punto
','	Píxel
'o'	Círculo
'^'	Triángulo arriba
'v'	Triángulo hacia abajo
's'	Cuadrado
'*'	Estrella
'+'	Signo más
'x'	X caracter
'D'	Diamante
'p'	Pentágono

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

Visualización Matplotlib con objetos

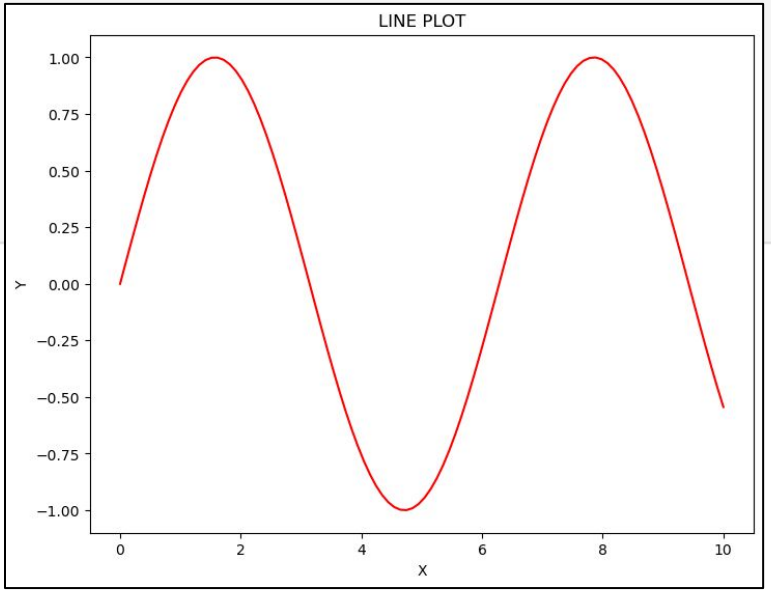
- Importe los módulos necesarios.

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
```

- Visualización con un objeto figura:

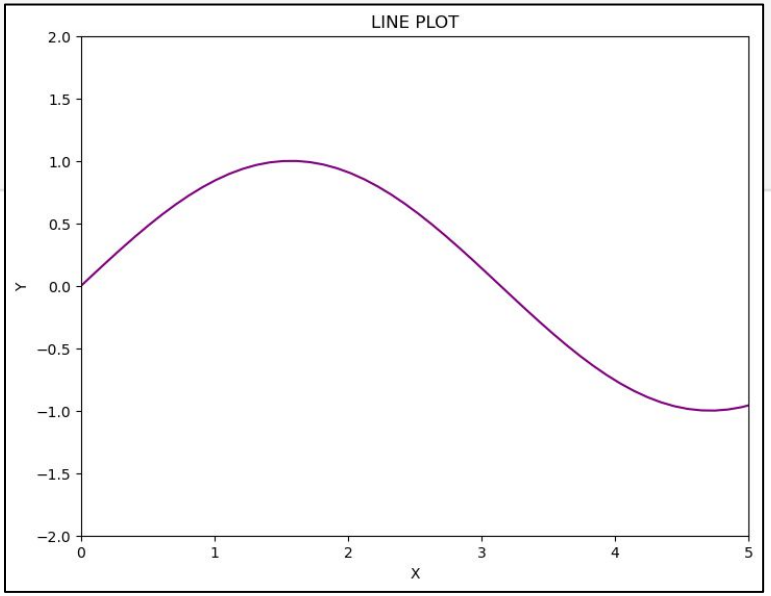
[2]:
```
x = np.linspace(0, 10, 100)
y = np.sin(x)
z = np.cos(x)
```

[3]:
```
fig0=plt.figure()
axes0 = fig0.add_axes([0, 0, 1, 1])
axes0.plot(x, y, color='red', linestyle='-')
axes0.set_xlabel('X')
axes0.set_ylabel('Y')
axes0.set_title('LINE PLOT')
plt.show()
```



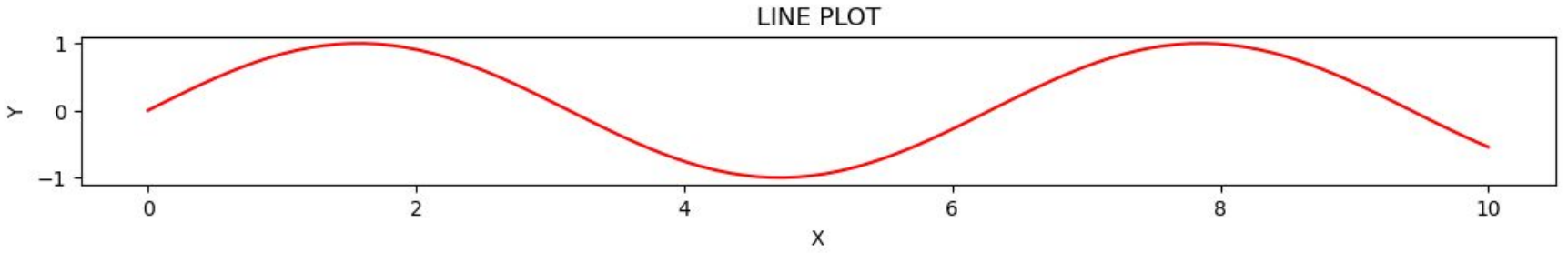
- Visualización con un objeto figura:

[4]:
```
fig0=plt.figure()
axes0 = fig0.add_axes([0, 0, 1, 1])
axes0.plot(x, y, color='purple', linestyle='-')
axes0.set_xlabel('X')
axes0.set_ylabel('Y')
axes0.set_title('LINE PLOT')
axes0.set_xlim([0, 5])
axes0.set_ylim([-2, 2])
plt.show()
```



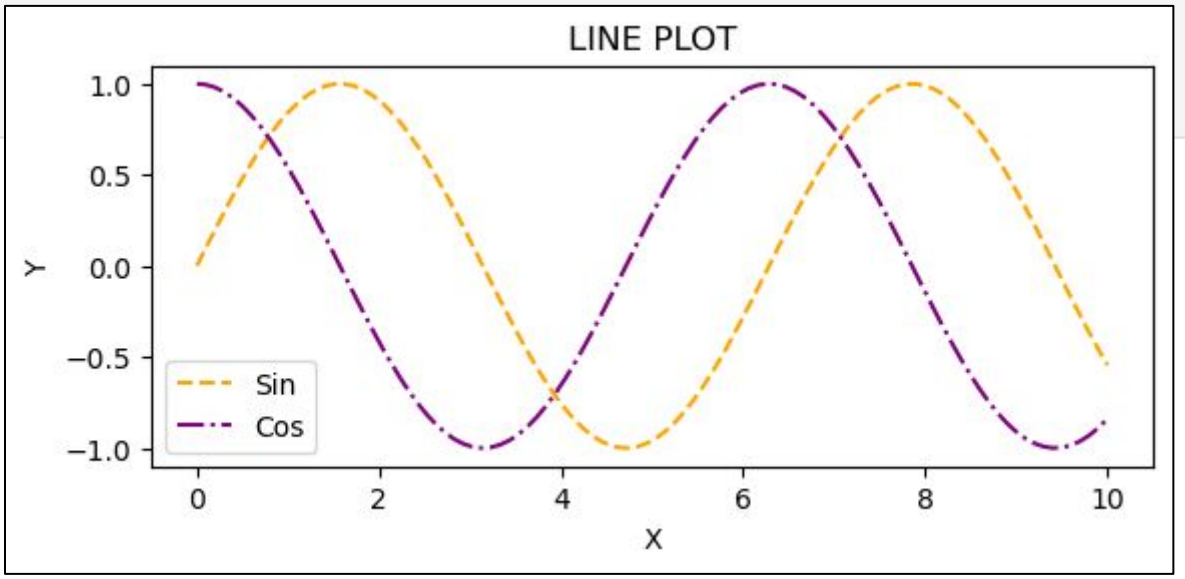
- Visualización con un objeto figura:

[5]:
```
fig0=plt.figure(figsize=(10, 1))          # Width and height specified with figsize.
axes0 = fig0.add_axes([0, 0, 1, 1])       # Left, bottom, width, height.
axes0.plot(x, y, color='red', linestyle='-')
axes0.set_xlabel('X')
axes0.set_ylabel('Y')
axes0.set_title('LINE PLOT')
plt.show()
```



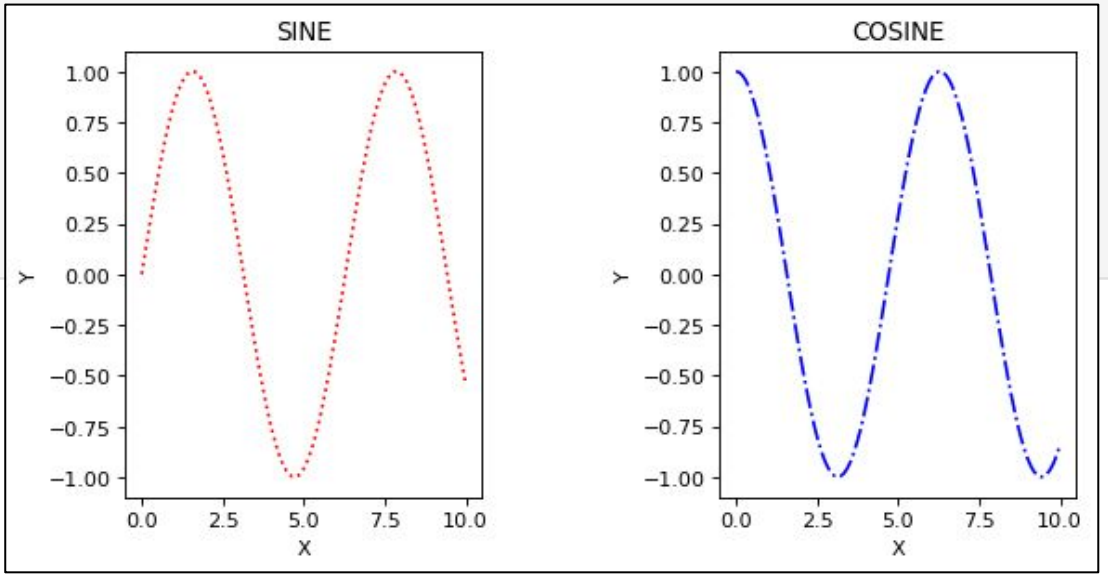
- Múltiples parcelas dentro de los mismos ejes:

[6]:
```
fig0=plt.figure(figsize=(5, 2), dpi=100)    # Width, height and DPI setting.
axes0 = fig0.add_axes([0, 0, 1, 1])         # Left, bottom, width, height.
axes0.plot(x, y, color='orange', linestyle='--', label='Sin')   # 'label' for the legend.
axes0.plot(x, z, color='purple', linestyle='-.', label='Cos')   # 'label' for the legend.
axes0.legend(loc=0)                         # Legend to be located at the top-right corner.
axes0.set_xlabel('X')
axes0.set_ylabel('Y')
axes0.set_title('LINE PLOT')
plt.show()
```



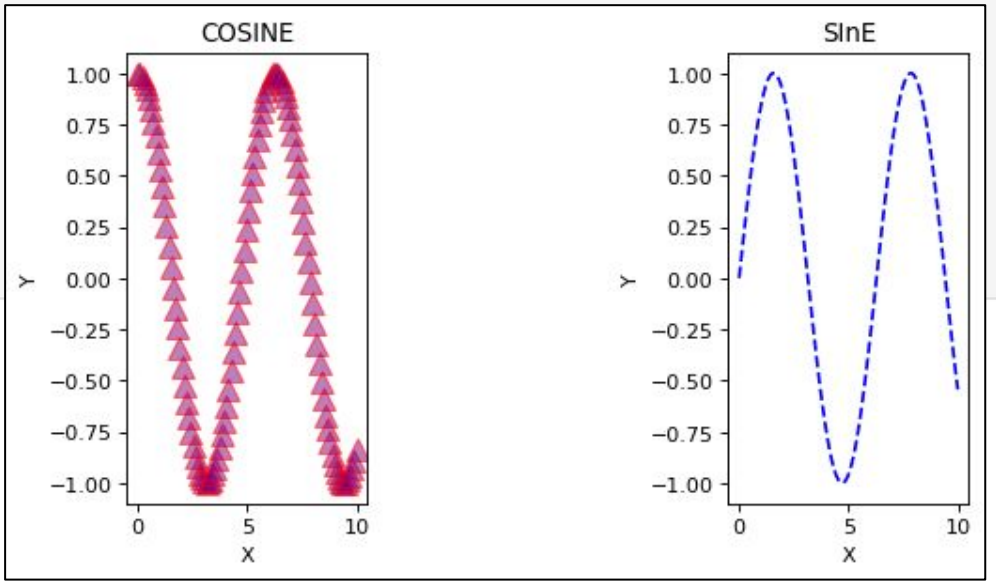
- Múltiples gráficos en ejes separados:

[7]:
```
fig0=plt.figure(figsize=(8, 3), dpi=80)
axes1 = fig0.add_axes([0, 0, 0.3, 1])
axes2 = fig0.add_axes([0.5, 0, 0.3, 1])
axes1.plot(x, y, color='red', linestyle=':')
axes2.plot(x, z, color='blue', linestyle='-.')
axes1.set_xlabel('X')
axes1.set_ylabel('Y')
axes1.set_title('SINE')
axes2.set_xlabel('X')
axes2.set_ylabel('Y')
axes2.set_title('COSINE')
plt.show()
```



- Múltiples gráficos en ejes separados:

[8]:
```
fig0=plt.figure(figsize=(8, 3), dpi=80)
axes1 = fig0.add_axes([0, 0, 0.2, 1])
axes2 = fig0.add_axes([0.5, 0, 0.2, 1])
axes1.plot(x, z, linestyle='none', marker='^', markersize=10,
           markerfacecolor='purple', markeredgecolor='red', alpha=0.5)
axes2.plot(x, y, color='blue', linestyle='--')
axes1.set_xlabel('X')
axes1.set_ylabel('Y')
axes1.set_title('COSINE')
axes2.set_xlabel('X')
axes2.set_ylabel('Y')
axes2.set_title('SInE')
plt.show()
```

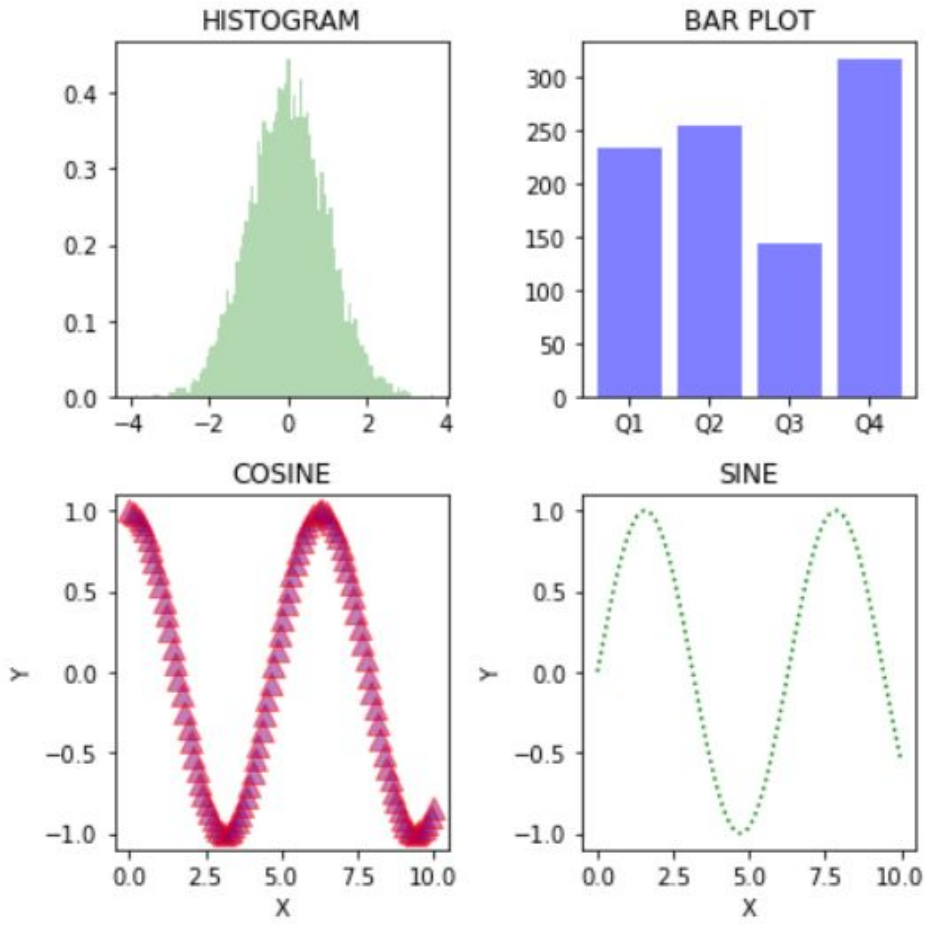


- Múltiples gráficos en un array de ejes:

[9]:
```
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(6,6))
# (0,0)
c = np.random.randn(1000)
axes[0,0].hist(c, bins=100, color='green', density=True, alpha=0.3)
axes[0,0].set_title('HISTOGRAM')
#(1,0)
axes[1,0].plot(x, z, linestyle='none', marker='^', markersize=10,
               markerfacecolor='purple', markeredgecolor='red', alpha=0.5)
axes[1,0].set_xlabel('X')
axes[1,0].set_ylabel('Y')
axes[1,0].set_title('COSINE')
#(0,1)
a = np.array(['Q1', 'Q2', 'Q3', 'Q4'])
b = np.array([234.0, 254.7, 144.6, 317.6])
axes[0,1].bar(a, b, color='blue', alpha=0.5)
axes[0,1].set_title('BAR PLOT')
#(1,1)
axes[1,1].plot(x, y, color='green', linestyle=':')
axes[1,1].set_xlabel('X')
axes[1,1].set_ylabel('Y')
axes[1,1].set_title('SINE')

plt.tight_layout()
plt.show()
```

- Múltiples gráficos en un array de ejes:



Visualización de pandas

- Importar módulos y datos.

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
```

[2]:
```
df = pd.read_csv('data_iris.csv', header='infer')
```

[3]:
```
df.shape
```

[3]:
```
(150, 5)
```

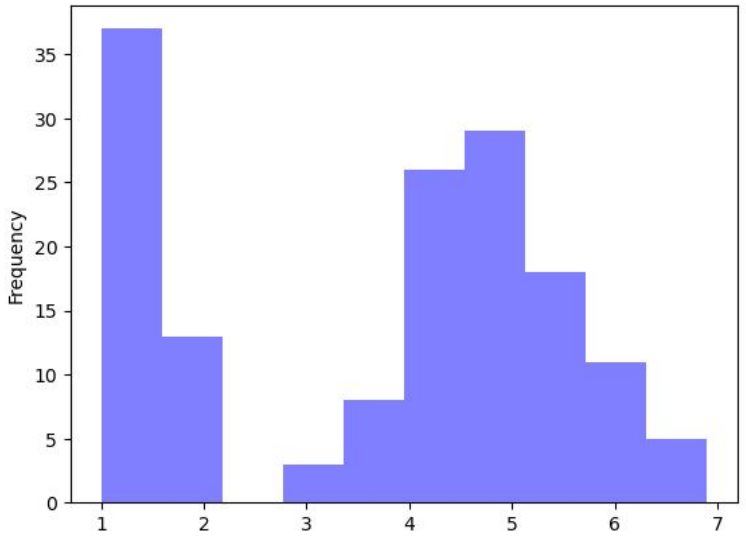
[4]:
```
df.head(5)
```

[4]:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

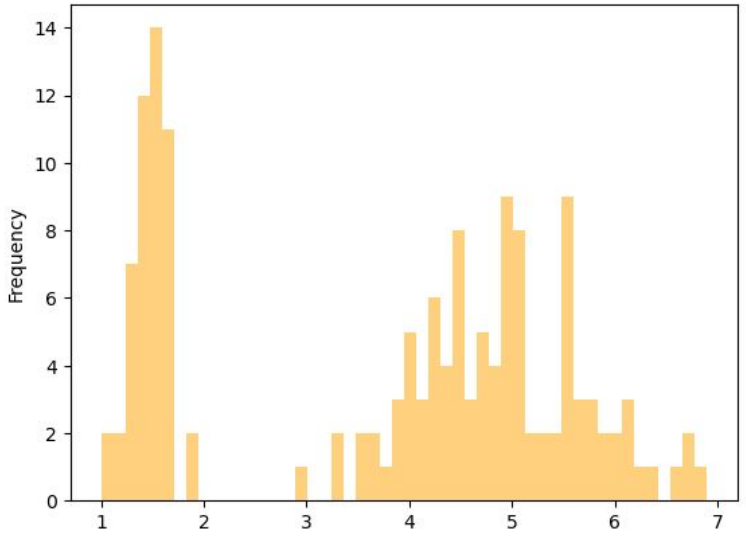
- Histograma:

```
[5]: df.loc[:, 'Petal.Length'].plot.hist(color='blue', alpha=0.5)
     plt.show()
```



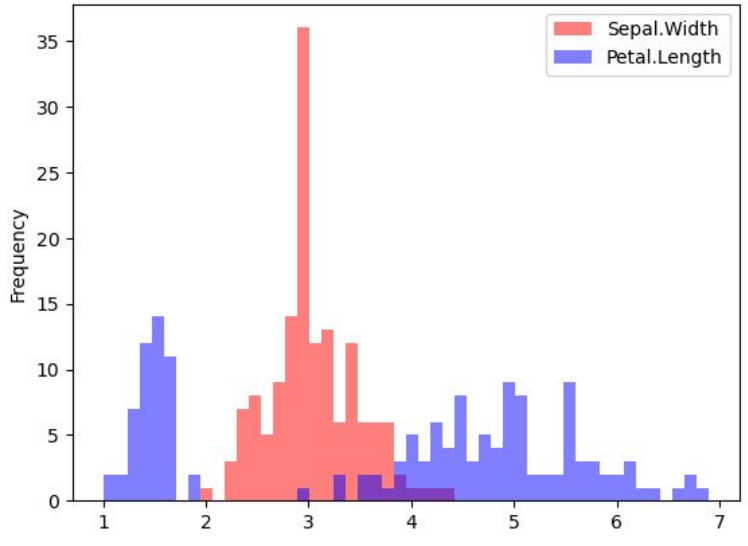
- Histograma:

[6]:
```
df.loc[:, 'Petal.Length'].plot.hist(bins=50, color='orange', alpha=0.5)
plt.show()
```



- Histograma:

```
[7]: df.loc[:, ['Sepal.Width', 'Petal.Length']].plot.hist(bins=50, color=['red', 'blue'], alpha=0.5)
     plt.show()
```



- Trama de barras:

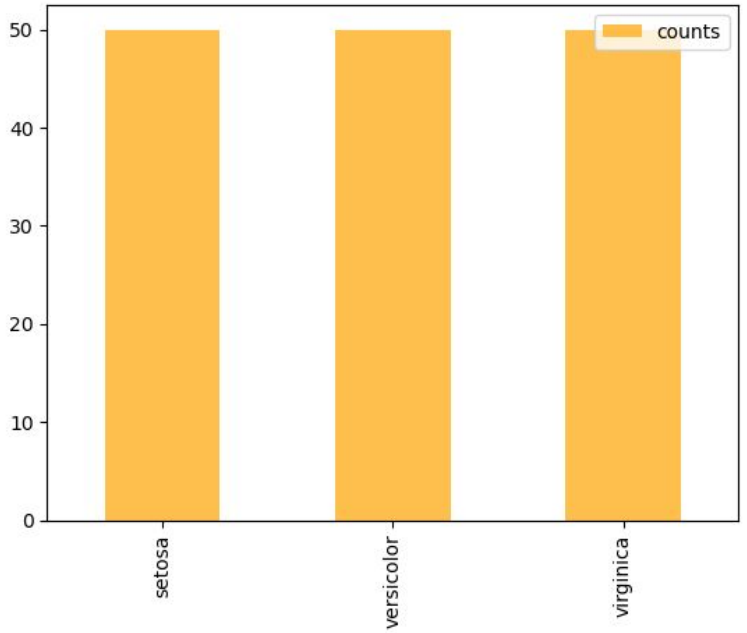
[8]:
```
frequencies = df.Species.value_counts()
my_counts = list(frequencies.values)
my_labels = list(frequencies.index)
df2 = pd.DataFrame({'counts': my_counts}, index=my_labels)
df2
```

[8]:

	counts
setosa	50
versicolor	50
virginica	50

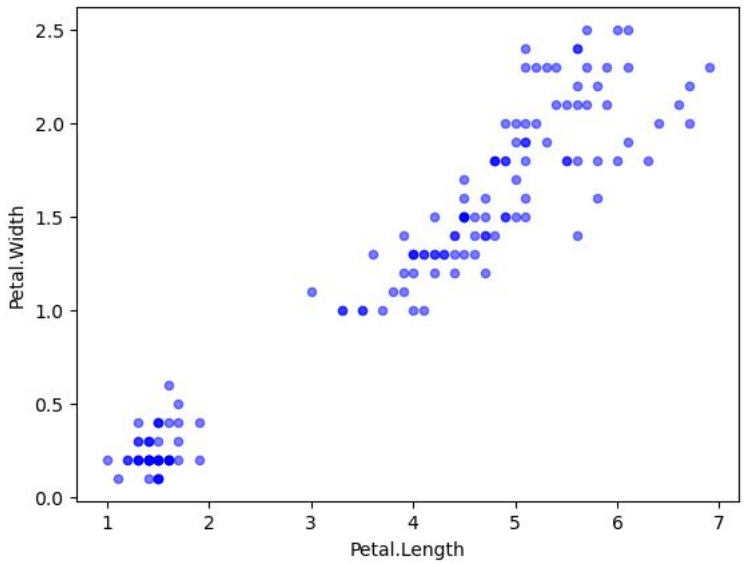
- Trama de barras:

[9]:
```
df2.plot.bar(color='orange', alpha=0.7)
plt.show()
```



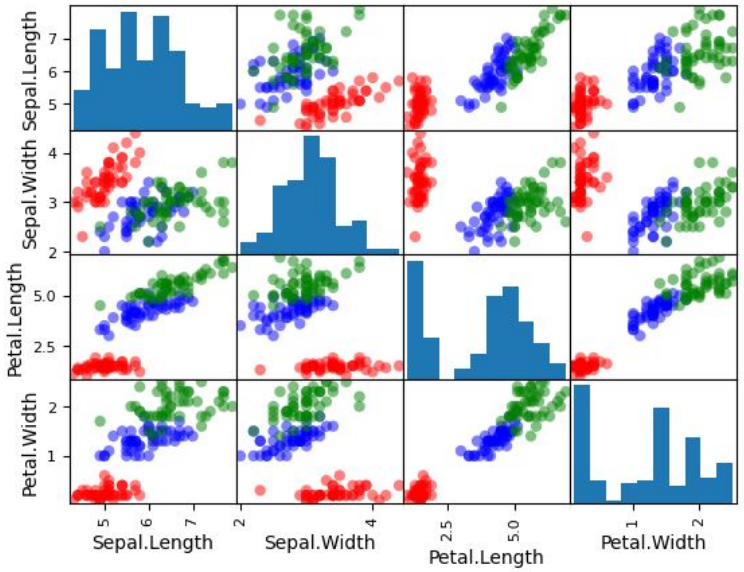
- Diagrama de dispersión único:

[10]:
```
df.plot.scatter(x='Petal.Length', y='Petal.Width', color='blue', alpha=0.5, marker='o')
plt.show()
```



- Un array de diagramas de dispersión:

[11]:
```
my_cols_dict = {'setosa':'red', 'virginica':'green', 'versicolor':'blue'}
my_cols = df['Species'].apply(lambda x: my_cols_dict[x])
pd.plotting.scatter_matrix(df, c=my_cols, marker='o', alpha=0.5)
plt.show()
```



Ejercicio de codificación n.° 01110111



Siga los pasos de práctica en el archivo 'ex_0111.ipynb'

Ejercicio de codificación n.° 0112



Siga los pasos de práctica en el archivo 'ex_0112.ipynb'

Unidad 4.

Visualización de datos para varias escalas de datos

- 4.1. Introducción a la visualización de datos
- 4.2. Gráficos para resumen de datos continuo
- 4.3. Gráficos para resumen de datos categóricos
- 4.4. Visualización para Matplotlib y Pandas
- 4.5. Gráficos avanzados con Seaborn

Visualización Seaborn library

- Qué proporciona el Seaborn library:
  - Conjunto de datos interno: load_dataset()
  - Tipos de gráficos básicos: distplot(), jointplot(), kdeplot(), rugplot(), barplot(), countplot(), etc.
  - arrays: pairplot(), PairGrid(), FacetGrid(), etc.
  - Con línea de regresión (tendencia): lmplot(), jointplot(), etc.
  - Tipos de gráficos especiales: heatmap(), clustermap(), etc.
  - Tipos de gráficos aplicados: violinplot(), swarmplot(), stripplot(), etc.

- Instale el Seaborn library.
  - Recuerde ejecutar primero el comando “!pip install seaborn” para la instalación de Seaborn library.
  - Importar módulos.

[1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import os
import warnings
%matplotlib inline
warnings.filterwarnings('ignore')    # Turn off the warning.
```

- Importar conjunto de datos.
  - Lea el conjunto de datos 'mpg' del library.

```
[2]: dat = sns.load_dataset('mpg')

[3]: dat.shape

[3]: (398, 9)

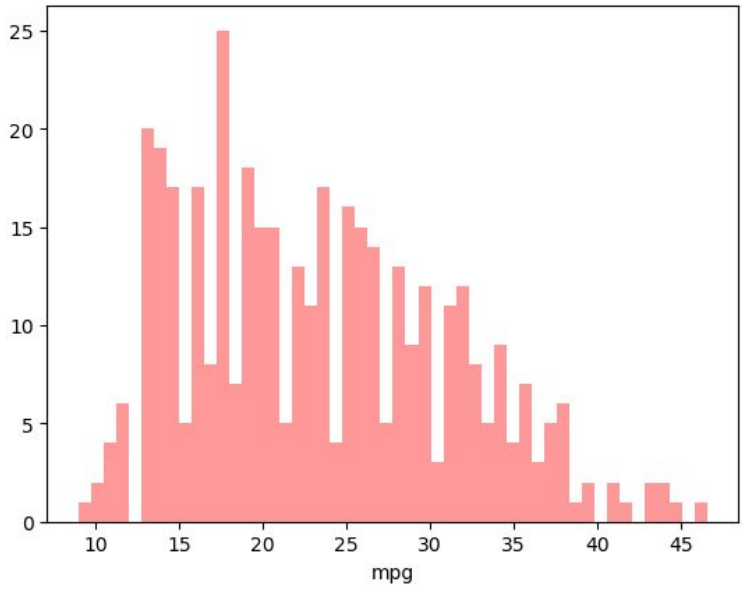
[4]: dat.head(5)
```

[4]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name
0	18.0	8	307.0	130.0	3504	12.0	70	usa	chevrolet chevelle malibu
1	15.0	8	350.0	165.0	3693	11.5	70	usa	buick skylark 320
2	18.0	8	318.0	150.0	3436	11.0	70	usa	plymouth satellite
3	16.0	8	304.0	150.0	3433	12.0	70	usa	amc rebel sst
4	17.0	8	302.0	140.0	3449	10.5	70	usa	ford torino

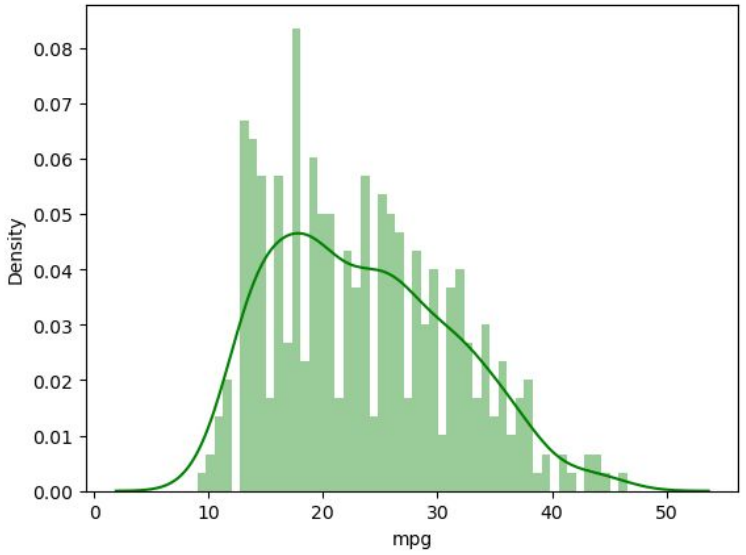
- Histograma:

```
[5]: sns.distplot(dat.mpg, kde=False, rug=False, bins=50, color='red')
     plt.show()
```



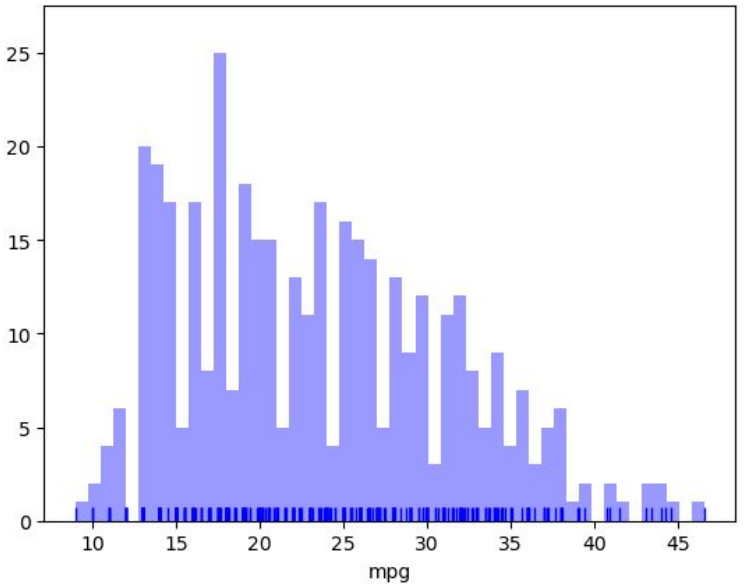
- Histograma + KDE:

[6]:
```
sns.distplot(dat.mpg, kde=True, rug=False, bins=50, color='green')
plt.show()
```



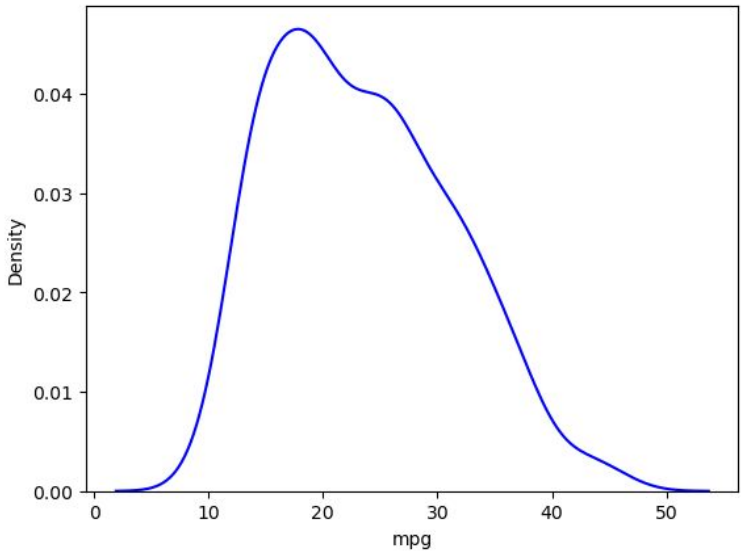
- Histograma + Alfombra:

[7]:
```
sns.distplot(dat.mpg, kde=False, rug=True, bins=50, color='blue')
plt.show()
```



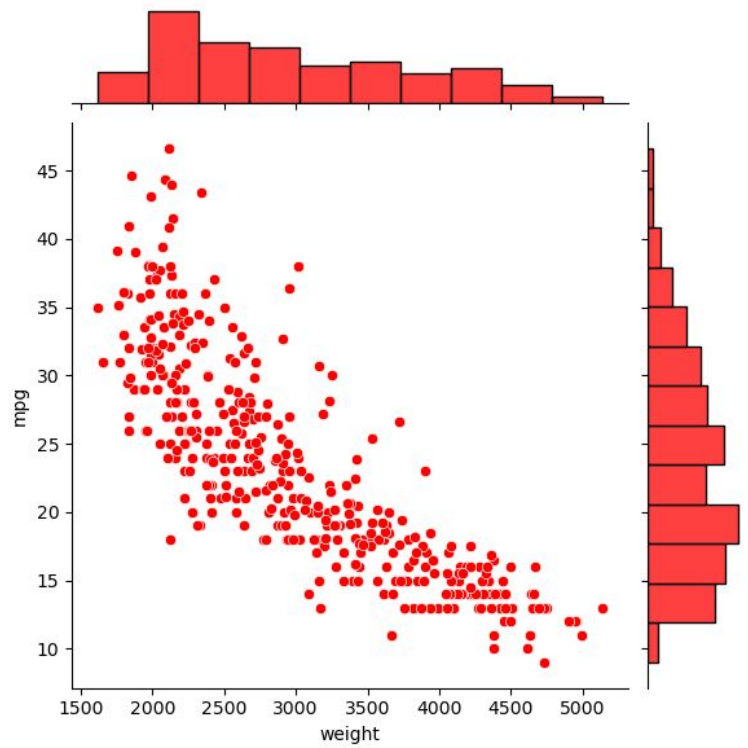
- KDE (Estimación de la densidad del kernel):

[8]:
```
sns.kdeplot(dat.mpg, color='blue')
plt.show()
```



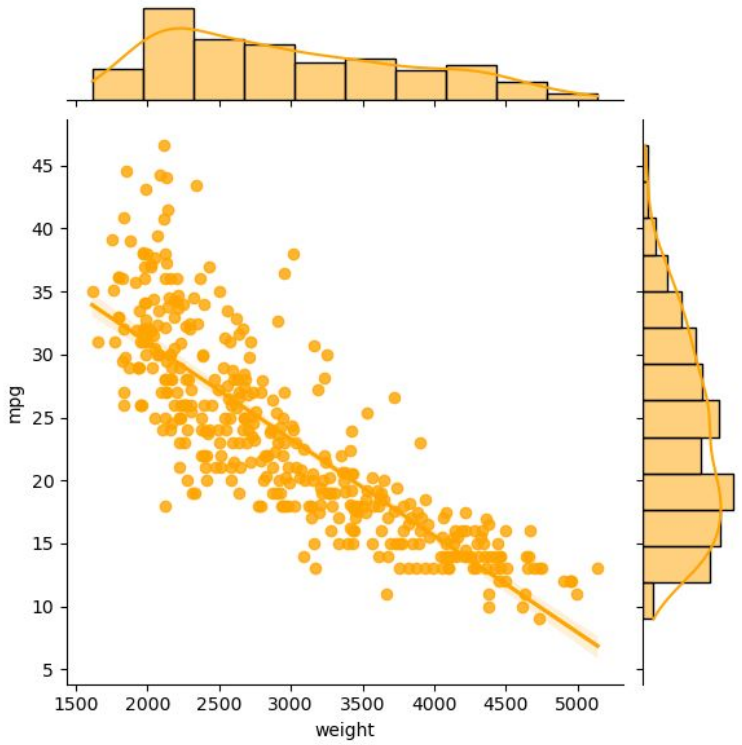
- Diagrama de dispersión:

```
[9]: sns.jointplot(x='weight', y='mpg', data=dat, color='red', kind='scatter')
     plt.show()
```



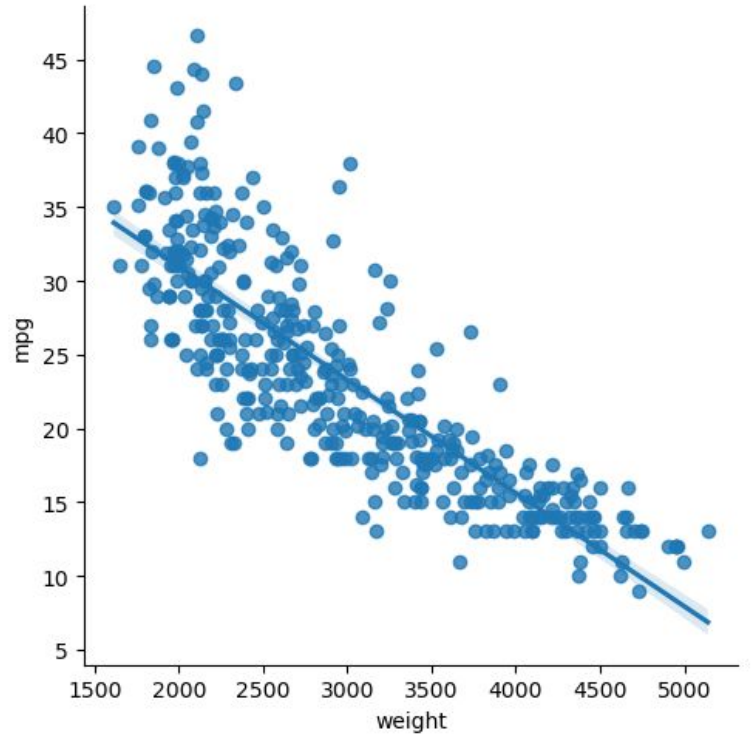
- Diagrama de dispersión + línea de regresión:

```
[10]: sns.jointplot(x='weight', y='mpg', data=dat, color='orange', kind='reg')
      plt.show()
```



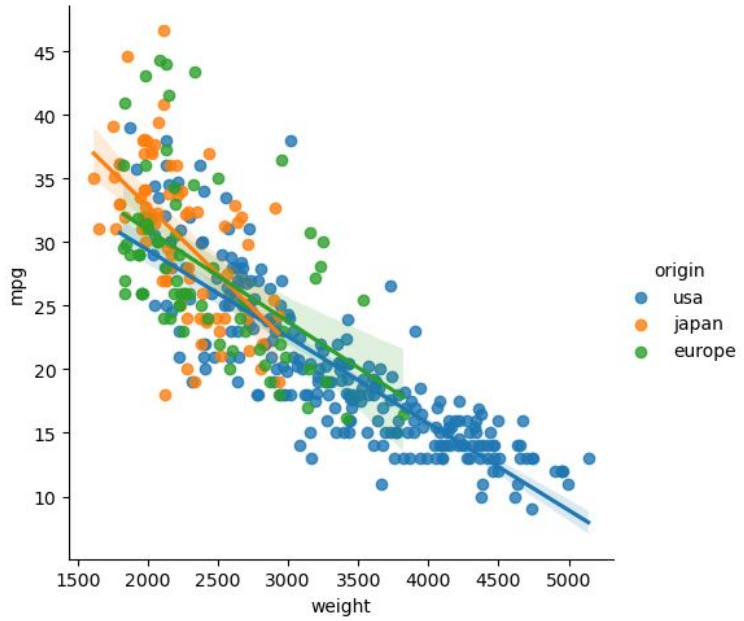
- Diagrama de dispersión + línea de regresión:

[11]:
```
sns.lmplot(data=dat, x='weight', y='mpg')
plt.show()
```



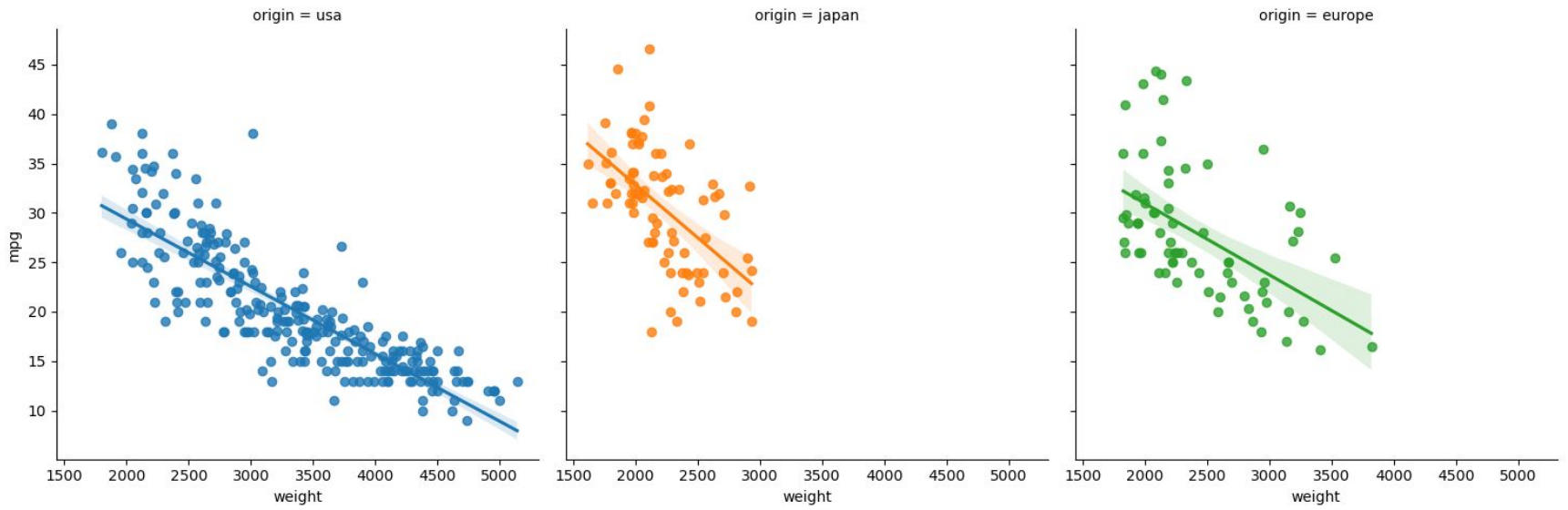
- Gráfico de dispersión + línea de regresión (usando el argumento 'matiz'):

```
[12]: sns.lmplot(data=dat, x='weight', y='mpg', hue='origin')
      plt.show()
```



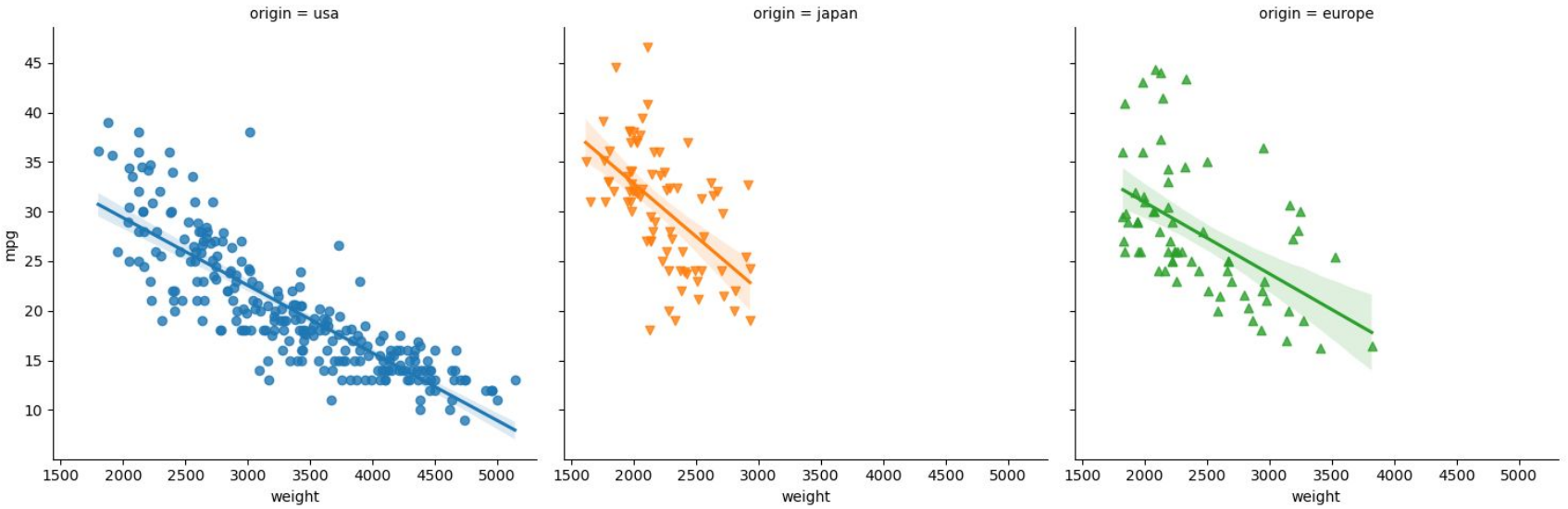
- Gráfico de dispersión + línea de regresión (múltiples gráficos):

```
[13]: sns.lmplot(data=dat, x='weight', y='mpg', col='origin', hue='origin')
      plt.show()
```



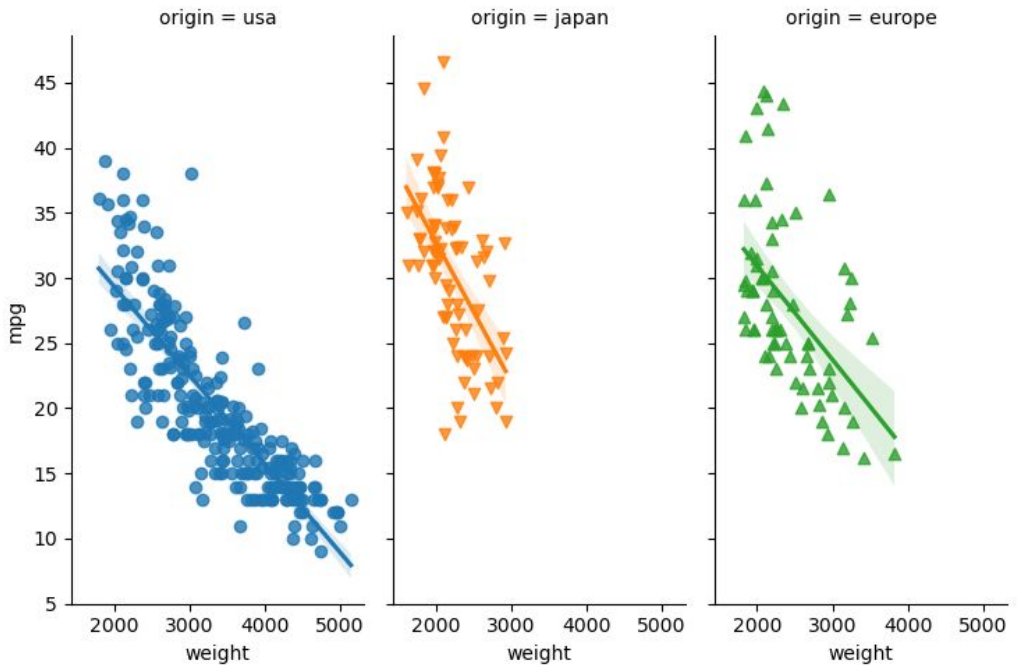
- Gráfico de dispersión + línea de regresión (múltiples gráficos):

```
[14]: sns.lmplot(data=dat, x='weight', y='mpg', col='origin', hue='origin', markers=['o', 'v', '^'])
      plt.show()
```



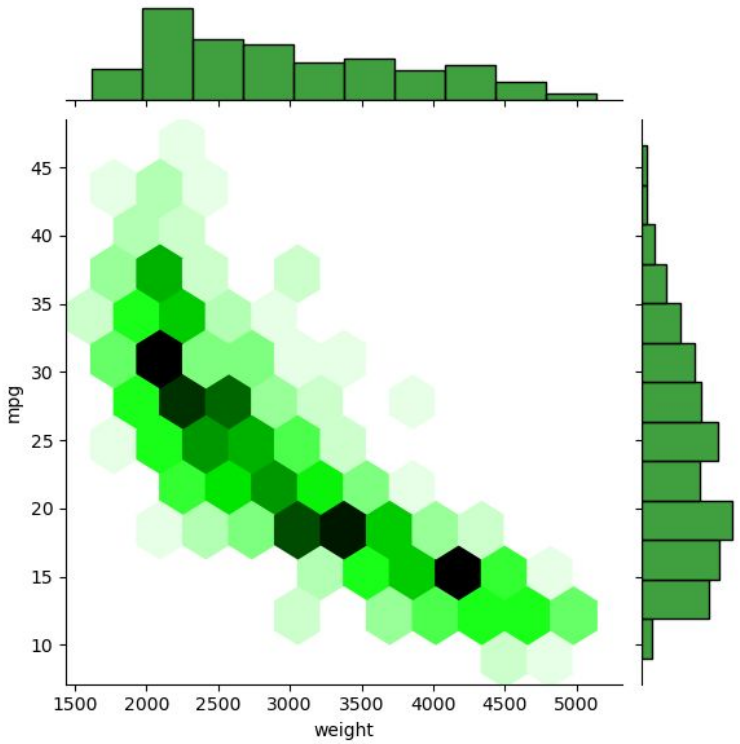
- Gráfico de dispersión + línea de regresión (múltiples gráficos):

```
[15]: sns.lmplot(data=dat, x='weight', y='mpg', col='origin', hue='origin', markers=['o', 'v', '^'],
                 aspect=0.5)
      plt.show()
```



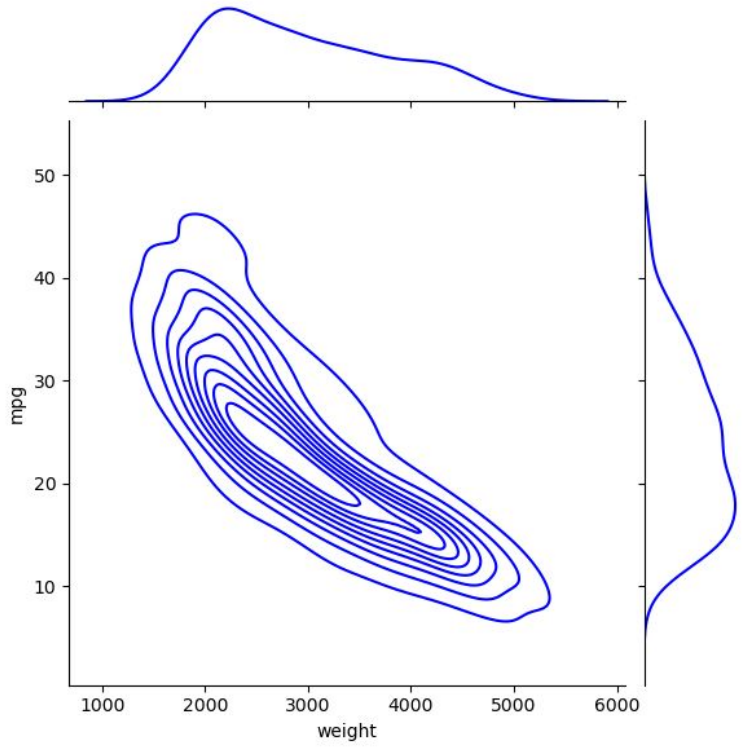
- Maleficio:

```
[16]: sns.jointplot(x='weight', y='mpg', data=dat, color='green', kind='hex')
      plt.show()
```



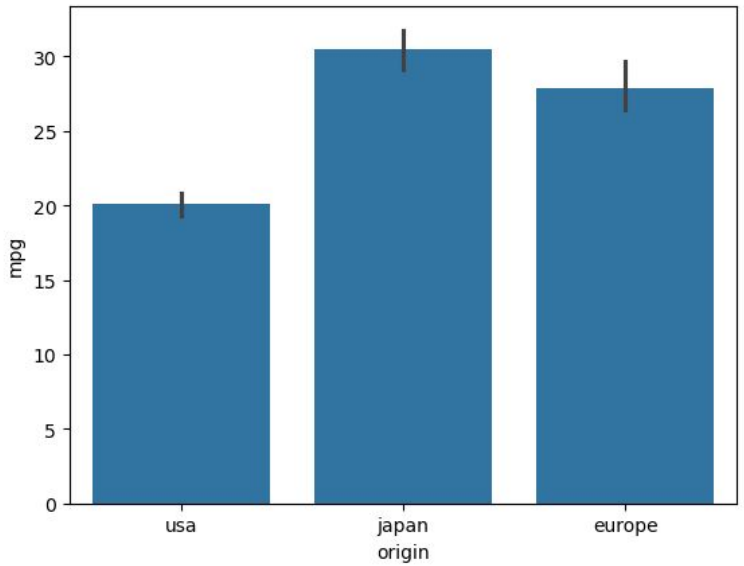
- KDE (Estimación de la densidad del kernel):

```
[17]: sns.jointplot(x='weight', y='mpg', data=dat, color='blue', kind='kde')
      plt.show()
```



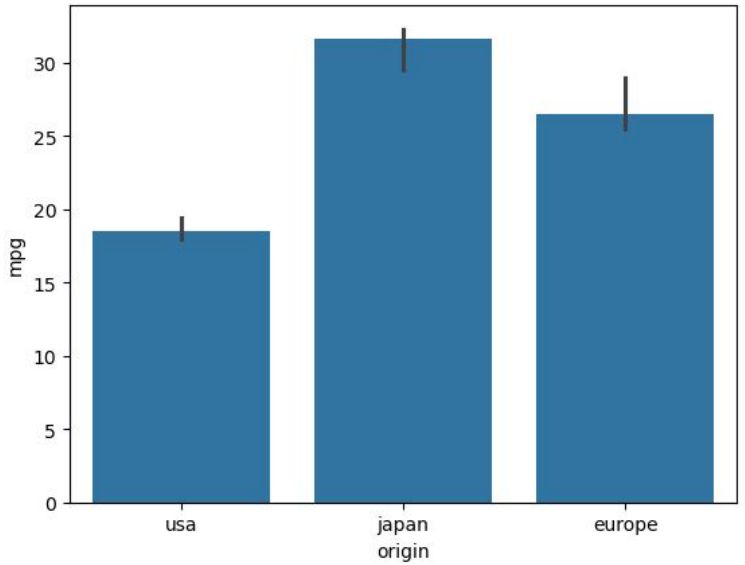
- Una variable categórica + una variable numérica. Agregado por la media.

[18]:
```
sns.barplot(x='origin', y='mpg', data=dat)
plt.show()
```



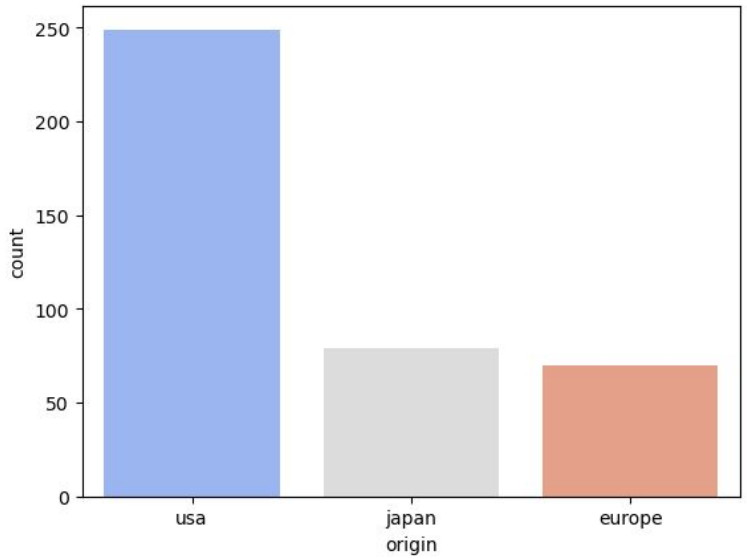
- Una variable categórica + una variable numérica. Agregado por la mediana.

```
[19]: sns.barplot(x='origin', y='mpg', data=dat, estimator=np.median)
      plt.show()
```



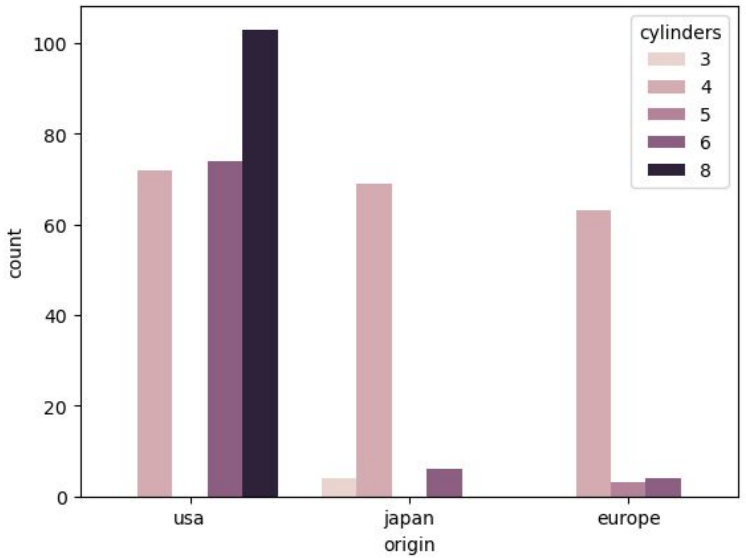
- Una sola variable categórica. La tabla de frecuencias se calcula implícitamente.

[20]:
```
sns.countplot(x='origin', data=dat, palette='coolwarm')
plt.show()
```



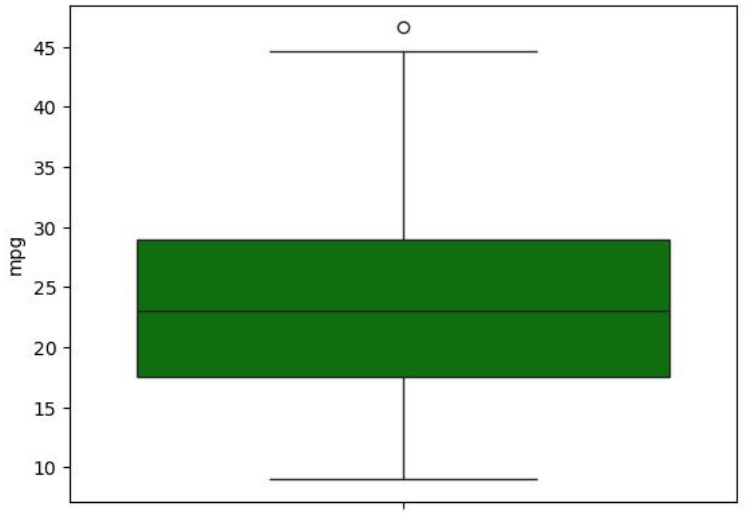
- Dos variables categóricas. Uso del argumento 'matiz'.

[21]:
```
sns.countplot(x='origin', data=dat, hue='cylinders')
plt.show()
```



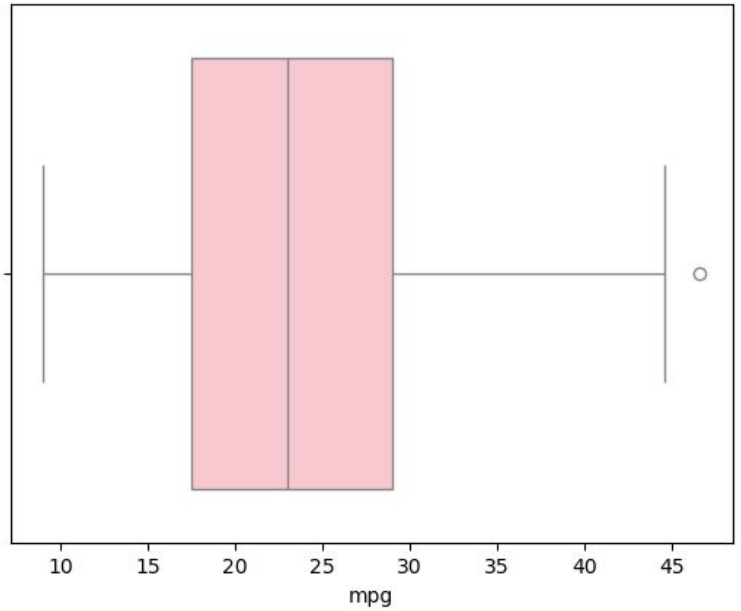
- Diagrama de caja horizontal:

```
[22]: sns.boxplot(y='mpg', data=dat, color='green')
      plt.show()
```



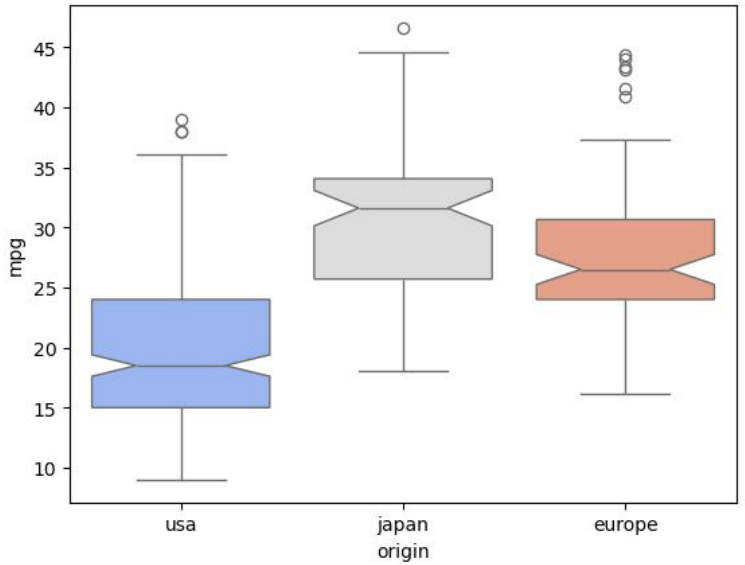
- Diagrama de caja vertical:

```
[23]: sns.boxplot(x='mpg', data=dat, color='pink')
      plt.show()
```



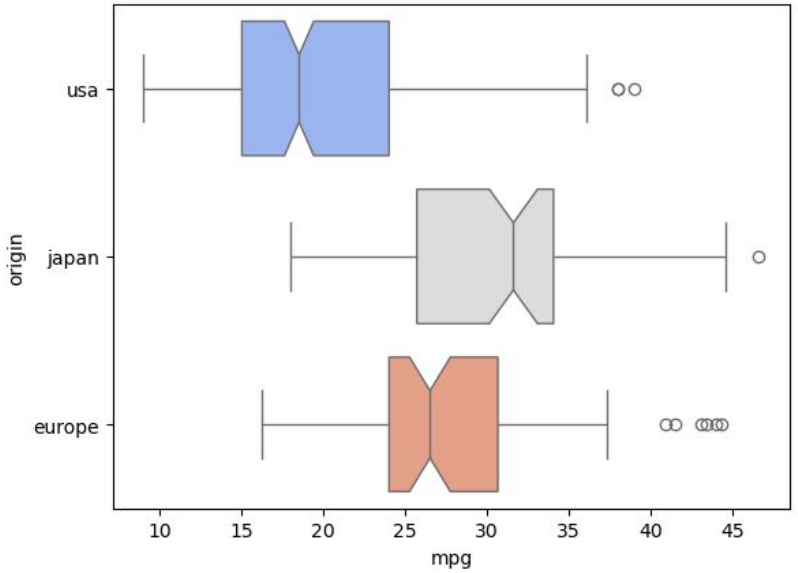
- Múltiples diagramas de caja:

```
[24]: sns.boxplot(x='origin', y='mpg', data=dat, palette='coolwarm', notch=True)
      plt.show()
```



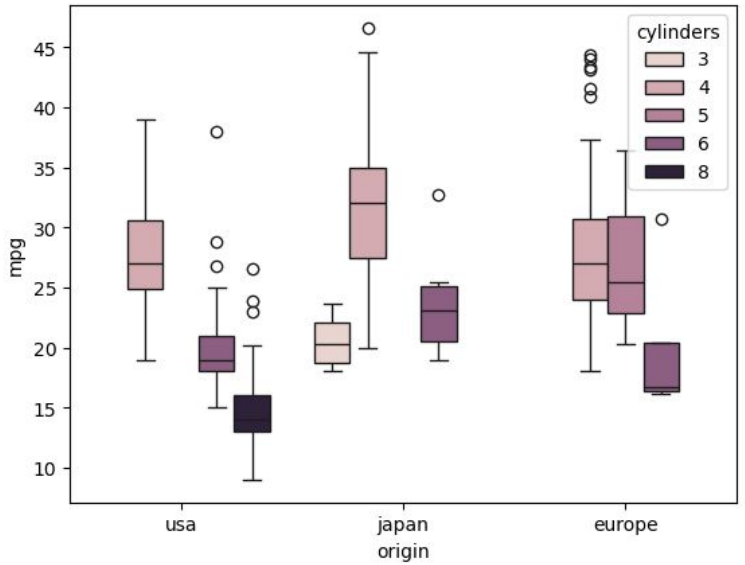
- Múltiples diagramas de caja:

```
[25]: sns.boxplot(x='mpg', y='origin', data=dat, palette='coolwarm', notch=True)
      plt.show()
```



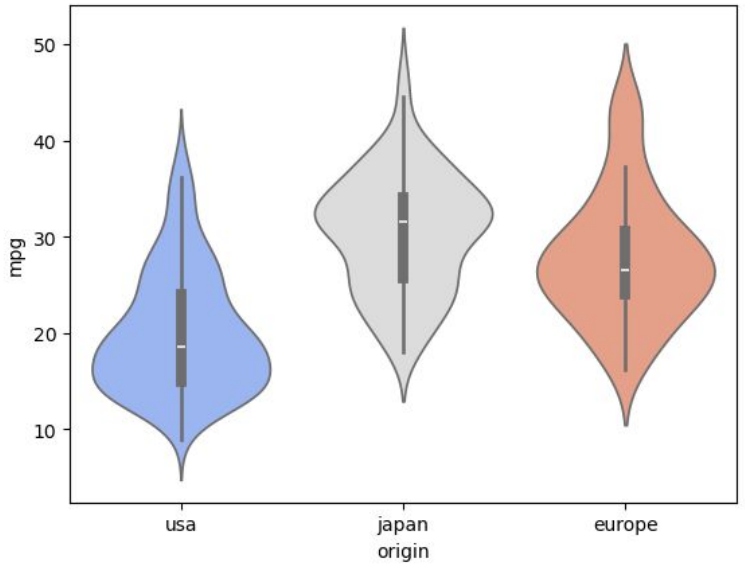
- Múltiples diagramas de caja. Dos variables categóricas + una variable numérica. Uso del argumento 'matiz'.

[26]:
```
sns.boxplot(x='origin', y='mpg', data=dat, hue='cylinders')
plt.show()
```



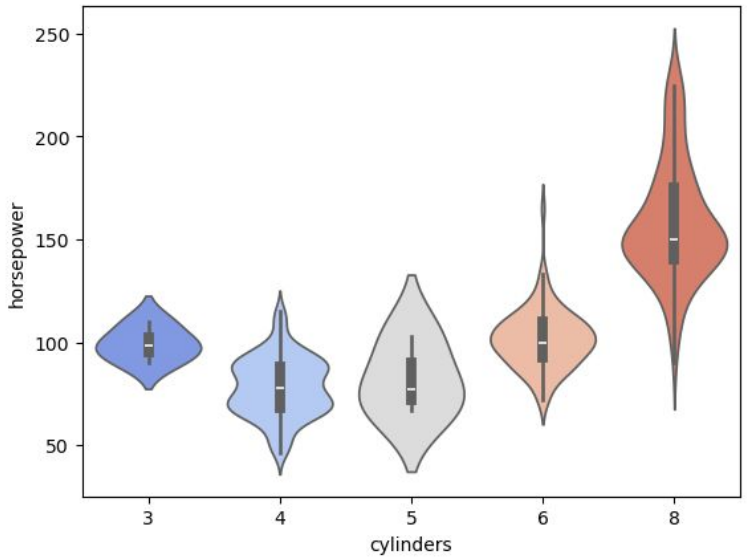
- Múltiples tramas de violín:

```
[27]: sns.violinplot(x='origin', y='mpg', data=dat, palette='coolwarm')
      plt.show()
```



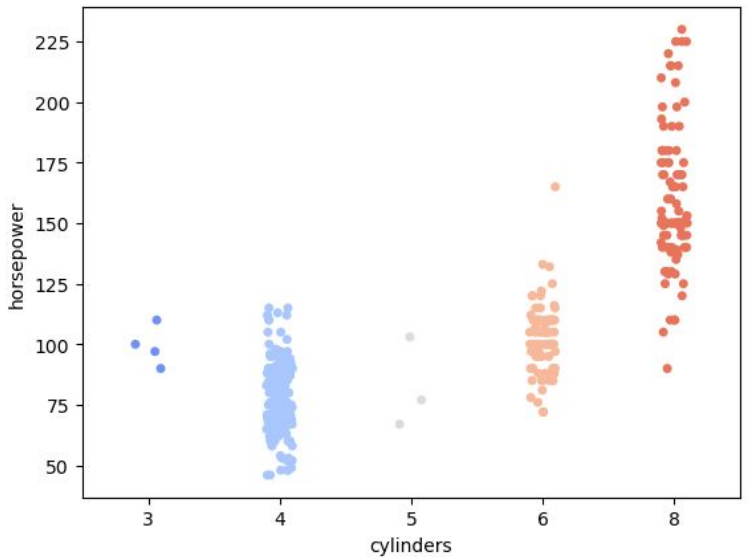
- Múltiples tramas de violín:

[28]:
```
sns.violinplot(x='cylinders', y='horsepower', data=dat, palette='coolwarm')
plt.show()
```



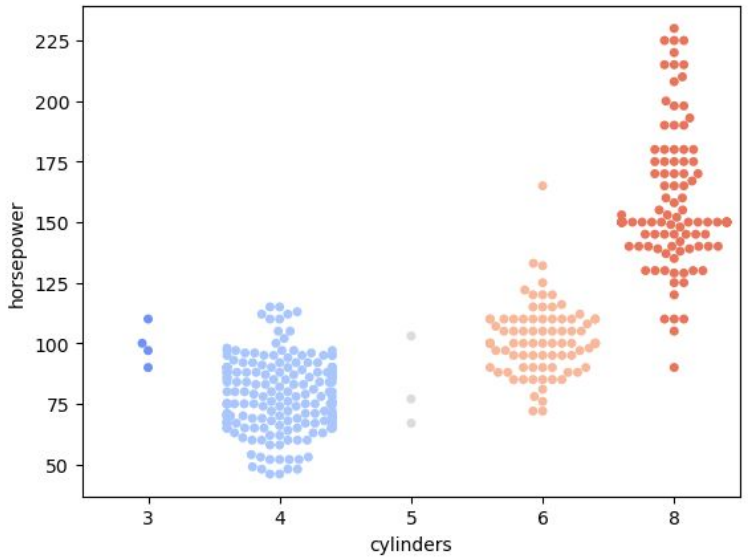
- Parcela de tira:

[29]:
```
sns.stripplot(x='cylinders', y='horsepower', data=dat, palette='coolwarm')
plt.show()
```



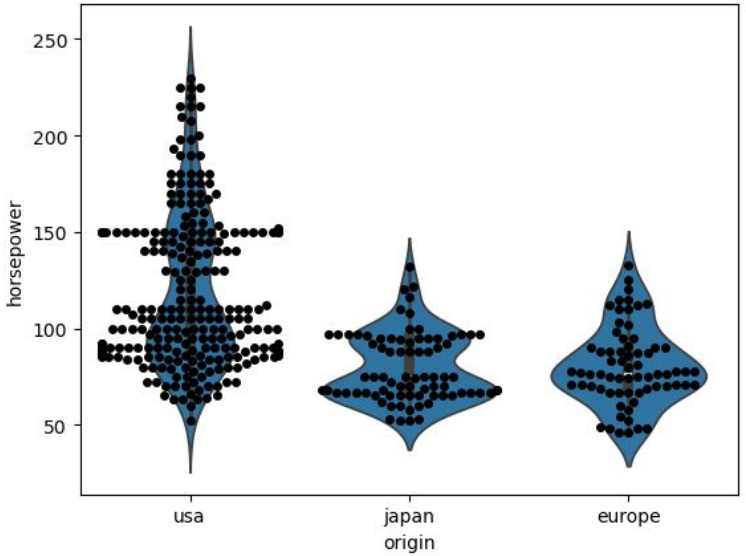
- Trama de enjambre:

```
[30]: sns.swarmplot(x='cylinders', y='horsepower', data=dat, palette='coolwarm')
      plt.show()
```



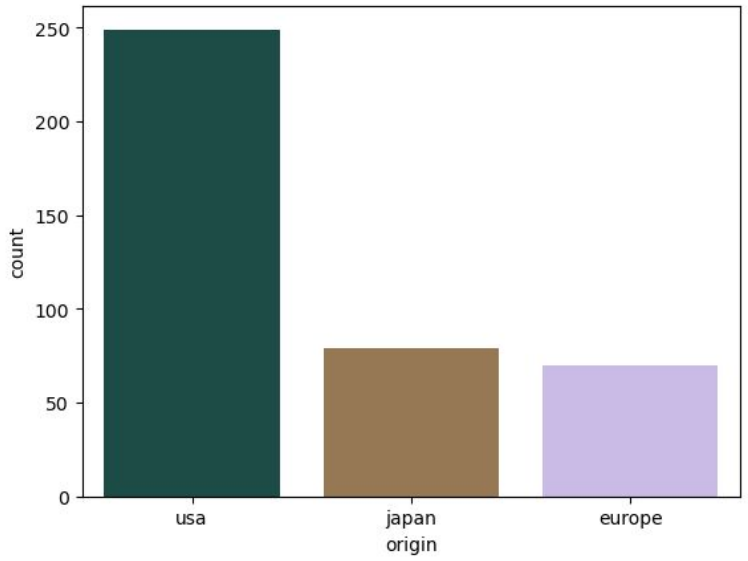
- Trama de violín + Trama de enjambre:

[31]:
```
sns.violinplot(x='origin', y='horsepower', data=dat)
sns.swarmplot(x='origin', y='horsepower', data=dat, color='black')
plt.show()
```



- Paleta de colores (la tabla de frecuencias se muestra como un diagrama de barras):

[32]:
```
sns.countplot(x='origin', data=dat, palette='cubehelix')
# sns.countplot(x='origin', data=dat, palette='coolwarm')
# sns.countplot(x='origin', data=dat, palette='magma')
# sns.countplot(x='origin', data=dat, palette='seismic')
plt.show()
```



Visualización Seaborn library (con más ejemplos)

- Importar módulos.

[1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import os
import warnings
%matplotlib inline
warnings.filterwarnings('ignore')
```

- Diagrama de dispersión Array:

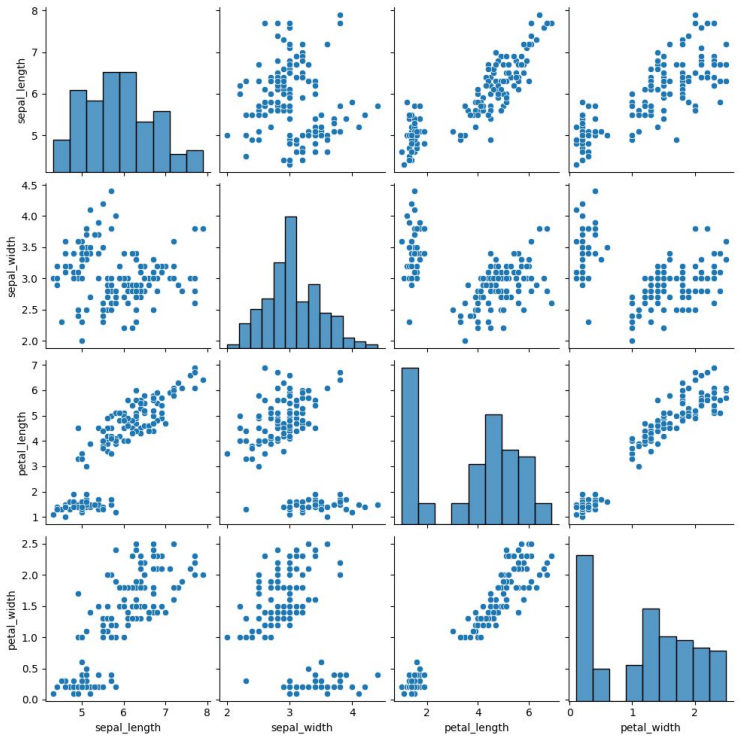
```
[2]: dat = sns.load_dataset('iris')
```

Línea 2

- Lea los datos del 'iris' del library.

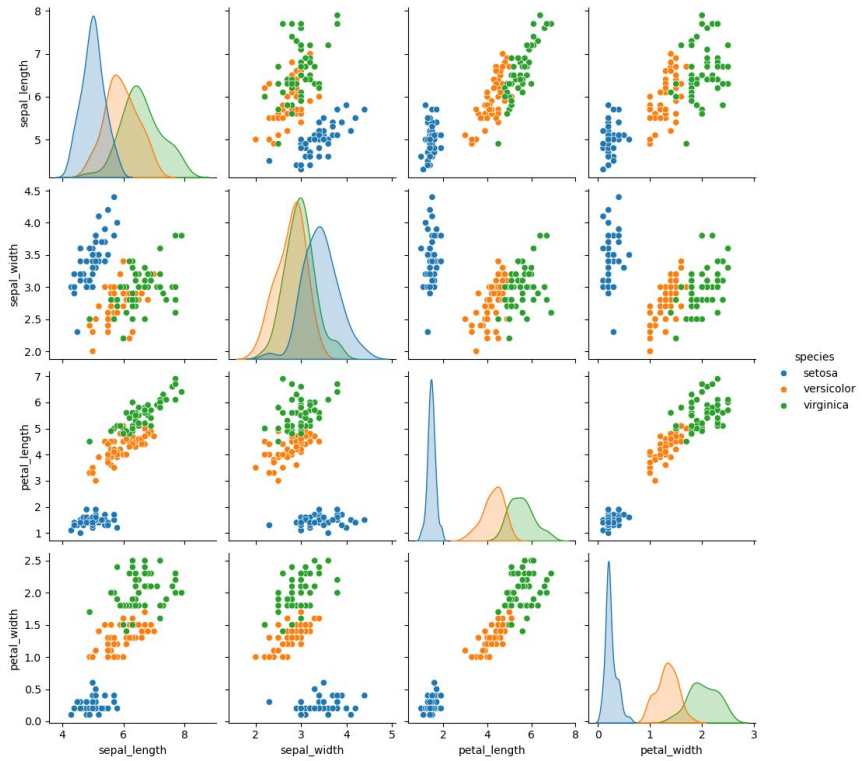
- Diagrama de dispersión Array:

```
[3]: sns.pairplot(dat)
     plt.show()
```



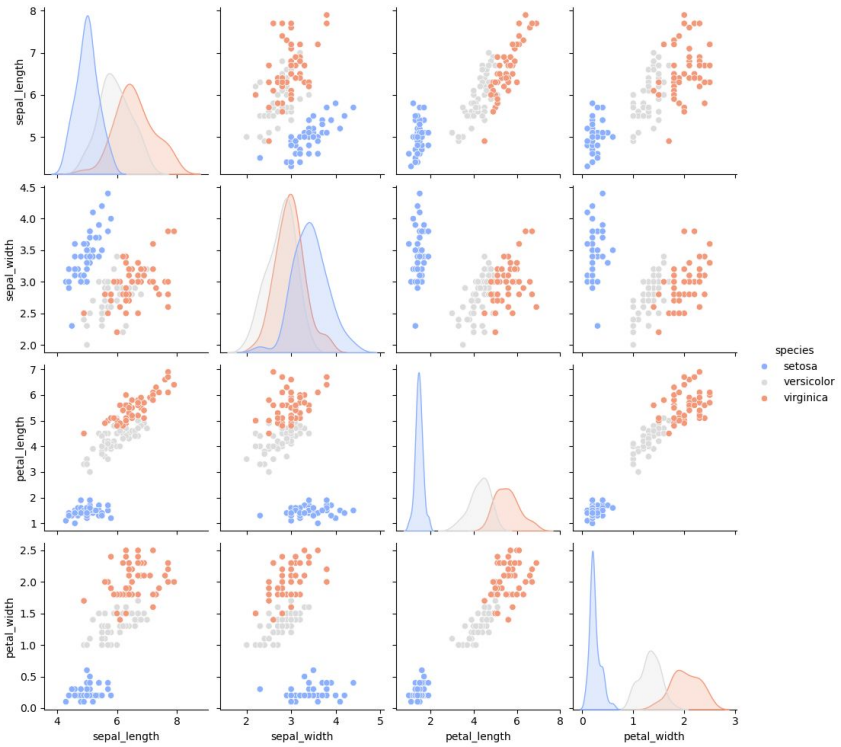
- Diagrama de dispersión Array (Colorea los marcadores con 'especie'):

[4]:
```
sns.pairplot(dat, hue='species')
plt.show()
```



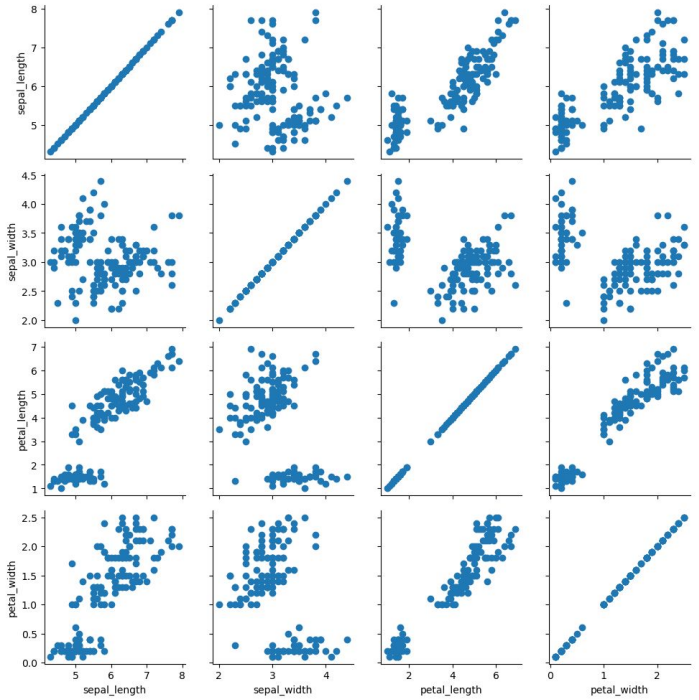
- Diagrama de dispersión Array (Colorea los marcadores con 'especie'. Aplicar paleta.):

[5]:
```
sns.pairplot(dat, hue='species', palette='coolwarm')
plt.show()
```



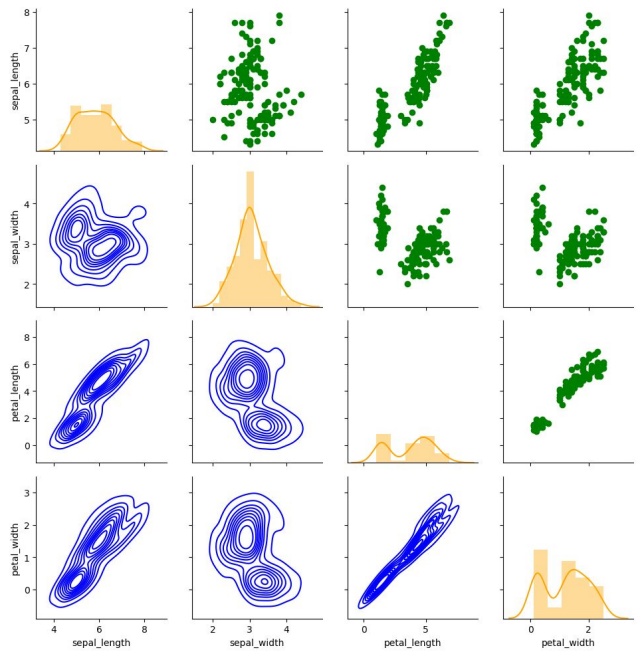
- PairGrid (Un Array de diagramas de dispersión):

[6]:
```
g = sns.PairGrid(dat)
g.map(plt.scatter)
plt.show()
```



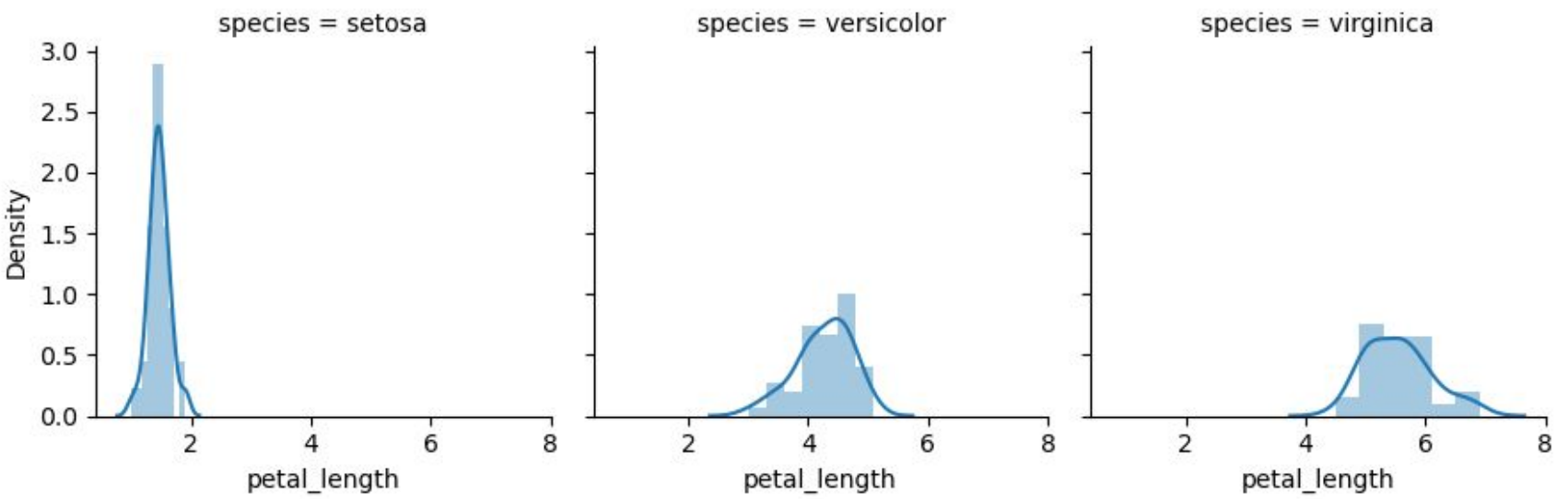
- PairGrid (Un Array de diferentes tipos de visualización):

[7]:
```
g = sns.PairGrid(dat)
g.map_diag(sns.distplot, color='orange')    # Diagonal = histogram.
g.map_upper(plt.scatter, color='green')     # Upper triangle = scatter plot.
g.map_lower(sns.kdeplot, color='blue')      # Lower triangle = KDE.
plt.show()
```



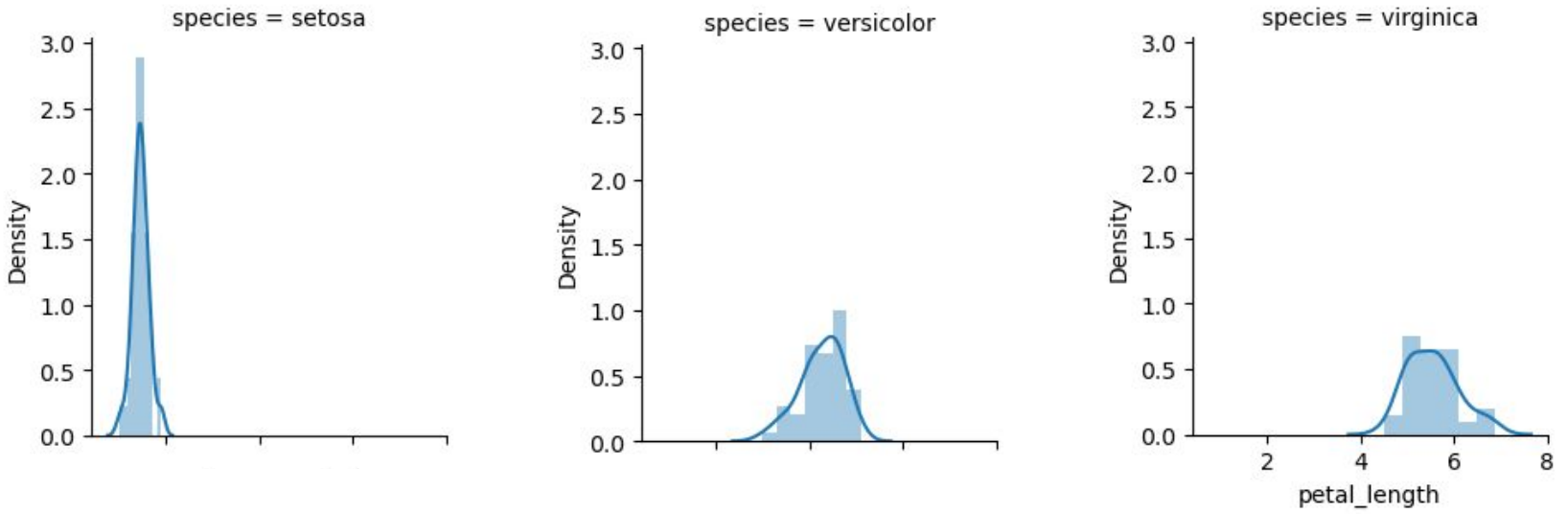
- FacetGrid (múltiples histogramas):

[8]:
```
g = sns.FacetGrid(dat, col='species')
g.map(sns.distplot, 'petal_length')
plt.show()
```



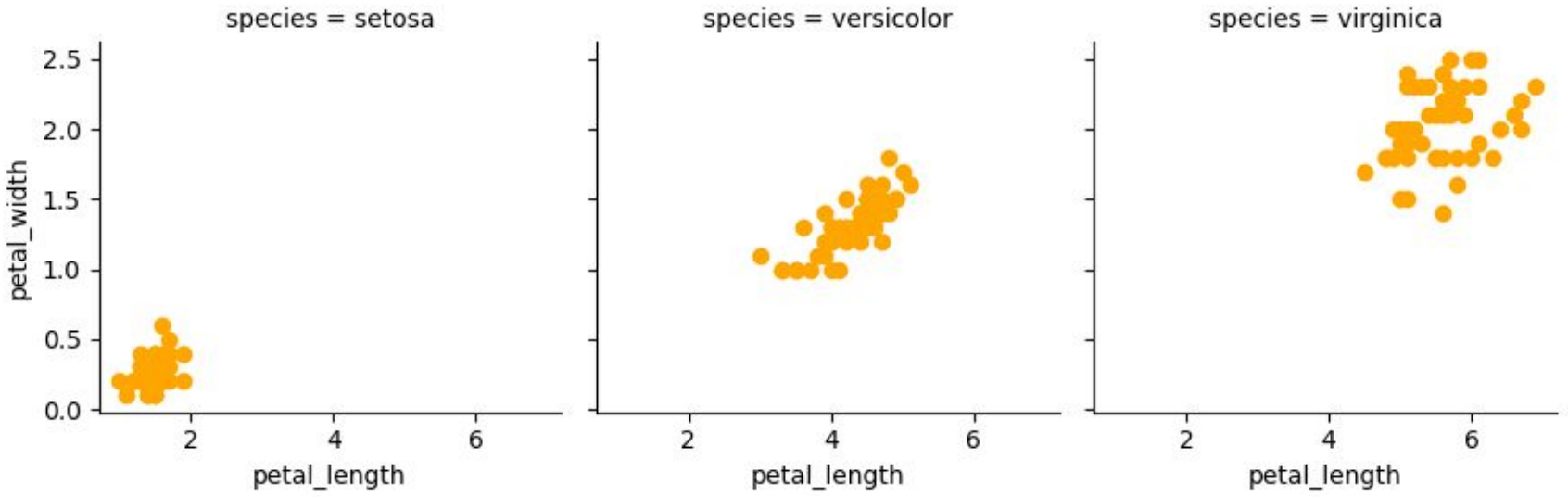
- FacetGrid (múltiples histogramas):

[9]:
```
g = sns.FacetGrid(dat, row='species')
g.map(sns.distplot, 'petal_length')
plt.show()
```



- FacetGrid (múltiples diagramas de dispersión):

[10]:
```
g = sns.FacetGrid(dat, col='species')
g.map(plt.scatter, 'petal_length', 'petal_width', color='orange')
plt.show()
```



- Mapa de calor:

```
[11]: dat = sns.load_dataset('mpg')
      dat.drop(columns=['origin', 'name'], inplace=True)
```

Línea 11

- Lea los datos de 'mpg' del library.

- Mapa de calor (matriz de correlación):

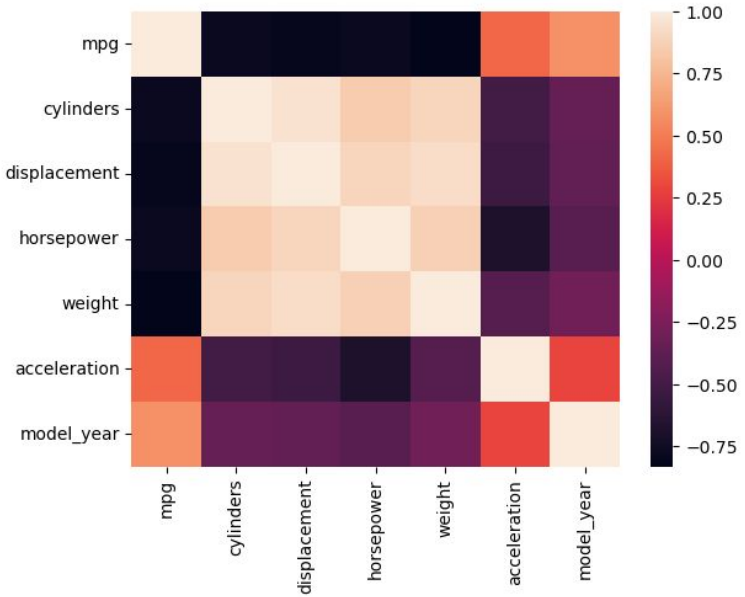
[12]:
```
x = dat.corr()
x
```

[12]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year
mpg	1.000000	-0.775396	-0.804203	-0.778427	-0.831741	0.420289	0.579267
cylinders	-0.775396	1.000000	0.950721	0.842983	0.896017	-0.505419	-0.348746
displacement	-0.804203	0.950721	1.000000	0.897257	0.932824	-0.543684	-0.370164
horsepower	-0.778427	0.842983	0.897257	1.000000	0.864538	-0.689196	-0.416361
weight	-0.831741	0.896017	0.932824	0.864538	1.000000	-0.417457	-0.306564
acceleration	0.420289	-0.505419	-0.543684	-0.689196	-0.417457	1.000000	0.288137
model_year	0.579267	-0.348746	-0.370164	-0.416361	-0.306564	0.288137	1.000000

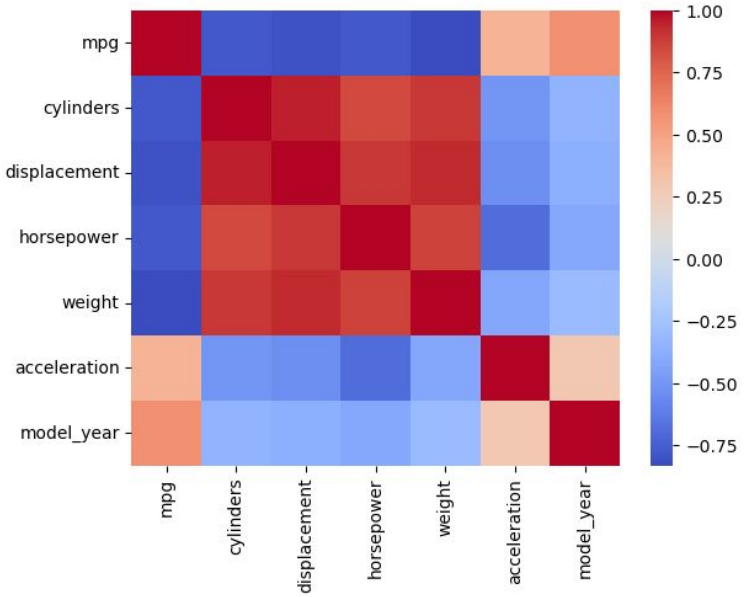
- Mapa de calor (mapa de calor predeterminado):):

[13]:
```
sns.heatmap(x)
plt.show()
```



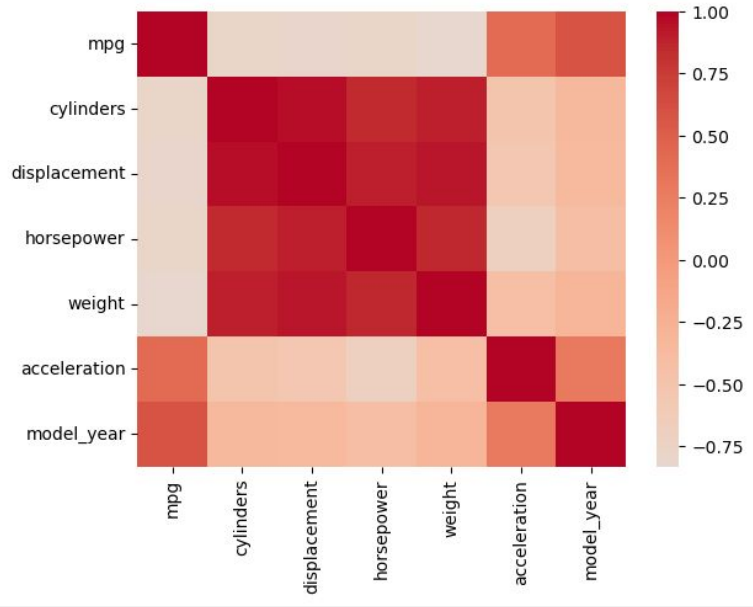
- Mapa de calor (use 'cmap' en lugar de 'paleta'):

[14]:
```
sns.heatmap(x, cmap='coolwarm')
plt.show()
```



- Mapa de calor (Ajustar el centro.):

```
[15]: sns.heatmap(x, cmap='coolwarm', center=-1)
      plt.show()
```



Ejercicio de codificación n.° 0113

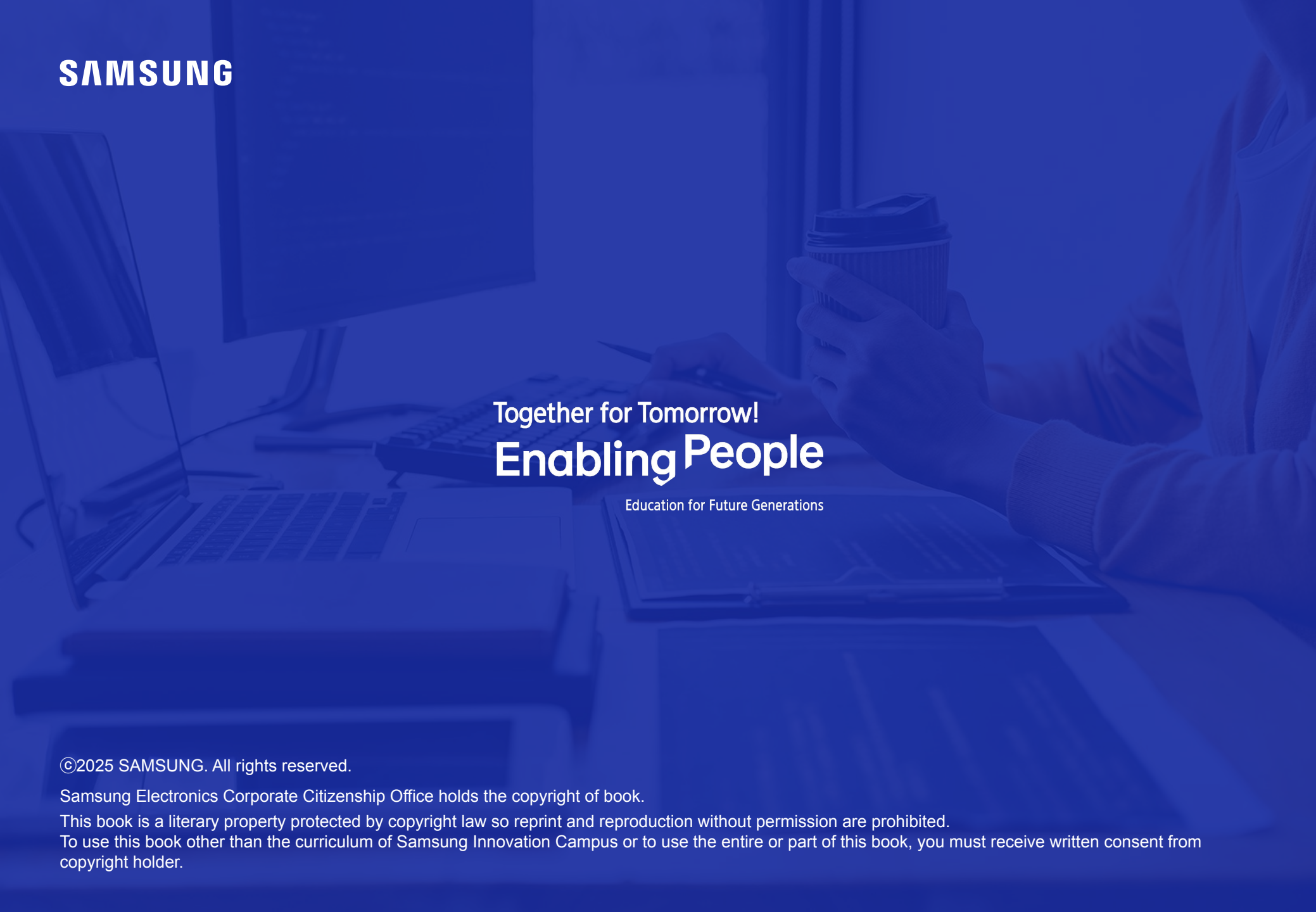


Siga los pasos de práctica en el archivo 'ex_0113.ipynb'



Fin de Documento

SAMSUNG



ⓒ2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of book.

This book is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this book other than the curriculum of Samsung Innovation Campus or to use the entire or part of this book, you must receive written consent from copyright holder.